UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI
DEPARTMENT OF INFORMATION AND COMMUNICATION TECHNOLOGY



BACHELOR THESIS

By
Phung Dam Tien Si - 23BI14384

Title:
Design and Implementation of a Full-Stack Web-Based Recruitment Platform for Students and Employers

External Supervisor: Eng. Dinh Viet Ha

Internal Supervisor: Prof. Dr. Giang Anh Tuan

Hanoi, October - 2026

To whom it may concern,

I, Dinh Viet Ha, certify that the thesis/ internship report of Mr. Phung Dam Tien Si is qualified to be presented in the Internship Jury 2025-2026.

Hanoi, October 25, 2026

Supervisor's signature

Acknowledgements

I would like to express my sincere gratitude to Mr. Dinh Viet Ha for his guidance, patience and mentorship throughout my internship. His support have greatly contributed to the completion of this project and my professional development.

I would also like to extend my sincere appreciation to SAVA META Technology Joint Stock Company for providing me with the opportunity to undertake my internship in a professional and supportive working environment.

Finally, I would like to thank everyone who has supported, encouraged, and accompanied me throughout my academic journey and personal development. Their encouragement has been a constant source of motivation during the completion of this thesis.

Contents

1 Introduction **1**
- 1.1 Background . . . 1
- 1.2 Problem Statement . . . 1
- 1.3 Aim, Objectives and Scope . . . 2
- 1.4 Methodology . . . 2
- 1.5 Thesis Organization . . . 2

2 System Analysis **3**
- 2.1 Proposed System . . . 3
- 2.2 Functional Requirement . . . 4
- 2.3 Non-Functional Requirement . . . 4
- 2.4 Use Case Analysis . . . 5
  - 2.4.1 Shared Use Cases . . . 5
  - 2.4.2 Specialized Use Cases . . . 6
- 2.5 Use Case Specification . . . 9
  - 2.5.1 Job Seeker: Apply for Job . . . 10
  - 2.5.2 Employer: Publish Job Posting . . . 12
  - 2.5.3 Employer: Review Application . . . 14

3 System Design **16**
- 3.1 Design Overview . . . 16
- 3.2 System Architecture . . . 17
  - 3.2.1 System Context . . . 17
  - 3.2.2 Container View . . . 17
  - 3.2.3 Component View: Frontend . . . 19
  - 3.2.4 Component View: Backend . . . 20
- 3.3 Database Design . . . 21
  - 3.3.1 Entity Relationship Overview . . . 21
  - 3.3.2 Company and Job Posting . . . 21
  - 3.3.3 Job Application . . . 22
- 3.4 Backend Design . . . 23
  - 3.4.1 Backend Project Structure . . . 24
  - 3.4.2 Shared Backend Components . . . 24
  - 3.4.3 Job Application Module . . . 25
  - 3.4.4 Employer Job Management Module . . . 26
- 3.5 Frontend Design . . . 29
  - 3.5.1 Frontend Structure and Shared Components . . . 29
  - 3.5.2 Job Application UI . . . 29

3.5.3 Employer Job Management UI . . . 29
3.6 Summary . . . 30

4 Implementation **31**
4.1 Overview . . . 31
4.1.1 Request Pipeline . . . 31
4.1.2 Error Handling . . . 32
4.2 Backend Implementation . . . 32
4.2.1 Job Application Module . . . 32
4.2.2 Employer Job Management Module . . . 33
4.2.3 Remaining Modules . . . 36
4.3 Frontend Implementation . . . 36
4.3.1 Job Application UI . . . 36
4.3.2 Employer Job Management UI . . . 37
4.3.3 Remaining Modules . . . 37
4.4 Summary . . . 37

5 Testing **39**
5.1 Testing Strategy . . . 39
5.2 Test Environment . . . 39
5.3 Requirements Traceability . . . 40
5.4 Results . . . 44
5.5 Limitations . . . 45

6 Conclusion **46**
6.1 Summary . . . 46
6.2 Limitations . . . 46
6.3 Future Work . . . 47

A Supplementary Use Case Specifications **52**
A.1 Guest: Login . . . 52
A.2 Authenticated User: Update Personal Information . . . 54
A.3 Back Office Worker: Approve Job Posting . . . 55

B Supplementary Entity Relationship Groups **56**
B.1 Auth and Account . . . 56
B.2 Profile and CV Management . . . 57
B.3 Administration and Audit . . . 57

C Supplementary Module Designs **59**
C.1 Authentication Module . . . 59
C.2 Profile Management Module . . . 61
C.3 Resume Management Module . . . 61
C.4 Job Discovery Module . . . 62
C.5 Back Office Module . . . 63
C.5.1 Back Office Worker UI . . . 64

D Sequence Diagrams **66**
D.1 Job Application 68
D.2 Employer Job Management 71
D.3 Authentication and Profile 77
D.4 Public 81
D.5 Back Office 83

E Domain Enumerations **87**
E.1 JobStatus 87
E.2 ApplicationStatus 87

List of Figures

2.1 Guest Use Case Diagram. 6
2.2 Authenticated User Use Case Diagram. 6
2.3 Job Seeker Use Case Diagram. 7
2.4 Employer Use Case Diagram. 8
2.5 Back Office Worker Use Case Diagram. 9

3.1 System Context Diagram. 17
3.2 Container Diagram. 18
3.3 Frontend Component Diagram. 19
3.4 Backend Component Diagram. 20
3.5 Entity Relationship Diagram. 21
3.6 Company and Job Posting Entity Relationship Diagram. 22
3.7 Job Application Entity Relationship Diagram. 23
3.8 Job Application Module Class Diagram. 26
3.9 Company Profile Management Class Diagram. 27
3.10 Job Posting Management Class Diagram. 27
3.11 Employer Application Management Class Diagram. 28

4.1 Job Posting Status Transitions. 35

B.1 Auth and Account Entity Relationship Diagram. 56
B.2 Profile and CV Management Entity Relationship Diagram. 57
B.3 Administration and Audit Entity Relationship Diagram. 58

C.1 Authentication Module Class Diagram. 60
C.2 Profile Management Module Class Diagram. 61
C.3 Resume Management Module Class Diagram. 62
C.4 Public Class Diagram. 63
C.5 Job Moderation Module Class Diagram. 63
C.6 User Management Module Class Diagram. 64
C.7 Audit Log Module Class Diagram. 64

D.1 Sequence Diagram: Apply for Jobs. 68
D.2 Sequence Diagram: Track Applications. 69
D.3 Sequence Diagram: Cancel Applications. 70
D.4 Sequence Diagram: Create Company Profile. 71
D.5 Sequence Diagram: Create Job Posting. 72
D.6 Sequence Diagram: Edit Job Posting. 73
D.7 Sequence Diagram: Close Job Posting. 74
D.8 Sequence Diagram: Delete Job Posting. 75

D.9 Sequence Diagram: Review Application. 76
D.10 Sequence Diagram: Register. 77
D.11 Sequence Diagram: Login. 77
D.12 Sequence Diagram: Forgot Password. 78
D.13 Sequence Diagram: Update Personal Information. 78
D.14 Sequence Diagram: Update Job Preferences. 79
D.15 Sequence Diagram: Change Avatar. 79
D.16 Sequence Diagram: Change Password. 80
D.17 Sequence Diagram: Upload CV. 81
D.18 Sequence Diagram: Search Jobs. 81
D.19 Sequence Diagram: View Company Profile. 82
D.20 Sequence Diagram: Get Job Postings List. 83
D.21 Sequence Diagram: Get Job Posting Detail. 83
D.22 Sequence Diagram: Approve Job. 84
D.23 Sequence Diagram: Reject Job. 85
D.24 Sequence Diagram: Delete Job. 86

List of Tables

2.1 Summary of Functional Requirements. 4
2.2 Summary of Non-Functional Requirements. 5
2.3 Use Case Specification: Apply for Job. 10
2.4 Use Case Specification: Publish Job Posting. 12
2.5 Use Case Specification: Review Application. 14

3.1 Design Principles and Rationale. 16
3.2 Backend Project Structure. 24

4.1 HTTP Status Code Conventions. 32
4.2 Job Application Endpoints. 32
4.3 Business Rules Enforced by `applyJob`. 33
4.4 Business Rules Enforced by `withdrawApplication`. 33
4.5 Company Profile Endpoints. 34
4.6 Business Rules Enforced by the Company Profile Endpoints. 34
4.7 Job Posting Endpoints. 34
4.8 Input Validation Rules for Creating or Editing a Job Posting. 35
4.9 Employer Application Review Endpoints. 36
4.10 Apply Page Error Display. 37

5.1 Requirements Traceability Matrix. 40
5.2 Test Execution Summary. 44

A.1 Use Case Specification: Login. 52
A.2 Use Case Specification: Update Personal Information. 54
A.3 Use Case Specification: Approve Job Posting. 55

Abbreviations

API	Application Programming Interface
CV	Curriculum Vitae
DTO	Data Transfer Object
ERD	Entity Relationship Diagram
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
JSON	JavaScript Object Notation
JWT	JSON Web Token
ORM	Object-Relational Mapping
REST	Representational State Transfer
SDK	Software Development Kit
SPA	Single-Page Application
SQL	Structured Query Language
UI	User Interface
UML	Unified Modeling Language

Chapter 1

Introduction

1.1 Background

Traditionally, job seekers and employers found each other through offline channels such as personal referrals, direct contact with employers, private employment agencies, and newspaper advertisements [1], and applications were handled through traditional, largely paper-based methods [2].
With the rapid growth of Internet technologies, the recruitment process has transformed as online recruitment platforms appear: employers now publish vacancies on job boards and corporate websites, and candidates apply electronically [1, 2]. Case studies of large organizations found that e-recruitment turns the traditional recruitment process into a time- and space-independent one, in which communicating with candidates becomes a central task [3]. The effect is also measurable from the job seeker's side: unemployed workers who search for jobs online are re-employed about 25% faster than comparable workers who do not [4].

The scale at which this transformation now operates is considerable. Vietnam's labor force numbered approximately 52.9 million people in 2024 [5], and platforms built to match workers to jobs serve a comparable share of it: TopCV alone reports over 8 million candidate profiles and more than 540,000 registered employers [6]. Yet scale has not resolved the underlying friction. A nationwide survey of 996 Vietnamese professionals found that 35% were actively seeking a new job at the time of the survey, with a further 21% planning to start within six months, and workers reported becoming still more likely to job-hunt when economic conditions worsen [7]. Against this volume of competing job seekers, hiring itself remains slow: a global survey of recruiters found that time-to-hire averages 44 days from an opening being posted to an offer being made [8], longer still for specialized roles. It is this combination - a large, actively searching workforce and a slow, largely manual matching process - that a centralized recruitment platform is positioned to address.

1.2 Problem Statement

The new approach brings its own friction. Each recruitment site has its own search interface, its own resume format, and its own application flow. A candidate who wants to reach more than one pool of jobs must recreate essentially the same profile and resume on every platform, then visit each one separately to check whether an application has progressed. The more platforms a job seeker uses, the more duplicated

effort the search costs.
Employers face similar challenge in reverse. Publishing a posting is only the first step; they still have to collect applications, read resumes, record decisions, and keep those records straight. Without a single place that ties postings, applications, and decisions together, the employer ends up stitching information across tools, recreating the inefficiency the platform was supposed to remove.

1.3 Aim, Objectives and Scope

The thesis aims to analyze the design and implementation of such centralized system: a platform where job seekers search jobs, maintain a profile, upload a resume, apply, and track their applications, and where employers manage company information, publish postings, and review applicants through one interface.
The implemented system consist of several functional modules. This thesis will discuss two modules as its primary focus. The supporting modules are built and described, but only briefly.

1.4 Methodology

Development was driven by use case documentation rather than a single fixed process. The approach borrows the use-case-driven, iterative, and incremental principle of the Unified Process [9], without adopting its full set of phases and artifacts. Work was decomposed into use case groups - profile, resume, job application, employer, and so on - and each group passed through the same small loop. First the use cases of the group were documented with their specifications and diagrams. Then the grouping was implemented, backend first and frontend after. Wherever implementation surfaced a use case that had not been anticipated in the documentation, the loop ran again: the new use case was implemented if it was needed immediately, and was always written back into the documentation so that the analysis in Chapter 2 and the implementation in Chapter 4 would stay consistent with each other.

1.5 Thesis Organization

The remainder of the thesis follows the loop above. Chapter 2 analyzes the requirements: actors, the functional and non-functional requirements, and the use cases of the featured modules. Chapter 3 turns those requirements into design, covering the system architecture, the database schema, and the module-level design. Chapter 4 describes how the design was implemented, and Chapter 5 summarizes verification. Chapter 6 closes with the results and limitations of the project.

Chapter 2

System Analysis

The challenges described in Chapter 1 motivate a centralized online recruitment platform.

2.1 Proposed System

The system's functionality is split into clear modules by business responsibility:

- Job Discovery: allows guests to search active job postings and view company profiles.
- Authentication: covers registration, login, logout, and password recovery.
- Profile Management: lets users maintain personal information, job preferences, avatar, and account password.
- Resume Management: lets job seekers upload and delete resumes used in applications.
- Job Application: lets a job seeker apply to an active posting, track submitted applications, and withdraw them while still eligible.
- Employer Job Management: lets an employer manage a company profile, run the full lifecycle of a job posting, and review the applications a posting receives.
- Administration: supports moderating job postings and managing users and logs.

Because the platform stores personal data and lets users modify shared records, access is controlled. Registration and login protect each user's account, and role-based permissions decide what each user may do.
This thesis develops the platform in full but examines two flows in depth: the Job Application flow on the job seeker's side and the Employer Job Management flow on the employer's side. These two flows form the round trip of a recruitment event - a job seeker submits an application, and an employer reviews it - and are therefore used to illustrate the analysis, design, and implementation in the remaining chapters.

2.2 Functional Requirement

Functional requirements are statements of the services a system should provide, how it should react to particular inputs, and how it should behave in particular situations [10]. For this platform, they describe what it must do for its stakeholders, and they are organized into modules based on business responsibility, as summarized in Table 2.1.

Table 2.1: Summary of Functional Requirements.

Module	**Description**
Authentication	Supports user registration, login, logout, and password recovery to ensure secure access to the system.
User Profile Management	Allows job seekers to maintain personal information, update job preferences, and manage uploaded resumes.
Job Discovery	Provides browsing, searching, filtering, and viewing of available job postings.
Job Application	Enables job seekers to submit applications, withdraw applications before processing, and monitor application status.
Company Management	Allows employers to create and maintain company information presented to prospective applicants.
Job Management	Allows employers to create, edit, publish, close, and manage job postings throughout their recruitment lifecycle.
Recruitment Management	Supports employers in reviewing submitted applications and updating the recruitment status of candidates.
Administration	Allows administrators to manage user accounts, moderate job postings, and maintain overall platform integrity.

2.3 Non-Functional Requirement

Non-functional requirements are constraints on the services or functions offered by a system, and often apply to the system as a whole rather than to individual features [10]. The categories used here (security, performance, usability, reliability, maintainability, and compatibility) are taken from the product quality characteristics of the ISO/IEC 25010 quality model [11]. Table 2.2 summarizes the main ones considered during design.

Table 2.2: Summary of Non-Functional Requirements.

Category	Description
Security	The system shall authenticate users before granting access to protected resources and enforce role-based authorization. Sensitive credentials shall be securely stored using an adaptive password hashing algorithm such as bcrypt [12, 13].
Performance	The system shall provide responsive access to common operations such as browsing job postings, submitting applications, and managing recruitment information.
Usability	The system shall provide a responsive web interface with consistent navigation, clear validation messages, and an intuitive user experience for all supported roles.
Reliability	The system shall ensure data consistency through proper validation, controlled data modification, and appropriate error handling mechanisms.
Maintainability	The system shall adopt a modular architecture with clearly separated components to simplify future maintenance and feature extension.
Compatibility	The system shall support modern desktop web browsers without requiring additional client-side software installation.

2.4 Use Case Analysis

Use case analysis, introduced by Jacobson et al. [14], models how users interact with the platform and shows the functional scope from each user role's perspective. The diagrams in this section follow the Unified Modeling Language (UML) use case notation [15].

2.4.1 Shared Use Cases

To keep the model compact, the actors are organized with UML generalization, in which a specialized actor inherits the use cases of its more general parent [15]. Visitors and signed-in users are represented by the Guest actor and Authenticated User actor respectively.

Guest Use Cases

As shown in Figure 2.1, guests can browse job postings, view company information, and register. These public functions give visitors access to the platform without requiring an account.

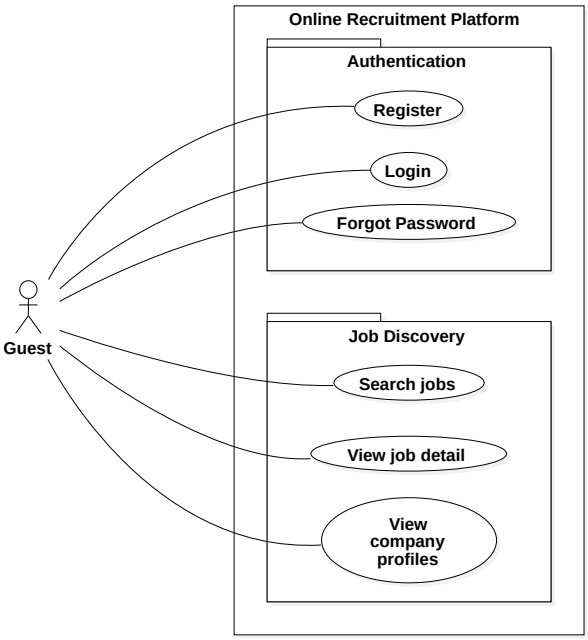


Figure 2.1: Guest Use Case Diagram.

Authenticated User Use Cases

Figure 2.2 covers the functions common to all signed-in users: managing account information, changing the password, viewing recommended jobs, and logging out. Business-specific functions are introduced through the specialized actors in the next section.

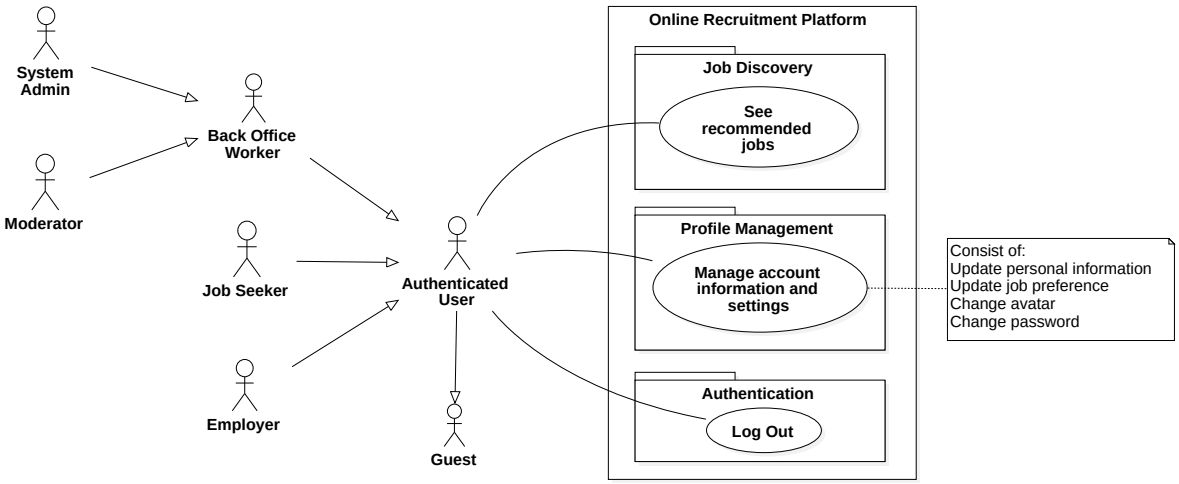


Figure 2.2: Authenticated User Use Case Diagram.

2.4.2 Specialized Use Cases

The following subsections describe the use cases of the three specialized actors: Job Seeker, Employer, and Back Office Worker. Each actor inherits the use cases of the Authenticated User and adds the functions specific to its role.

Job Seeker Use Cases

The Job Seeker is the main user on the demand side of the platform: this actor looks for jobs, manages their resume, also called a curriculum vitae (CV), and participates in recruitment through job applications.

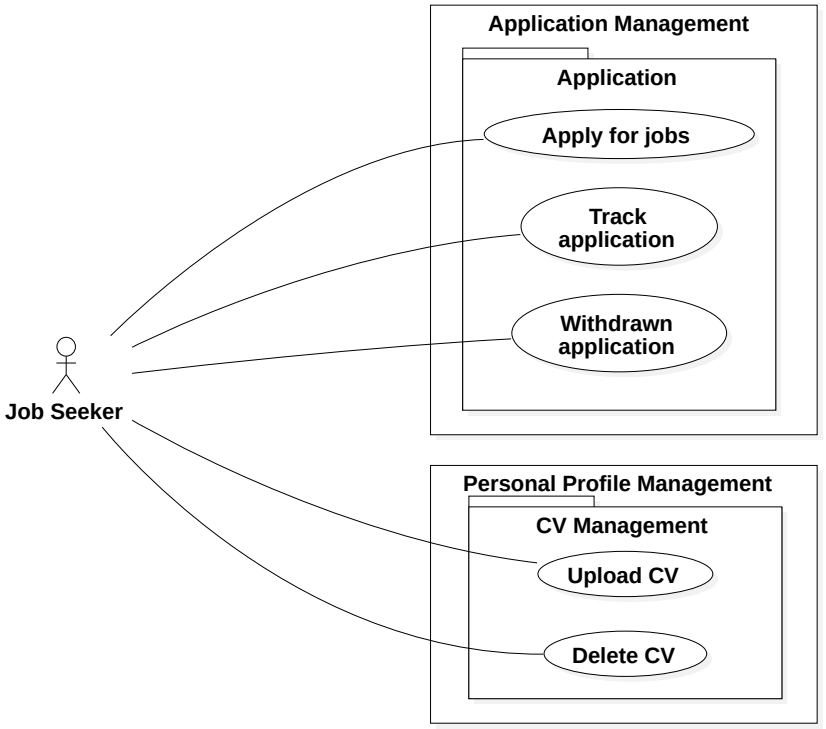


Figure 2.3: Job Seeker Use Case Diagram.

Figure 2.3 shows the Job Seeker's capabilities, including the CV Management and Application packages. The three application-related use cases: Applying, tracking, and withdrawing form the Job Application functionality examined in this thesis.

Employer Use Cases

The Employer represents the organizations and recruiters who publish job postings and evaluate applicants.

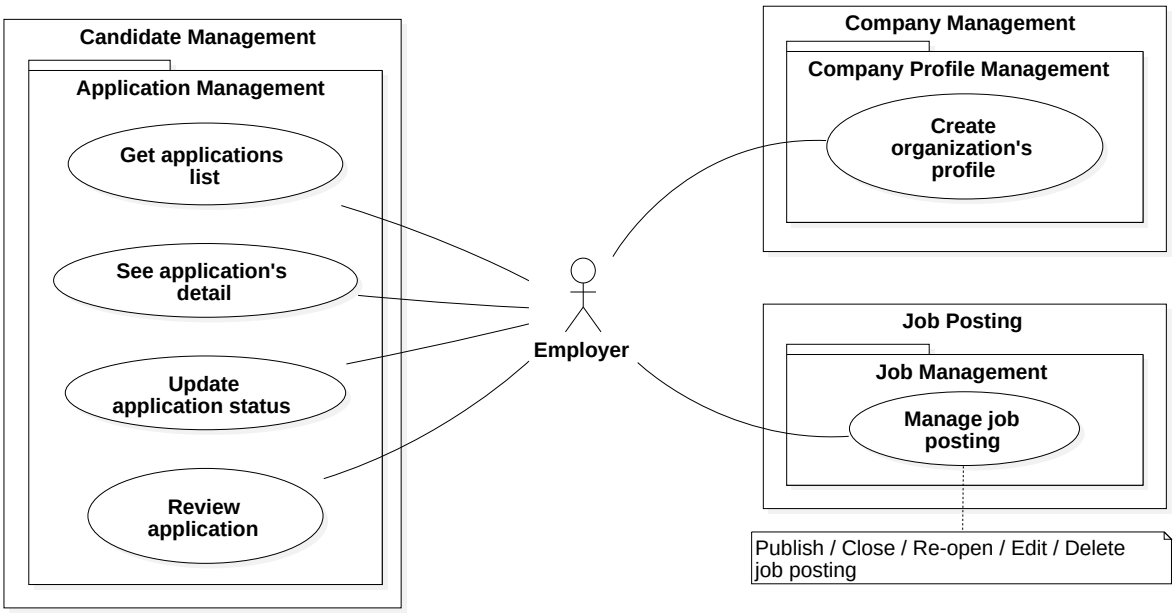


Figure 2.4: Employer Use Case Diagram.

Figure 2.4 shows the Employer's capabilities, including Company Profile Management, Job Management, and Application Management. These use cases collectively support the complete recruitment lifecycle on the employer's side, from creating a company profile and managing job postings to reviewing and updating the status of received applications. This functionality constitutes the Job Management module examined in this thesis.

Back Office Worker Use Cases

The Back Office Worker, consists of Moderator and System Administrator, represents internal personnel responsible for maintaining platform integrity and ensuring that the recruitment platform operates correctly.

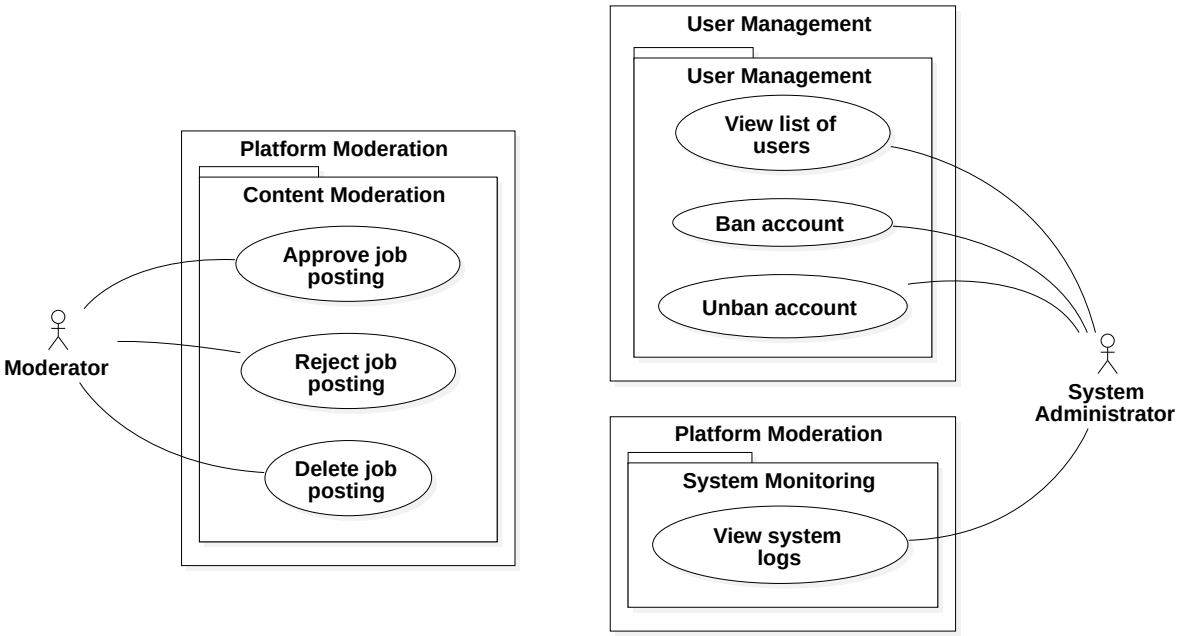


Figure 2.5: Back Office Worker Use Case Diagram.

Figure 2.5 shows the Back Office Worker's capabilities, including Platform Moderation and User Management. These use cases highlight the supervisory tasks of the back office worker, from managing user's status, moderating job postings, and overseeing important system events through logs.

2.5 Use Case Specification

The use case diagrams above give an overview of the system's functional scope. This section specifies the main use cases of the two featured modules in detail. Each specification follows the fully dressed template described by Cockburn [16]: the goal, preconditions, trigger, main success scenario, alternative flows (extensions), and postconditions (success guarantees) of the use case.

2.5.1 Job Seeker: Apply for Job

This subsection provides a detailed specification of the Apply for Job use case identified in the Job Application module.

Table 2.3: Use Case Specification: Apply for Job.

Field	**Specification**
Use Case ID	UC-JS-01
Use Case Name	Apply for Job
Primary Actor	Job Seeker
Goal	Submit a job application to an employer.
Description	Allows a Job Seeker to submit an application for a published job posting.
Preconditions	• Job Seeker is authenticated. • The job posting is open for applications. • Job Seeker has an uploaded resume.
Trigger	The Job Seeker selects the **Apply** option on a job posting.
Main Success Scenario	1. Job Seeker views the job posting. 2. Job Seeker selects Apply. 3. System displays the application form and the job summary. 4. Job Seeker selects a resume. 5. Job Seeker submits the application. 6. System validates that the job is active, the resume belongs to the user, and no duplicate application exists. 7. System records the application as `SUBMITTED`. 8. System confirms the submission.

Table 2.3 (Continued)

Field	Specification
Alternative Flows	**A1. No resume uploaded** 1. System asks the Job Seeker to upload a resume first. **A2. Already applied** 1. System rejects the submission. 2. System informs the Job Seeker that an application already exists. **A3. Posting no longer active** 1. System rejects the submission. 2. System informs the Job Seeker that the posting is no longer available.
Postconditions	• A new application with status `SUBMITTED` is created.

2.5.2 Employer: Publish Job Posting

This subsection provides a detailed specification of the Publish Job Posting use case identified in the Job Management module.

Table 2.4: Use Case Specification: Publish Job Posting.

Field	**Specification**
Use Case ID	UC-EMP-01
Use Case Name	Publish Job Posting
Primary Actor	Employer
Goal	Submit a job posting for moderation so it can be published to potential applicants.
Description	Allows an Employer to create a new job posting and submit it for moderation. The posting becomes publicly visible only after a moderator approves it.
Preconditions	• Employer is authenticated. • Employer has a registered company profile.
Trigger	Employer selects the **Post a Job** function.
Main Success Scenario	1. Employer fills in the job title, description, requirements, location, employment type, and deadline. 2. Employer submits the posting. 3. System validates the input and checks that the deadline is in the future. 4. System assigns the employer's company to the posting. 5. System creates the posting with status `PENDING_APPROVAL`. 6. System confirms the creation and shows the posting in the employer's list.
Alternative Flows	**A1. No company profile** 1. System requires the Employer to create a company profile first. **A2. Invalid input** 1. System identifies the invalid fields. 2. Employer corrects and resubmits.

Table 2.4 (Continued)

Field	**Specification**
Postconditions	• A job posting with status `PENDING_APPROVAL` is stored. • The posting is hidden from public search until approved by a moderator.

2.5.3 Employer: Review Application

This subsection provides a detailed specification of the Review Application use case identified in the Recruitment Management module.

Table 2.5: Use Case Specification: Review Application.

Field	Specification
Use Case ID	UC-EMP-06, UC-EMP-07, UC-EMP-08
Use Case Name	Review Application
Primary Actor	Employer
Goal	Review the applications a posting received and update their status.
Description	Allows an Employer to view the applications submitted to a posting and move each one through the review pipeline.
Preconditions	• Employer is authenticated. • The posting belongs to the employer's company.
Trigger	Employer opens the application list of a job posting.
Main Success Scenario	1. Employer selects a job posting. 2. System lists the applications with candidate name, resume, and status. 3. Employer opens an application's detail. 4. Employer marks the application `UNDER_REVIEW`. 5. Employer decides to accept or reject the application, with an optional reason on rejection. 6. System updates the application to `ACCEPTED` or `REJECTED` and records the decision time.
Alternative Flows	**A1. Application withdrawn** 1. System blocks further processing of the application. **A2. Application already decided** 1. System rejects the status change. **A3. Application not under review** 1. System only allows accept or reject on applications that are `UNDER_REVIEW`.

Table 2.5 (Continued)

Field	**Specification**
Postconditions	• The application is `ACCEPTED` or `REJECTED`, with the decision time recorded.

Other use case specifications, covering the supporting modules, are shown in Appendix A.

Chapter 3

System Design

This chapter presents the technical design of the proposed system based on the requirements identified in Chapter 2. It describes the system architecture, database design, backend structure, and frontend organization that form the blueprint for the implementation discussed in Chapter 4.

3.1 Design Overview

The design follows a small set of guiding principles, each addressing one or more non-functional requirements identified previously. Table 3.1 summarizes these principles and their rationale.

Table 3.1: Design Principles and Rationale.

Principle	**Rationale**
Layered Architecture	Separates presentation, application programming interface (API), business logic, and data access concerns, improving maintainability and testability [17].
Feature-Oriented Modular Design	Organizes code by business domain (e.g., Job, Application) rather than by technical type, keeping related logic together and easing future extension; modules are drawn around design decisions likely to change [18].
Thin Controller, Rich Service	Controllers handle only request/response concerns; business logic and validation reside in services, reducing duplication and improving reusability, following the Service Layer pattern [17].
Security by Design	Every access to a restricted resource is checked for authentication and authorization [19], while credentials are stored as adaptive password hashes [12, 13].
Data Integrity	Relational constraints, foreign keys, and database integrity rules ensure data consistency and prevent invalid relationships [20].

These principles are applied consistently across all modules of the system. They are illustrated in detail for the two featured modules, Job Application and Employer Job Management; the design of the remaining modules is documented in Appendix C.

The remainder of this chapter is organized as follows. Section 3.2 presents the system architecture, including the C4-based container view. Section 3.3 details the database design and entity-relationship model for the featured modules. Section 3.4 describes the backend design, including the layered structure and module organization. Section 3.5 covers the frontend design. Section 3.6 summarizes the chapter.

3.2 System Architecture

The system architecture is documented using the C4 model proposed by Simon Brown [21], which describes a software system at progressively narrower levels of zoom: Context, Container, and Component.

3.2.1 System Context

The System Context diagram identifies the system's boundary and its interactions with external actors, without describing internal structure.

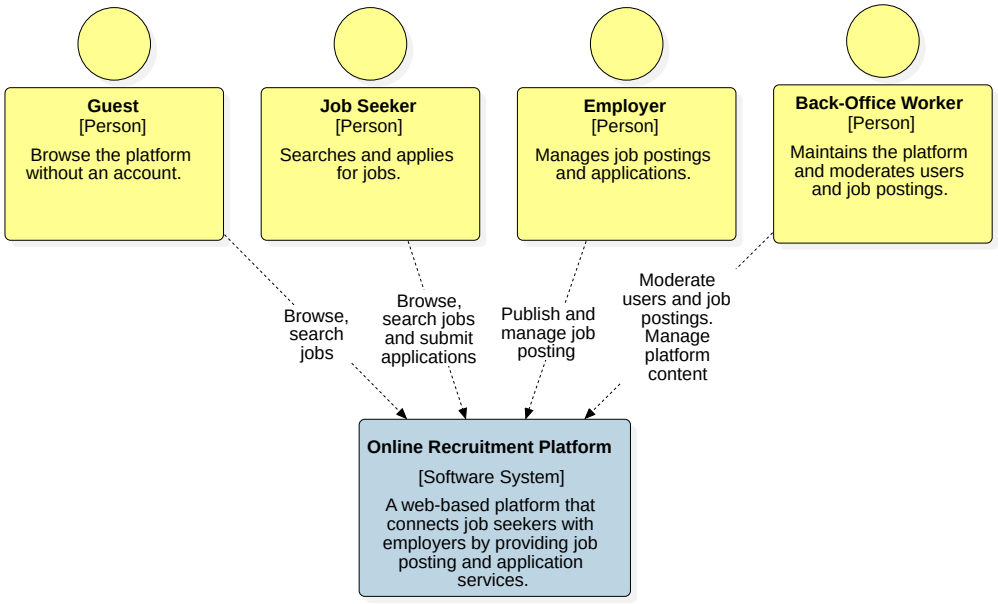


Figure 3.1: System Context Diagram.

As shown in Figure 3.1, four actors interact with the platform: the Guest browses for jobs without needing to authenticate, the Job Seeker applies for jobs, the Employer publishes and manages job postings, and the System Administrator moderates users and platform content.

3.2.2 Container View

The Container diagram decomposes the system into its major deployable units and shows how they communicate.

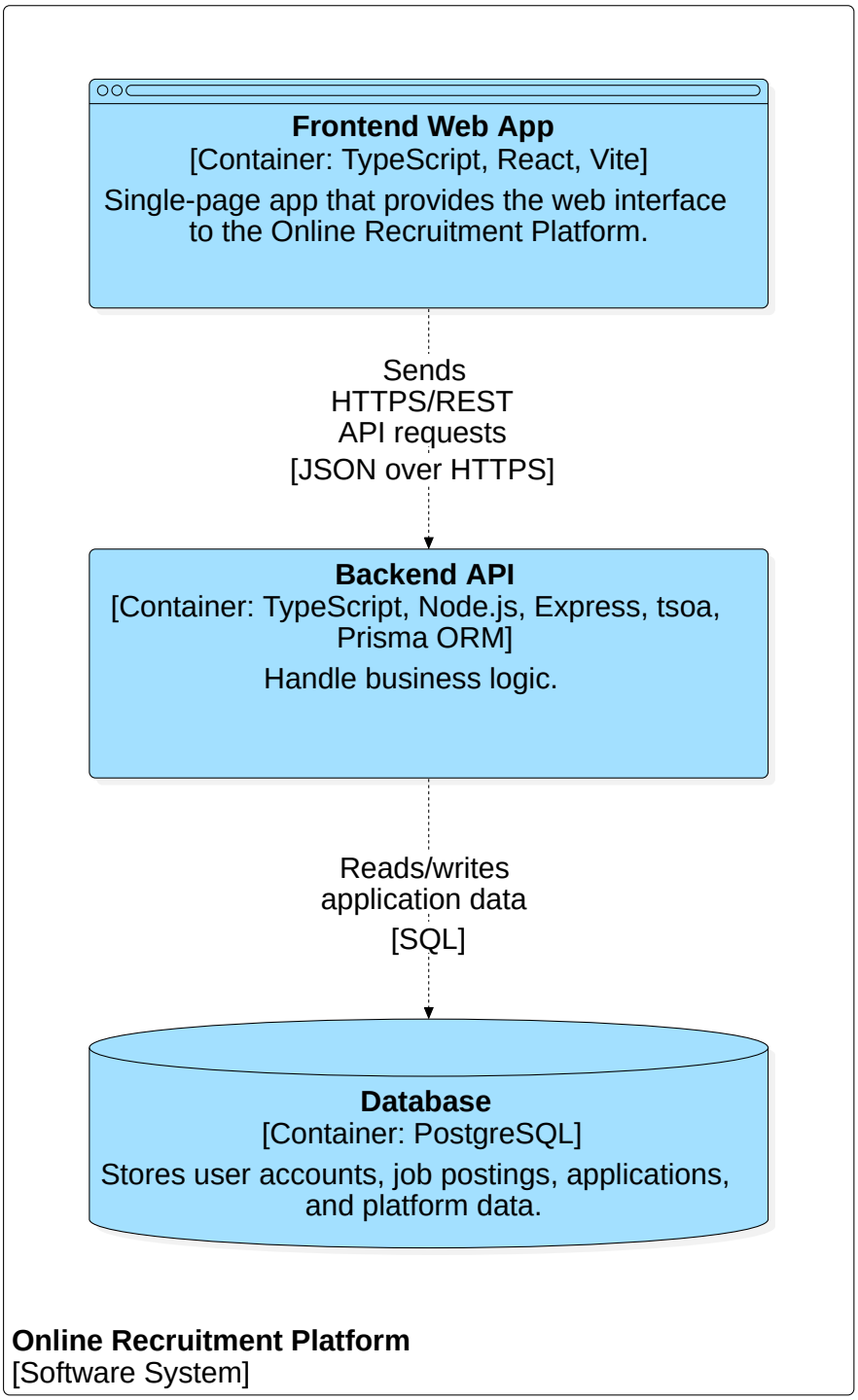


Figure 3.2: Container Diagram.

As illustrated in Figure 3.2, the system consists of three containers:

- The Frontend Web App is a React [22] single-page application (SPA) that renders the user interface (UI).
- The Backend API is a Node.js/Express service [23, 24], built with TypeScript [25] and the Prisma object-relational mapping (ORM) library [26], that implements all business logic and exposes it as a Representational State Transfer (REST) API [27].

- The Database is a PostgreSQL [28] instance that persists user accounts, job postings, applications, and other platform data.

The Frontend communicates with the Backend over Hypertext Transfer Protocol Secure (HTTPS) [29] using JavaScript Object Notation (JSON) [30], and the Backend communicates with the Database using Structured Query Language (SQL) through Prisma.

3.2.3 Component View: Frontend

The Frontend component diagram decomposes the Frontend Web App container into its internal building blocks.

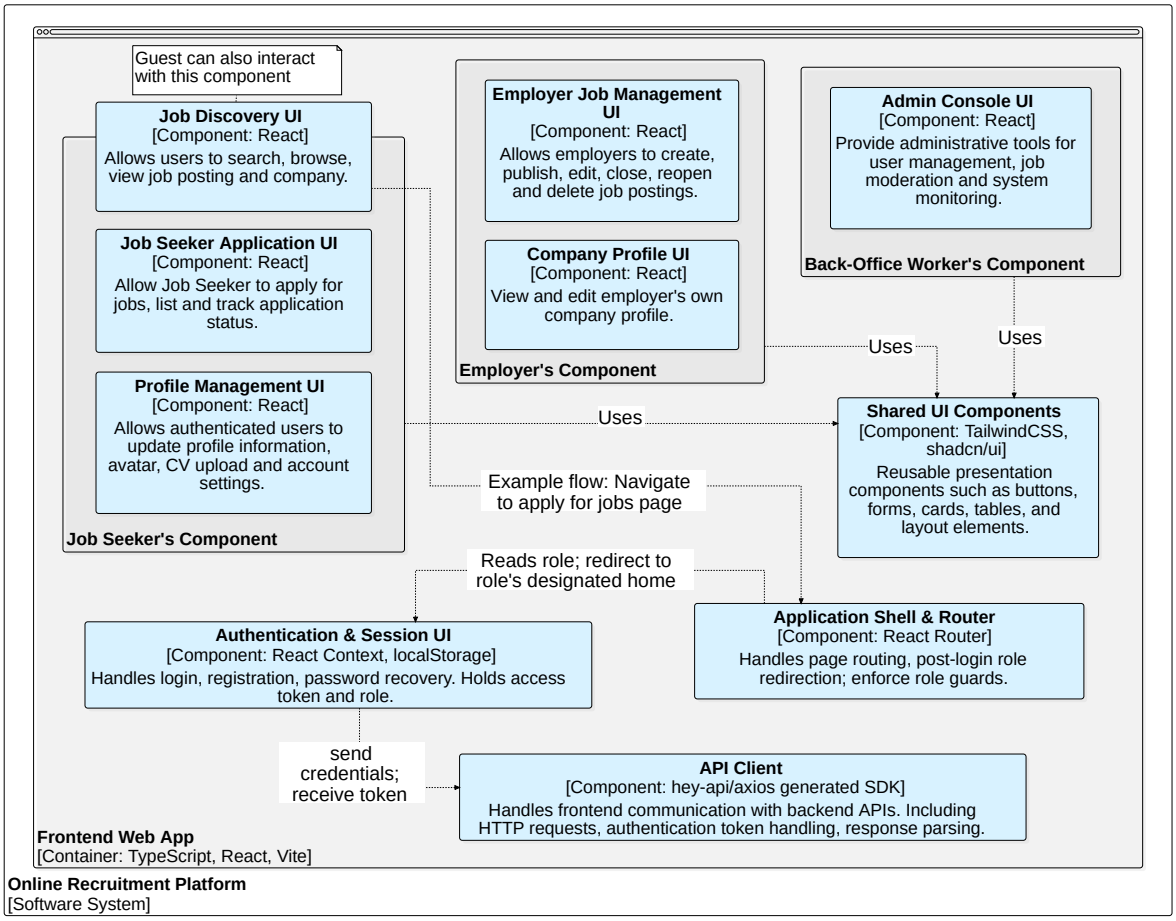


Figure 3.3: Frontend Component Diagram.

As shown in Figure 3.3, the feature components are organized primarily by the roles they serve. Job Seeker, Employer, Back Office Worker functionality is grouped separately, while Job Discovery is shared across access contexts and can be used by both Guest and authenticated users.

Two shared components support the feature layers. The Application Shell and Router manages navigation and role-based access, while the Shared UI Kit provides reusable interface elements and common loading and error states.

Every feature component communicates with the backend exclusively through a centralized API Client, a generated software development kit (SDK) that wraps REST

calls, injects the Bearer token, handles 401 interception, and parses responses. No feature component makes direct Hypertext Transfer Protocol (HTTP) requests; the API Client is the single outbound integration point.

3.2.4 Component View: Backend

The Backend component diagram decomposes the Backend API container into functional components that implement the platform's business capabilities.

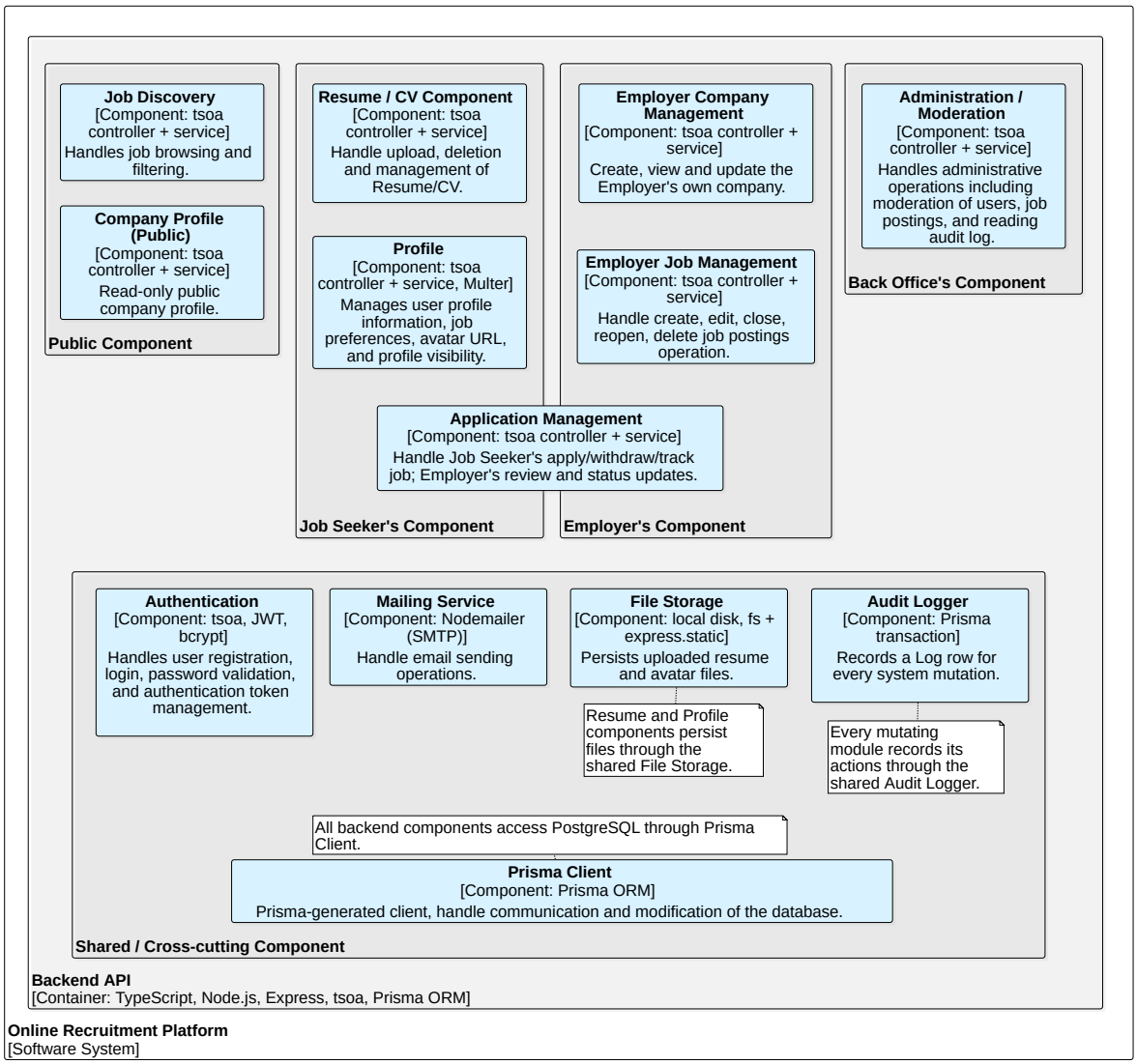


Figure 3.4: Backend Component Diagram.

As illustrated in Figure 3.4, the backend components are organized primarily by functional responsibility and user role. Public-facing functionality is separated from role-specific components, while Job Seeker, Employer, and Back Office components are grouped according to their respective workflows.

A shared infrastructure layer supports the role-specific components by providing common services such as authentication, mailing, file storage, and audit logging.

All components access PostgreSQL through a single Prisma Client instance, providing a consistent data access mechanism and avoiding service-to-service API calls within the application.

3.3 Database Design

The database design defines how the system's persistent data is organized and managed. It identifies the main business entities, their relationships, and the constraints that ensure data consistency. The design follows the principles introduced in Section 3.1, particularly data integrity, by using a normalized relational schema with clearly defined primary and foreign key relationships [20].

3.3.1 Entity Relationship Overview

Figure 3.5 presents an overview of the complete database schema used by the platform as an entity-relationship diagram (ERD) [31]. To improve readability, the overview diagram includes the primary entities, key attributes, and relationships while omitting system-generated fields such as timestamps and audit metadata.

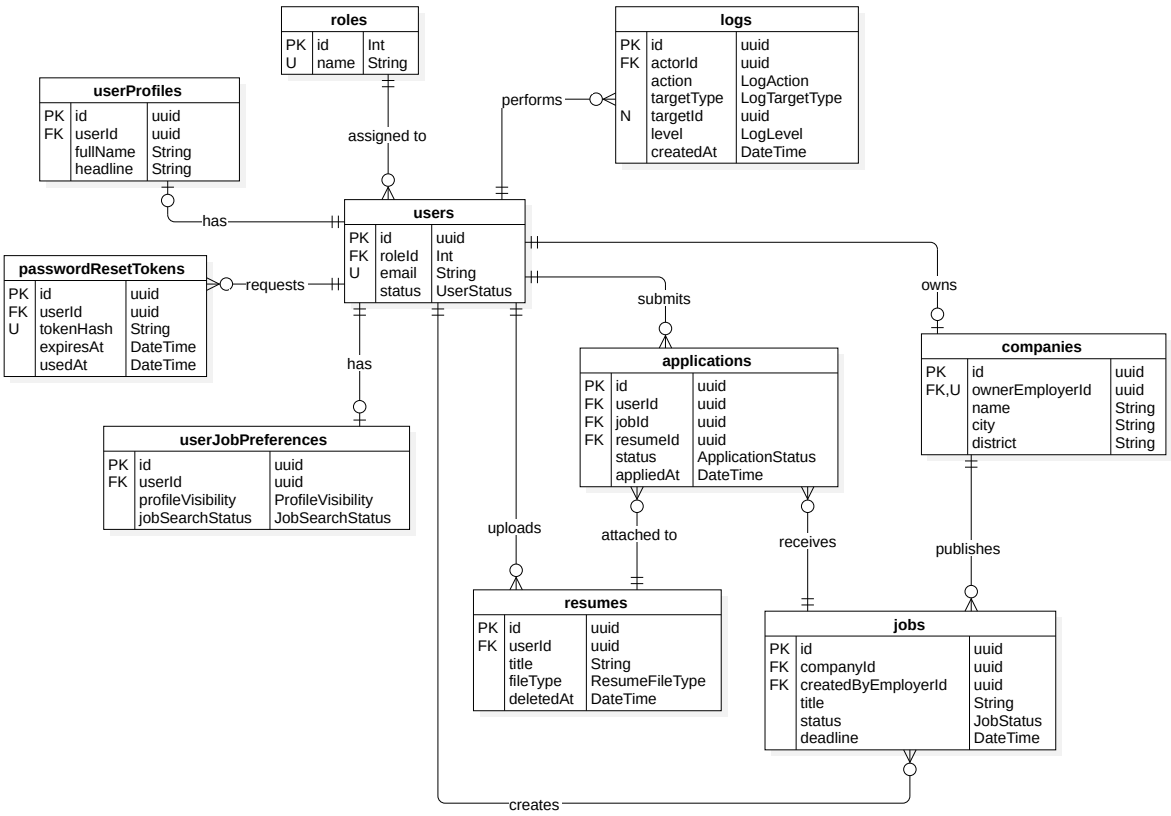


Figure 3.5: Entity Relationship Diagram.

The remainder of this section details the entity groups that back the two featured modules - the company and job posting domain and the job application domain. The entities supporting the remaining modules are described in Appendix B.

3.3.2 Company and Job Posting

Figure 3.6 details the entities involved in company and job posting management.

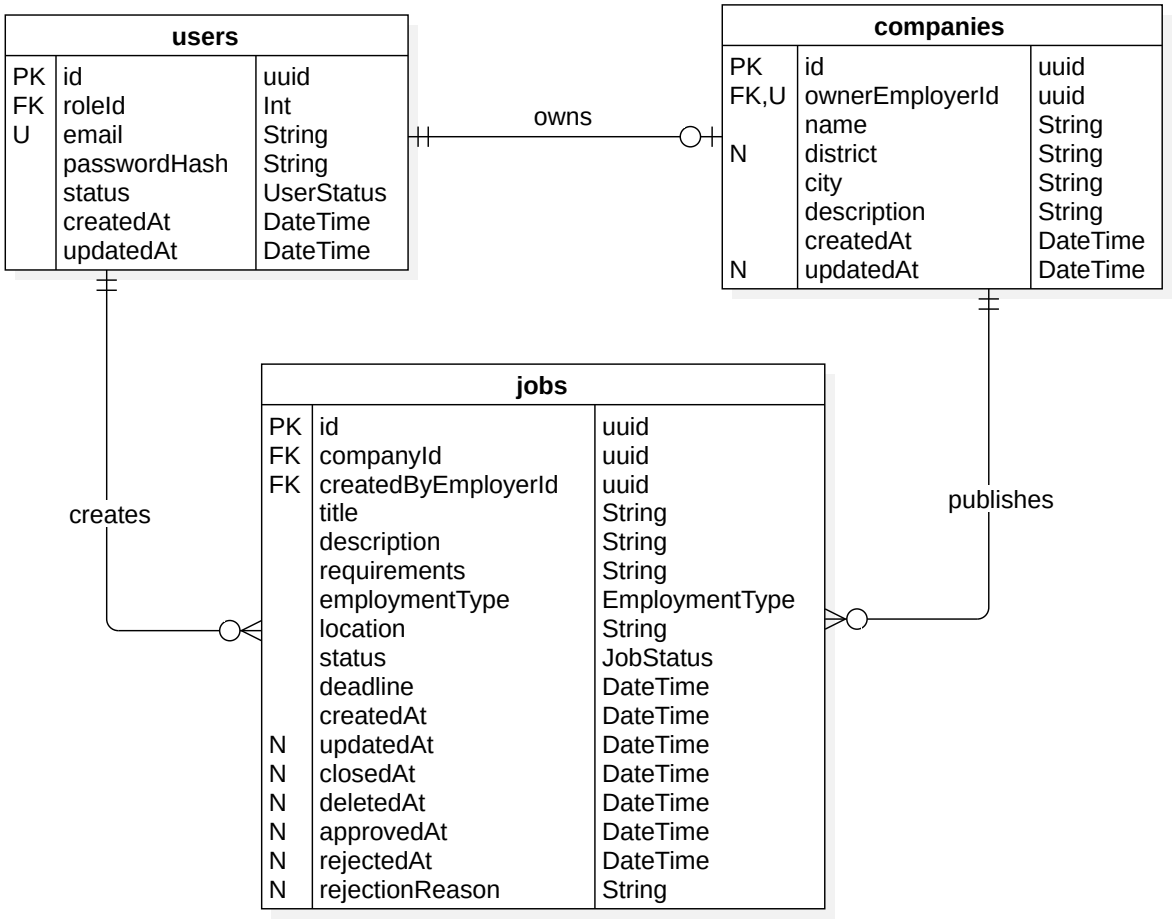


Figure 3.6: Company and Job Posting Entity Relationship Diagram.

An employer may own at most one company, and each company belongs to exactly one employer. Similarly, an employer may create multiple job postings, but each job posting is created by exactly one employer. Each job posting belongs to exactly one company, while a company may publish any number of job postings. The `status` attribute represents the current state of a job posting through the `JobStatus` enumeration.

3.3.3 Job Application

Figure 3.7 details the entities involved in the job application workflow.

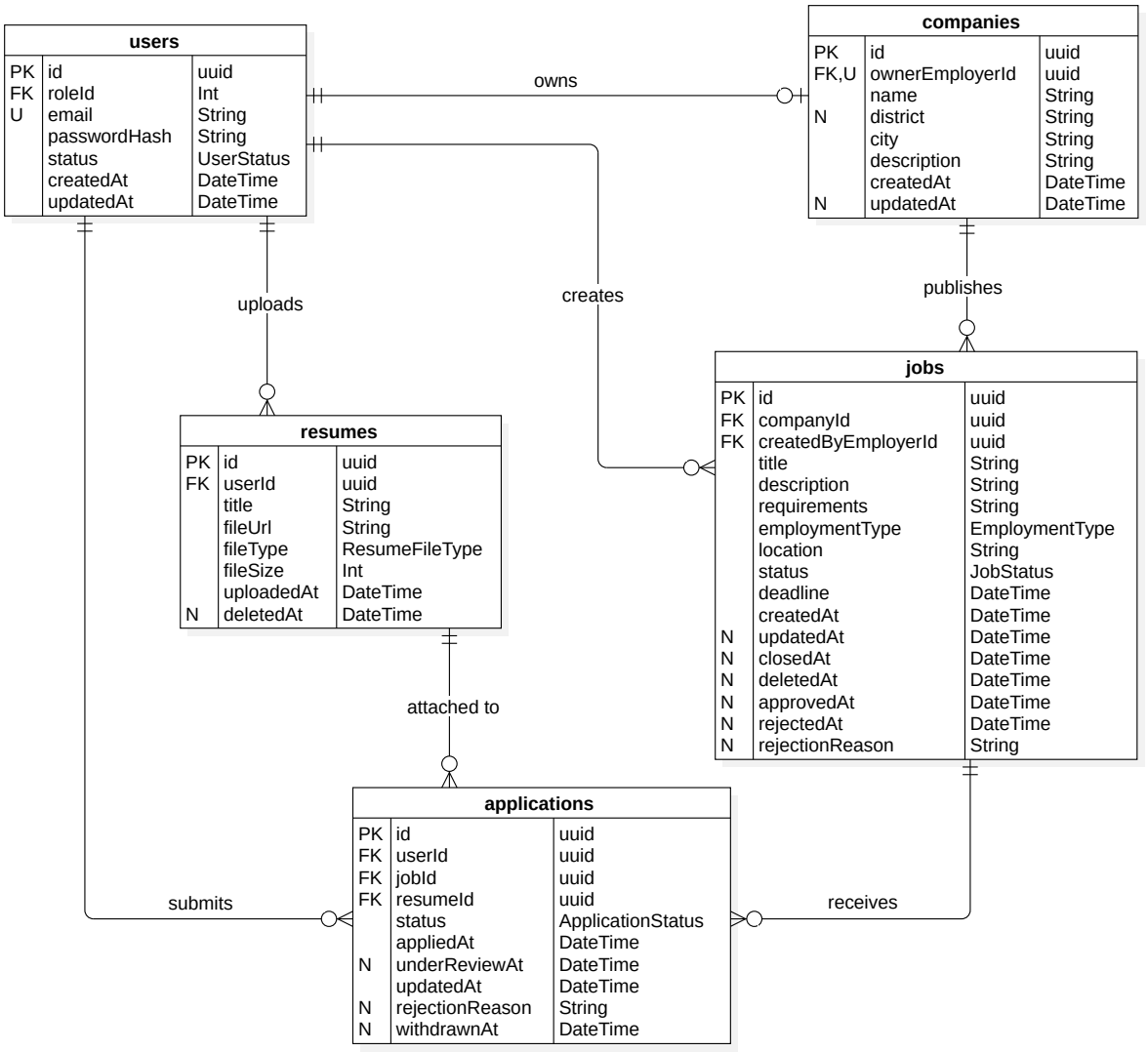


Figure 3.7: Job Application Entity Relationship Diagram.

A job seeker may upload multiple resumes and submit multiple applications, but each resume and each application belongs to exactly one job seeker. Each application is submitted for exactly one job posting, while a job posting may receive any number of applications. A resume may be attached to multiple applications, but each application references exactly one resume. The application's `status` attribute records where the application stands in the review process through the `ApplicationStatus` enumeration.

3.4 Backend Design

This section details the internal design of the Backend API container introduced in Section 3.2. It first describes the project structure and shared infrastructure common to all modules, then presents the two featured business modules following the same template: an introduction, a class diagram, and a design explanation. The class diagrams of the remaining modules are presented in Appendix C.

3.4.1 Backend Project Structure

The backend source code is organized as follows:

```
src
├── app.ts
├── server.ts
├── authentication.ts
├── api-external
├── api-internal
├── api-shared
├── email
├── logging
├── generated
├── lib
├── security
└── utils
```

Table 3.2 summarizes the purpose of each top-level entry.

Table 3.2: Backend Project Structure.

Entry	**Purpose**
app.ts, server.ts	Wire the Express application: register global middleware and the generated routes, and bootstrap the HTTP server.
authentication.ts	Implements `expressAuthentication` for tsoa: verifies the JSON Web Token (JWT) on protected routes and rejects banned accounts.
api-external	Implements business modules that expose application functionality to external users through RESTful APIs.
api-internal	Implements internal modules that support administrative and platform management operations.
api-shared	Provides reusable guards and logic shared across multiple business domains to reduce duplication.
email	Wraps the mail transport and holds transactional email templates.
logging	Provides transactional audit logging services and message formatting for system event tracking.
generated	Contains files generated by tsoa and Prisma during the build.
lib	Maintains shared library instances and application-wide configurations used by multiple modules.
security	Defines the `CurrentUser` type that carries the authenticated user's identifier and role into services.
utils	Provides general-purpose utilities such as `HttpError`, `PasswordHasher`, `JwtService`, `ResetTokenUtils`, and `FileStorageService`.

3.4.2 Shared Backend Components

A small set of reusable infrastructure components is shared across all business modules, avoiding duplication and ensuring consistent behavior.

- JWT Authentication: issues and verifies signed JWTs [32] carrying the user's identifier and role, and is enforced through the `expressAuthentication` hook on protected routes.
- Role Guards: helpers such as `assertJobSeeker` and `assertEmployer` reject requests from users who lack the required role, applied as the first step of every service method. The guards are always paired with the tsoa `@Security` decorator to form defense in depth, a strategy of layering several independent safeguards so that a failure in one is caught by another [33].
- HttpError: a uniform error type carrying an HTTP status and message, translated into the proper JSON error response.
- Prisma Client: the generated ORM client, instantiated once in `lib/prisma.ts` and imported by services for all database access.
- File Storage: abstracts file persistence behind a common interface, used for avatar and resume uploads.
- Email Service: wraps Nodemailer [34] to send transactional emails, such as password reset notifications.
- PasswordHasher and ResetTokenUtils: wrap bcrypt [12] and token generation for credential handling and password recovery.
- AuditLogger: records a structured entry for a system event inside the same database transaction as the mutation it accompanies.

3.4.3 Job Application Module

The Job Application Module lets job seekers apply to active job postings using one of their resumes, view their own submitted applications, and withdraw an application while it is still eligible. It resides in `api-external`, and every endpoint requires both a valid JWT and the `JOB_SEEKER` role.

Class Diagram

Figure 3.8 presents the class diagram of the Job Application Module, drawn in UML class diagram notation [15].

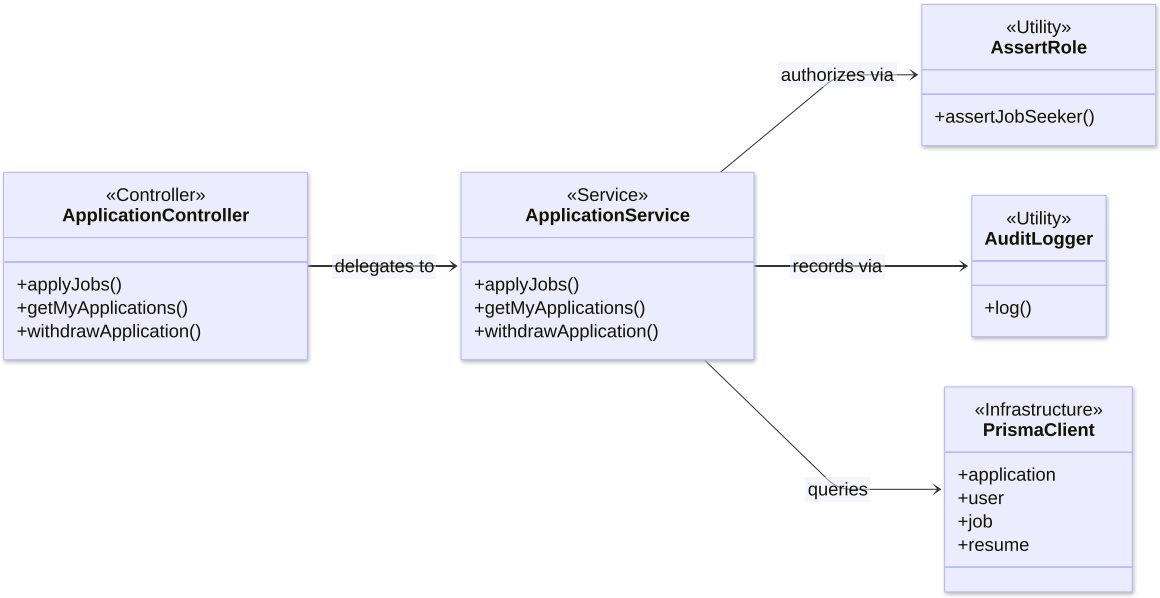


Figure 3.8: Job Application Module Class Diagram.

Design Explanation

Following the thin controller, rich service principle, `ApplicationController` exposes three REST endpoints and delegates each directly to a matching `ApplicationService` method, without containing business logic itself. This separation keeps HTTP adaptation concerns - request parsing, response shaping, and error mapping - distinct from the business rules that determine how an application can be created, listed, or withdrawn.

`ApplicationService` is the only class in the module that owns business logic and the only class that accesses the database, doing so through the shared Prisma Client. It operates on the User, Job, Resume, and Application entities whose relationships are detailed in Figure 3.7. The service depends on two shared infrastructure components: `AssertRole`, and `AuditLogger`. These components are documented in Section 3.4.2.

Authentication and role authorization are enforced at two levels. At the controller boundary, tsoa's `@Security("jwt", ["JOB_SEEKER"])` decorator causes the framework to verify the JWT and confirm the caller's role before the controller method is entered. The service then re-asserts the same role via `AssertRole`, providing defense in depth: business methods remain protected regardless of how they are invoked. Each service method corresponds to a single use case described in Chapter 2.

3.4.4 Employer Job Management Module

The Employer Job Management Module covers the capabilities exposed to employers: managing their company profile, managing the lifecycle of their job postings, and reviewing submitted applications. Together with the Job Application Module, it forms the complete round trip of a recruitment event studied in this thesis.

Class Diagram

Figures 3.9, 3.10, and 3.11 present the class diagrams of the Employer Job Management Module.

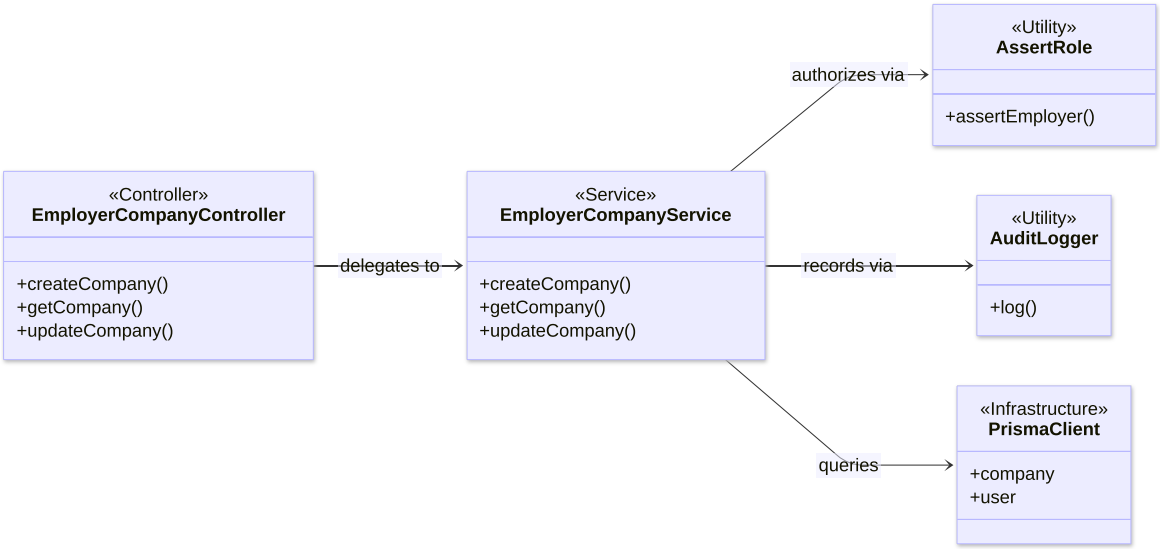


Figure 3.9: Company Profile Management Class Diagram.

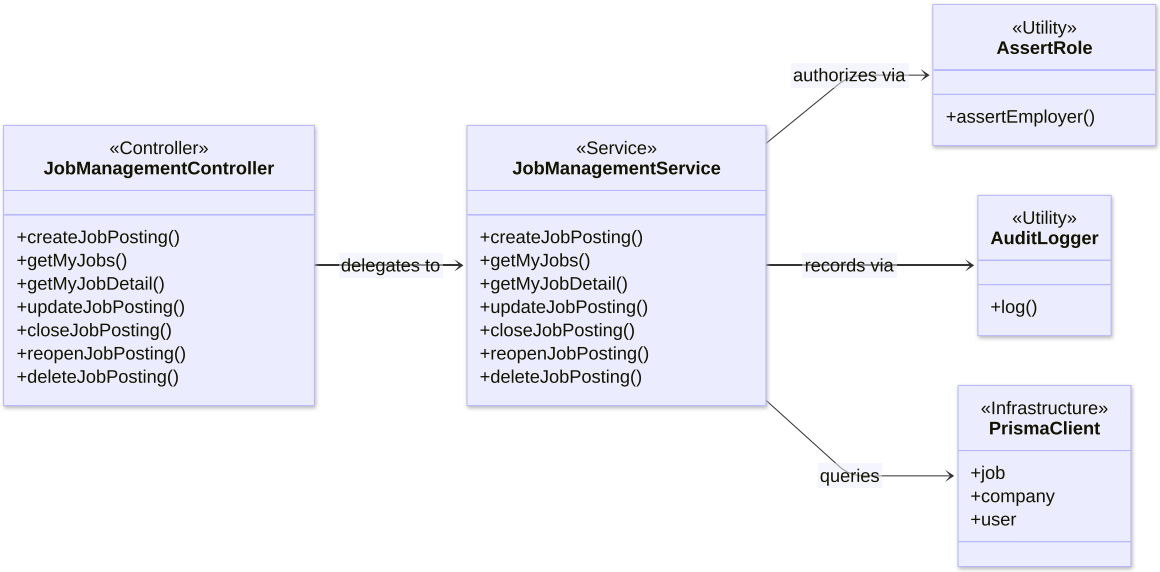


Figure 3.10: Job Posting Management Class Diagram.

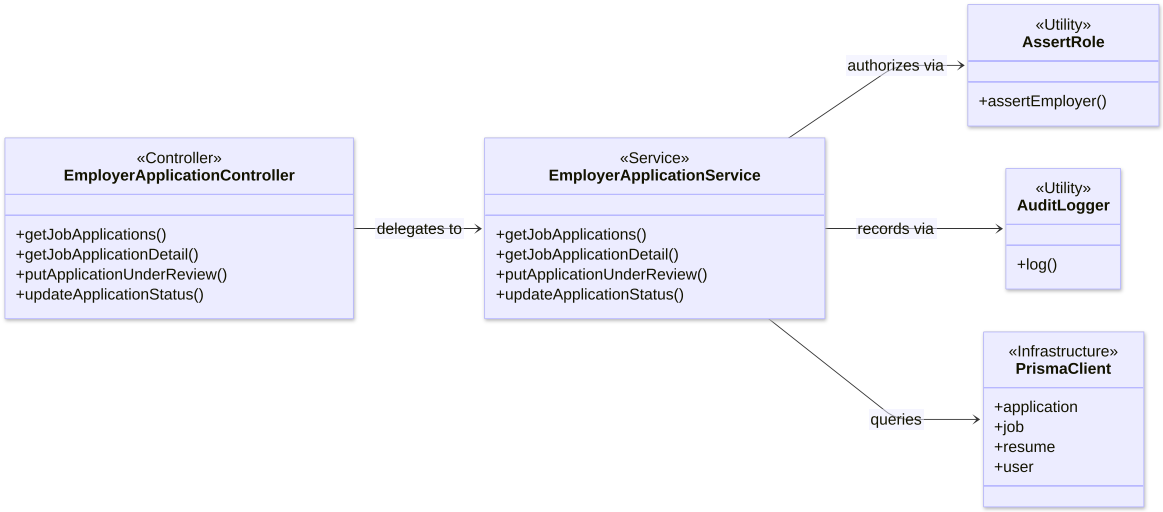


Figure 3.11: Employer Application Management Class Diagram.

Design Explanation

The Employer Job Management Module is decomposed into three cohesive controller and service pairs, each owning a distinct subdomain of the employer workflow.

- `EmployerCompanyController` and `EmployerCompanyService` handle company profile management.
- `JobManagementController` and `JobManagementService` manage the lifecycle of job postings.
- `EmployerApplicationController` and `EmployerApplicationService` cover the review of incoming applications.

This separation keeps each pair focused on a single business concern and allows it to evolve independently.

Across all three pairs the same structural pattern is applied: each controller is annotated with tsoa's `@Security("jwt", ["EMPLOYER"])` decorator at the class level, while exposing REST endpoints and delegating each directly to a matching service method. Each service is the sole owner of business rules and the only class in its pair that accesses the database through the shared Prisma Client. Every service method depends on `AssertRole` and `AuditLogger`, which are described in Section 3.4.2.

Company Profile Management takes no company identifier in its operations because the employer-company relationship is one-to-one: the owning employer is resolved from the authenticated user. Job Posting Management's own methods can only move a posting into `PENDING_APPROVAL`, `CLOSED`, or `DELETED`, so a posting cannot become publicly visible without first passing through this external moderation gate. Employer Application Management exposes operations for the employer-driven status transitions that complement the job-seeker-initiated `SUBMITTED` and `WITHDRAWN` transitions defined in the Job Application Module. Together the three pairs implement the complete round trip of a recruitment event studied in Chapter 2.

3.5 Frontend Design

The frontend is a React SPA. Pages are grouped in feature folders that mirror the backend module structure, so a feature can be traced end to end.

3.5.1 Frontend Structure and Shared Components

Each feature module under `ui-external/` follows the same internal shape: page-level components sit directly in the module's folder, and any component used only within that module is placed in a nested `components/` folder beneath it. For example, the Job Application module is organized as follows:

```
ui-external/applications
├── ApplyJobPage.tsx
├── MyApplicationsPage.tsx
└── components
    ├── ApplicationCard.tsx
    ├── ApplicationStatusBadge.tsx
    └── ResumeSelector.tsx
```

The same structure is applied to the Employer Job Management UI and other feature modules. Shared interface elements are provided through the common UI layer and reused across feature components rather than being duplicated within individual modules. The project uses shadcn/ui [35] and Tailwind CSS [36] for these reusable interface elements.

3.5.2 Job Application UI

The Job Application UI follows the feature structure described above. Its page-level components are separated according to the two primary application workflows: applying for a job and managing submitted applications. Reusable elements specific to these workflows, such as application cards, status indicators, and resume selection, are kept within the feature's `components/` directory.

Components are implemented as `.tsx` files, keeping their rendering logic and component-specific styling together. This convention keeps each UI component self-contained and avoids unnecessary separation between files that belong to the same component.

3.5.3 Employer Job Management UI

The Employer Job Management UI applies the same feature-based organization to the employer workflows. Page-level components handle the major job-management and company-profile screens, while reusable elements are colocated within their respective feature directories.

The UI reuses shared components and the centralized API Client for common interface and backend-integration concerns. This keeps the employer-specific UI focused on its feature responsibilities while avoiding duplication of shared frontend functionality.

3.6 Summary

This chapter presented the technical design of the platform. The system follows a layered, feature-oriented architecture split into a React frontend, a Node.js/Express API, and a PostgreSQL database. The database design centered on the entity groups that serve the two featured modules: company and job posting, and job application. The backend design introduced the project structure, the shared infrastructure components, and the class-level design of the Job Application and Employer Job Management modules. The frontend design described the page structure for both featured modules. The next chapter describes how this design was realized in code.

Chapter 4

Implementation

This chapter describes how the design presented in Chapter 3 was realized in code. It first covers the technology stack and the cross-cutting conventions shared by the whole system, then follows the same two-module focus as the previous chapters: the Job Application and Employer Job Management modules are implemented in full, while the remaining modules are covered briefly.

4.1 Overview

The design in Chapter 3 is implemented as a pnpm [37] monorepo, a single version-controlled repository that holds several related projects [38], in this case two applications. The backend, `apps/api`, is a Node.js/Express service written in TypeScript: it uses Prisma for database access and tsoa [39] to generate both the OpenAPI [40] specification and the route handlers from annotated controllers. The frontend, `apps/web`, is a React SPA built with Vite [41] and TypeScript. It uses the generated OpenAPI client for all API calls, Tailwind CSS with shadcn/ui components for styling, and React Router [42] for navigation.

The two applications exchange data exclusively over REST: the API exposes JSON endpoints and the web client calls them through generated functions whose request and response types match the backend data transfer objects (DTOs). Because the client is regenerated from the backend's OpenAPI specification, a change to a response type on the server is reflected in the client at the next build.

4.1.1 Request Pipeline

Every request follows the same path through the backend:

```
HTTP request
  -> generated/routes.ts        (tsoa-generated route table)
  -> Controller                 (tsoa decorators; routing + status codes)
  -> Service                    (business rules; owns all Prisma access)
  -> Prisma Client              (generated typed queries, translated to SQL)
  -> PostgreSQL                 (executes the SQL)
```

tsoa reads the `@Route`, `@Get`/`@Post`/`@Patch`, and `@Security` decorators on each controller and emits `routes.ts` plus `swagger.json`. The `@Security("jwt", [role])`

decorator marks an endpoint as protected and declares the required scope; at runtime, tsoa calls `expressAuthentication`, implemented in `authentication.ts`, which verifies the JWT, rejects banned accounts, and attaches the authenticated user as `currentUser` on the request. The service then re-checks the role itself as defense in depth.

4.1.2 Error Handling

Services raise a uniform `HttpError(status, message)`; a global error handler converts it into a JSON response, while validation performed by tsoa on the incoming body is answered automatically. The status codes used throughout the API follow their meanings in the HTTP specification [29] and are summarized in Table 4.1.

Table 4.1: HTTP Status Code Conventions.

Code	Meaning
400	Validation failure, unmet precondition, or illegal status transition.
401	Missing or invalid access token.
403	Wrong role, banned account, or request on a resource the caller does not own.
404	Resource does not exist or must not be revealed to the caller.
409	The resource is already in the requested state (for example, a duplicate application).

4.2 Backend Implementation

This section describes how the two featured modules are realized in the backend. Each module follows the request pipeline described above: controllers handle routing and status codes, while services enforce the business rules from the use case specifications before any data is written.

4.2.1 Job Application Module

The module lives in `api-external/job-application/` and exposes the endpoints in Table 4.2, all protected with the `JOB_SEEKER` scope.

Table 4.2: Job Application Endpoints.

Endpoint	Purpose
POST `/applications`	Submit an application for a job using a chosen resume.
GET `/applications/my`	List the caller's applications with job and company summary.
PATCH `/applications/{id}/withdraw`	Withdraw a submitted application.

applyJob The core of the module is the `applyJob` method, which guards every business rule from the use case specification before any row is written. Table 4.3 lists

these checks in the order they are applied; the method fails on the first one that does not hold.

Table 4.3: Business Rules Enforced by `applyJob`.

Check (applied in order)	**Failure Response**
Caller holds the `JOB_SEEKER` role	403 Forbidden
The job posting exists	404 Job not found
The job posting is `ACTIVE`	400 Job posting is no longer available
The resume exists and is not soft-deleted	404 Resume not found
The resume belongs to the caller	403 You do not own this resume
No prior application exists for this job and caller	409 You have already applied for this job

The duplicate check relies on the composite unique constraint `userId_jobId` defined on the `Application` table, which makes the one-application-per-job rule enforceable at the database level as well as in the service.

getMyApplications This method returns only the caller's applications.

withdrawApplication Table 4.4 lists the checks applied before an application can be withdrawn. On success the status is set to `WITHDRAWN` and the `withdrawnAt` timestamp is recorded; the row itself is kept for the job seeker's history.

Table 4.4: Business Rules Enforced by `withdrawApplication`.

Check (applied in order)	**Failure Response**
Caller holds the `JOB_SEEKER` role	403 Forbidden
The application exists	404 Application not found
The application belongs to the caller	403 You are not allowed to withdraw this application
The application is in `SUBMITTED` status	400 Application cannot be withdrawn at its current status

4.2.2 Employer Job Management Module

This module spans three folders in `api-external`: `employer-company/`, `employer-job-management/`, and `employer-application-management/`, all protected with the `EMPLOYER` scope.

Company Profile

The company controller is mounted at `/employer/company` and exposes the endpoints in Table 4.5. Because an employer owns at most one company, no company identifier is ever sent by the client: every method resolves the company by `ownerEmployerId`, taken from the authenticated token rather than the request body.

Table 4.5: Company Profile Endpoints.

Endpoint	Purpose
GET /employer/company	View the caller's own company profile.
POST /employer/company	Create the caller's company profile.
PATCH /employer/company	Update the caller's company profile.

Table 4.6 lists the check each endpoint applies beyond the shared role check.

Table 4.6: Business Rules Enforced by the Company Profile Endpoints.

Endpoint	Check	Failure Response
All	Caller holds the EMPLOYER role	403 Forbidden
GET	Caller already has a company	404 You haven't created a company
POST	Caller does not already have a company	409 You have already created a company
PATCH	Caller already has a company	404 You haven't created a company

Job Posting Lifecycle

The endpoints in Table 4.7 implement the full lifecycle of a job posting. Every mutating method first loads the posting through a shared helper that both checks existence and confirms the caller created it, answering 404 when the posting does not exist and 403 when the caller is not its owner.

Table 4.7: Job Posting Endpoints.

Endpoint	Purpose
GET /employer/jobs	List the employer's postings, newest first.
GET /employer/jobs/{id}	Detail of one owned posting.
POST /employer/jobs	Create a posting (status PENDING_APPROVAL).
PATCH /employer/jobs/{id}	Edit a posting; resets its status to PENDING_APPROVAL.
PATCH /employer/jobs/{id}/close	Close an ACTIVE posting.
PATCH /employer/jobs/{id}/reopen	Re-open a CLOSED posting.
PATCH /employer/jobs/{id}/delete	Soft-delete a posting.

Input validation Before creating or editing a posting, the service validates every required field in order and rejects the request with 400 on the first check that fails; Table 4.8 lists these checks, each producing a message that names the specific field.

Table 4.8: Input Validation Rules for Creating or Editing a Job Posting.

Check	**Failure Response**
Title, description, requirement, and location are non-empty after trimming	400 *Field* cannot be empty
Title and location are at most 255 characters	400 *Field* can only be at most 255 character long
Description and requirement are at most 10,000 characters	400 *Field* can only be at most 10000 character long
Deadline parses as a valid date	400 Deadline is not a valid date
Deadline lies in the future	400 Deadline must be a future date
A company profile already exists for the caller, since `companyId` is resolved from it rather than supplied by the client	400 You need to register a Company before you can publish

Status transitions Figure 4.1 presents the job posting lifecycle as a UML state machine [15], where each transition corresponds to one guarded method call.

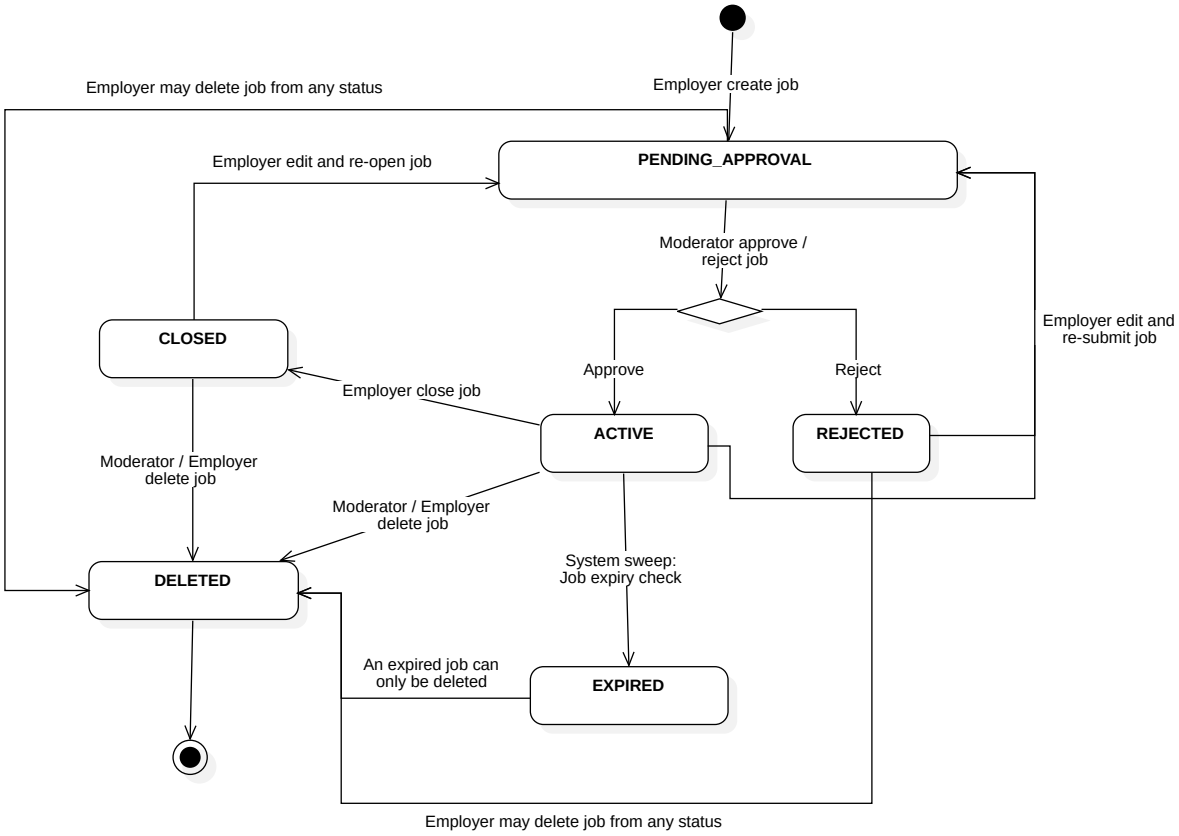


Figure 4.1: Job Posting Status Transitions.

Two transitions are not driven by this module at all: `Approve` and `Reject` belong to the Job Moderation module described in Chapter 3, and `Deadline passed` is not a request but a scheduled sweep that turns overdue `ACTIVE` postings into `EXPIRED` on its own, independent of any employer or moderator action. Deleting is always a soft

delete: the status is set to `DELETED` and a `deletedAt` timestamp is recorded, so the record and its applications are preserved for history rather than removed.

Employer Application Review

The endpoints in Table 4.9 close the loop on the application flow: the employer sees what the job seeker submitted and moves it towards a decision.

Table 4.9: Employer Application Review Endpoints.

Endpoint	Purpose
GET `/employer/jobs/{id}/applications`	List applications received by a posting.
GET `/employer/applications/{id}`	Full detail of one application (candidate and resume).
PATCH `/employer/applications/{id}/under-review`	Move `SUBMITTED` to `UNDER_REVIEW`.
PATCH `/employer/applications/{id}/status`	Accept or reject an application under review.

The application list and the detail both verify that the application belongs to a job posting owned by the employer before returning data. The two state-moving methods apply the same status gates: an already `WITHDRAWN`, `ACCEPTED`, or `REJECTED` application cannot be changed again, under-review requires the current status `SUBMITTED`, and accept/reject requires the current status `UNDER_REVIEW`. On rejection an optional reason is trimmed, capped at 1000 characters, and stored in `rejectionReason`; the decision time is recorded in `decidedAt`.

4.2.3 Remaining Modules

The remaining modules follow the same controller - service pattern; they are covered briefly in Appendix C.

4.3 Frontend Implementation

This section describes how the two featured modules are realized in the React frontend.

4.3.1 Job Application UI

Apply page `ApplyJobPage` (`/jobs/:jobId/apply`) loads the posting and the user's resumes on mount. The page shows the job summary and a `ResumeSelector`; the first available resume is preselected. On submit it calls the `applyJobs` API; Table 4.10 shows how each failure from Table 4.3 is displayed to the user. A success state replaces the form with a confirmation and links to the applications page.

Table 4.10: Apply Page Error Display.

Response	Displayed Message
409 (duplicate application)	"You have already applied for this job. Duplicate applications are not allowed."
401 (missing or expired token, any endpoint)	"Your session has expired. Please log in again."
403 (wrong role or unowned resume)	"Only job seekers are authorized to apply for jobs."
Any other failure (404 job/resume not found, 400 job not `ACTIVE`)	The backend's own error message, or a generic fallback if none is given

Tracking page `MyApplicationsPage` (`/applications`) fetches the user's applications and renders one `ApplicationCard` per row, showing the posting, company, status badge, and application date. Withdrawal runs through a shared `ConfirmationDialog`; after the API succeeds, the card's status is updated locally to `WITHDRAWN` without refetching the whole list.

4.3.2 Employer Job Management UI

Post a job `PostJobPage` (`/employer/jobs/new`) collects the posting through `JobPostingForm`: title, description, requirements, location, a segmented employment-type control, and a deadline date picker. The deadline picker only allows dates from tomorrow onward, and on submit the helper `toDeadlineIso` converts the chosen calendar date into an ISO timestamp at the end of that day in the employer's local time zone, matching the backend rule that a deadline must be strictly in the future. After submission the page returns to the list, where the new posting is visible in `PENDING_APPROVAL`.

My job postings `MyJobPostingsPage` (`/employer/jobs`) lists the employer's postings from newest to oldest through `JobPostingCard`, which shows the summary fields and a `JobStatusBadge`. The close, re-open, and delete actions each require confirmation in a `ConfirmationDialog` with copy that reflects the real consequence of the action; on success the list is refreshed so the card reflects the new status.

4.3.3 Remaining Modules

The remaining pages follow the same conventions described in this chapter; they are covered briefly in Appendix C.

4.4 Summary

The platform was implemented as a two-application monorepo. The backend follows the controller-service architecture established in the design chapter: tsoa provides routing and the OpenAPI contract, services own all business rules, and Prisma provides typed database access. The two featured modules were implemented in full - the job seeker can apply, track, and withdraw applications, while the employer can manage a company, run the lifecycle of a job posting, and review its applications - and the remaining modules follow the same pattern, including the complete Back Office, where

job moderation, user management, and the audit log are served to administrators through a dedicated, role-guarded admin area. The frontend mirrors the backend with role-guarded pages and a generated, typed API client.

Chapter 5

Testing

This chapter reports how the implementation described in Chapter 4 was verified against the requirements established in Chapter 2. It states the testing strategy and why that strategy was chosen, describes the environment the tests run in, and presents a traceability matrix mapping each verified business rule back to the use case that specifies it. It closes with an explicit account of what was not tested.

5.1 Testing Strategy

The system's defining property is what it refuses to do, not what it does correctly: most use cases in Chapter 2 are constraints, such as an Employer managing only their own postings or a posting becoming visible only after moderation. A suite that exercised only the successful path would therefore prove little, since it would never attempt what the specification forbids. This follows the long-standing testing principle that test cases must be written for invalid and unexpected input conditions as well as valid ones [43]. Negative cases carry the evidential weight, and are cheap to add once a fixture exists.

Testing is concentrated at two levels, distinguishing tests of individual units in isolation from tests of units combined into a working whole [43].

- **Unit tests**: cover the recommendation algorithm described in Section 4. `scoreJob` is a pure function over plain objects, so its tests need no database, no HTTP layer and no fixtures; every input is visible in the test itself. This makes the algorithm's behaviour reproducible from the text of the test alone.
- **Integration tests**: drive the real Express application through its HTTP interface, exercising the full request pipeline against a real database. This is the level at which business rules actually live, since the services own every rule and every database access.

5.2 Test Environment

The suite uses Vitest [44] as the test runner and supertest [45] to issue HTTP requests against the Express application without binding a network port. Because `app.ts` exports the configured application separately from `server.ts`, which starts the listener and the expiry scheduler, the tests can load the application without starting any background work.

Three properties of the environment are worth recording, since each affects whether the results can be trusted.

- **The test database is derived, not configured.** The connection string is computed by appending the suffix `_test` to the application's own `DATABASE_URL`, so the database `mydb` yields `mydb_test`. The derivation is a safety measure rather than a convenience: the suite truncates every table before each test case, so a manually configured connection string pointing at the development database would destroy it.
- **The schema is built from the migrations.** Before the suite runs, `prisma migrate deploy` creates the test database and applies every migration in sequence. The tests therefore run against the same schema a deployment would produce, so a defective migration fails here rather than in production.
- **Each test case begins from an empty database.** Every table is truncated and the three role records re-inserted before each case, so a test can never pass or fail depending on what an earlier test left behind. For the same reason, test files run sequentially rather than in parallel, since they share one database.

Fixtures create database rows and issue authentication tokens directly rather than through the registration and login endpoints, so a test fails only when the behaviour it targets is actually broken, not because an unrelated setup endpoint has a defect. Those setup flows are covered by their own tests.

5.3 Requirements Traceability

Requirements traceability is the ability to describe and follow the life of a requirement in both a forwards and backwards direction [46]. Table 5.1 applies it backwards from the tests, mapping every business rule verified by the suite to the use case that specifies it. It is the primary evidence that the implementation satisfies the requirements: each row names a rule from Chapter 2, the test suite that checks it, and the outcome.

Table 5.1: Requirements Traceability Matrix.

Use Case	Rule or Behaviour Verified	Test Suite	Result
UC-PUB-01	Approved, open postings appear in public search	PublicJobAccess	Pass
UC-PUB-01	A posting awaiting moderation is hidden from search	PublicJobAccess	Pass
UC-PUB-01	A posting past its deadline is hidden even while its stored status is still `ACTIVE`	PublicJobAccess	Pass
UC-PUB-01	Keyword search matches against the job title	PublicJobAccess	Pass
UC-PUB-01	Job detail is refused for a lapsed or unapproved posting	PublicJobAccess	Pass

continued on next page

Table 5.1 — continued from previous page

Use Case	Rule or Behaviour Verified	Test Suite	Result
UC-PUB-02	A company profile lists only that company's open postings	PublicJobAccess	Pass
UC-AUTH-01	Registration creates an account holding the Job Seeker role	AuthenticationFlow	Pass
UC-AUTH-01	The password is stored as a bcrypt digest, never in plain text	AuthenticationFlow	Pass
UC-AUTH-01	An email address already registered is rejected	AuthenticationFlow	Pass
UC-AUTH-01	A request missing the password is rejected by validation	AuthenticationFlow	Pass
UC-AUTH-02	Correct credentials return an access token	AuthenticationFlow	Pass
UC-AUTH-02	An incorrect password is rejected	AuthenticationFlow	Pass
UC-AUTH-02	A banned account cannot log in	AuthenticationFlow	Pass
UC-AUTH-03	Logout requires authentication, so every logout has an actor	AuthenticationFlow	Pass
UC-JS-01	An application is stored as `SUBMITTED` against the chosen CV	JobSeekerFlow	Pass
UC-JS-01	A second application to the same posting is refused	JobSeekerFlow	Pass
UC-JS-01	A posting past its deadline accepts no applications	JobSeekerFlow	Pass
UC-JS-01	A posting not yet approved accepts no applications	JobSeekerFlow	Pass
UC-JS-02	A CV belonging to another user cannot be attached	JobSeekerFlow	Pass
UC-JS-03	A seeker sees their own applications and the postings they target	JobSeekerFlow	Pass
UC-JS-03	An application remains in history after its posting expires	JobSeekerFlow	Pass
UC-JS-03	A seeker never sees another seeker's applications	JobSeekerFlow	Pass
UC-JS-04	The owner may withdraw; status and timestamp are recorded	JobSeekerFlow	Pass
UC-JS-04	Another seeker's application cannot be withdrawn	JobSeekerFlow	Pass
UC-JS-04	An application cannot be withdrawn twice	JobSeekerFlow	Pass

continued on next page

Table 5.1 — continued from previous page

Use Case	Rule or Behaviour Verified	Test Suite	Result
UC-JS-05	Results are ranked against stated preferences, each carrying an explanation	JobSeekerFlow	Pass
UC-JS-05	A posting already applied to is never recommended	JobSeekerFlow	Pass
UC-JS-05	With no preferences stated, the newest postings are returned unranked	JobSeekerFlow	Pass
UC-JS-05	A seeker set to Not Looking receives an empty list	JobSeekerFlow	Pass
UC-JS-05	A posting matching nothing the seeker asked for is omitted	JobSeekerFlow	Pass
UC-JS-05	The scoring weights sum to 1, so every score lies in $[0, 1]$	RecommendationScoring	Pass
UC-JS-05	A posting with no explanation is never recommended, although it still scores	RecommendationScoring	Pass
UC-JS-05	A job located in the preferred city is distinguished from a company merely based there	RecommendationScoring	Pass
UC-JS-05	Freshness breaks ties but is never offered as a reason	RecommendationScoring	Pass
UC-EMP-01	A new posting is created as `PENDING_APPROVAL`, never `ACTIVE`	EmployerFlow	Pass
UC-EMP-01	A new posting is absent from public search until approved	EmployerFlow	Pass
UC-EMP-01	A deadline in the past is rejected	EmployerFlow	Pass
UC-EMP-01	An empty title is rejected	EmployerFlow	Pass
UC-EMP-01	An overdue `ACTIVE` posting is moved to `EXPIRED`	JobExpirySweep	Pass
UC-EMP-01	A posting whose deadline is still ahead is left unchanged	JobExpirySweep	Pass
UC-EMP-01	A posting awaiting moderation is never expired	JobExpirySweep	Pass
UC-EMP-01	A closed posting is never expired	JobExpirySweep	Pass
UC-EMP-01	Each expiry is recorded in the audit log with no actor	JobExpirySweep	Pass
UC-EMP-02	Only the caller's own postings are listed	EmployerFlow	Pass
UC-EMP-03	Another employer's posting can be neither viewed nor edited	EmployerFlow	Pass

continued on next page

Table 5.1 — continued from previous page

Use Case	Rule or Behaviour Verified	Test Suite	Result
UC-EMP-04	Closing removes the posting from public search	EmployerFlow	Pass
UC-EMP-04	A posting cannot be closed twice	EmployerFlow	Pass
UC-EMP-04	Re-opening is refused while the deadline lies in the past	EmployerFlow	Pass
UC-EMP-04	A re-opened posting returns to `PENDING_APPROVAL` for re-moderation	EmployerFlow	Pass
UC-EMP-03	An edited `ACTIVE` posting returns to moderation and is unpublished	EmployerFlow	Pass
UC-EMP-03	An edited `REJECTED` posting returns to moderation with its rejection cleared	EmployerFlow	Pass
UC-EMP-03	An `EXPIRED` posting cannot be edited	EmployerFlow	Pass
UC-EMP-05	An Employer may delete a posting awaiting moderation	EmployerFlow	Pass
UC-EMP-05	An Employer may delete an expired posting	EmployerFlow	Pass
UC-ADMIN-02	A Moderator may not delete a posting awaiting moderation	BackOfficeFlow	Pass
UC-ADMIN-02	A Moderator may delete a published posting	BackOfficeFlow	Pass
UC-ADMIN-01	Approval publishes the posting and records the approval time	BackOfficeFlow	Pass
UC-ADMIN-01	A posting becomes publicly visible only after approval	BackOfficeFlow	Pass
UC-ADMIN-01	Approval writes exactly one audit entry, attributed to the moderator	BackOfficeFlow	Pass
UC-ADMIN-01	Rejection stores the reason and leaves the posting unpublished	BackOfficeFlow	Pass
UC-ADMIN-04	A ban takes effect on a token the user already holds	BackOfficeFlow	Pass
UC-ADMIN-04	Unbanning restores access	BackOfficeFlow	Pass

continued on next page

Table 5.1 — continued from previous page

Use Case	Rule or Behaviour Verified	Test Suite	Result
UC-ADMIN-05	The audit trail is returned to a moderator	BackOfficeFlow	Pass
Cross-cutting	An unauthenticated request to a protected route is refused	PublicJobAccess	Pass
Cross-cutting	A malformed token is refused	PublicJobAccess	Pass
Cross-cutting	A Job Seeker cannot reach Employer or Back-Office endpoints	JobSeekerFlow	Pass
Cross-cutting	An Employer cannot reach Job Seeker or Back-Office endpoints	EmployerFlow	Pass
Cross-cutting	Neither a Job Seeker nor an Employer can read the audit log	BackOfficeFlow	Pass

5.4 Results

The suite comprises 89 test cases across seven files and completes in approximately 30 seconds. Table 5.2 summarises the outcome by suite.

Table 5.2: Test Execution Summary.

Test Suite	Use Cases	Cases	Passed	Failed
RecommendationScoring	UC-JS-05	15	15	0
PublicJobAccess	UC-PUB-01, UC-PUB-02	10	10	0
AuthenticationFlow	UC-AUTH-01 to 03	8	8	0
JobSeekerFlow	UC-JS-01 to 05	18	18	0
EmployerFlow	UC-EMP-01 to 05	20	20	0
BackOfficeFlow	UC-ADMIN-01, 02, 04, 05	11	11	0
JobExpirySweep	UC-EMP-01	7	7	0
Total		**89**	**89**	**0**

Two behaviours deserve separate mention, because the suite is the only place either can be observed.

The first is job expiry. It cannot be demonstrated interactively, since it requires waiting for a real deadline to elapse; the test instead backdates a posting and invokes the sweep directly, confirming both the status change and its audit entry. The sweep's limits are deliberate: a posting awaiting moderation or already closed is left untouched, since a moderator cannot approve a lapsed posting and an expired posting cannot be edited to carry a new deadline.

The second is account banning. A ban must take effect on the access token the user is already holding, not merely at their next login attempt. The test confirms that

a request succeeding before the ban is refused immediately afterwards using the same token, which verifies that the check is performed during authentication rather than at login.

5.5 Limitations

The following were not covered by automated testing, and are recorded here rather than left implicit.

- **The UI is untested**: No automated test exercises the React application. Interface behaviour was verified manually.
- **Several use cases have no automated coverage**: These are UC-AUTH-04 (Forgot Password), UC-PROFILE-01 to UC-PROFILE-06, UC-EMP-00 (Create Company Profile), UC-EMP-06 to UC-EMP-08 (application review and status updates) and UC-ADMIN-03 (View Users). The password-reset flow was excluded deliberately, since testing it would require dispatching real electronic mail; the remainder were omitted for reasons of time.
- **File upload is not exercised end to end**: Fixtures create CV records directly without writing files to disk, so the storage layer is not covered.
- **Concurrency and load were not tested**: Every test runs against a single-user database with no contention, so behaviour under simultaneous access is unverified.
- **Only status codes are asserted, not error bodies**: The application registers no error-handling middleware, so errors are rendered by the framework's default handler rather than as structured responses. There is consequently no error contract for the tests to verify.

Chapter 6

Conclusion

6.1 Summary

This thesis designed and implemented an online recruitment platform that brings job seeking and hiring into a single system. The work centered on two flows. On the job seeker's side, the Job Application module lets a candidate apply for an active posting using one of their resumes, track each application through its lifecycle, and withdraw an application that has not yet been decided. On the employer's side, the Employer Job Management module lets an employer maintain a company profile, publish and manage job postings through their lifecycle, and review the applications each posting receives until a decision is recorded. Around these, the platform provides authentication with role-based access control, personal profiles and resume management, public job search, and back-office moderation.

The system was built as a pnpm monorepo with two applications. The backend, `apps/api`, is an Express service written in TypeScript: tsoa generates the OpenAPI contract and the route handlers from annotated controllers, services own every business rule, and Prisma provides typed database access. The frontend, `apps/web`, is a React application whose typed API client is generated from that same contract, so the two sides can never drift in the shape of the data they exchange. The two featured modules are described in full, and all supporting modules follow the same controller - service pattern.

The project met its stated objectives. The iterative loop described in the introduction kept the documentation honest: every use case that surfaced during implementation was written back into the analysis, so the design and the code never fell out of step.

6.2 Limitations

Due to time and knowledge constraints, the final product presents several limitations:

- The platform is a prototype run as two local applications. It has not been deployed, and it has not been exercised under real-scale traffic or by real users.
- The thesis details two of the modules; the remaining modules are built and described, but only briefly.

6.3 Future Work

The limitations above point to the most natural directions for future work:

- Notify users when something changes - job seekers when an application status updates, employers when a posting is moderated.
- Add a direct communication channel between employers and applicants inside the platform.
- Prepare the platform for deployment and scale: containerization, load balancing, and caching of search results.
- Build a mobile application.

Bibliography

[1] D. H. Autor, "Wiring the labor market," *Journal of Economic Perspectives*, vol. 15, no. 1, pp. 25–40, 2001. [Online]. Available: https://doi.org/10.1257/jep.15.1.25

[2] D. S. Chapman and J. Webster, "The use of technologies in the recruiting, screening, and selection processes for job candidates," *International Journal of Selection and Assessment*, vol. 11, no. 2–3, pp. 113–120, 2003. [Online]. Available: https://doi.org/10.1111/1468-2389.00234

[3] A. B. Holm, "E-recruitment: Towards an ubiquitous recruitment process and candidate relationship management," *German Journal of Human Resource Management: Zeitschrift für Personalforschung*, vol. 26, no. 3, pp. 241–259, 2012. [Online]. Available: https://doi.org/10.1177/239700221202600303

[4] P. Kuhn and H. Mansour, "Is internet job search still ineffective?" *The Economic Journal*, vol. 124, no. 581, pp. 1213–1233, 2014. [Online]. Available: https://doi.org/10.1111/ecoj.12119

[5] National Statistics Office of Vietnam, "Report on labour force survey 2024," 2024. [Online]. Available: https://www.nso.gov.vn/en/default/2026/05/report-on-labour-force-survey-2024

[6] TopCV Vietnam, "2023 recruitment market and 2024 recruitment need report," 2023. [Online]. Available: https://insights.topcv.vn/recruitment-report-2023-2024-eng

[7] Michael Page Vietnam, "Talent trends 2023: The invisible revolution," 2023. [Online]. Available: https://vietnamnews.vn/media-outreach/1551888/56-of-vietnam-in-search-of-new-job-opportunities-a-bright-future-ahead.html

[8] AMS and The Josh Bersin Company, "New research shows that hiring is harder than ever: Time to hire increasing significantly for almost all roles," 2023. [Online]. Available: https://www.weareams.com/news/new-research-shows-that-hiring-is-harder-than-ever-time-to-hire-increasing-significantly-for-aln

[9] I. Jacobson, G. Booch, and J. Rumbaugh, *The Unified Software Development Process*. Reading, MA, USA: Addison-Wesley, 1999.

[10] I. Sommerville, *Software Engineering*, 10th ed. Boston, MA, USA: Pearson, 2016.

[11] International Organization for Standardization, "ISO/IEC 25010:2011 – systems and software engineering – systems and software quality requirements and evaluation (SQuaRE) – system and software quality models," ISO, Geneva, Switzerland, 2011. [Online]. Available: https://www.iso.org/standard/35733.html

[12] N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proceedings of the 1999 USENIX Annual Technical Conference, FREENIX Track*, Monterey, CA, USA, 1999, pp. 81–91. [Online]. Available: https://www.usenix.org/legacy/event/usenix99/provos/provos.pdf

[13] OWASP Foundation, "Password storage cheat sheet," OWASP Cheat Sheet Series, accessed: Oct. 23, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

[14] I. Jacobson, M. Christerson, P. Jonsson, and G. Övergaard, *Object-Oriented Software Engineering: A Use Case Driven Approach*. New York, NY, USA: ACM Press / Addison-Wesley, 1992.

[15] Object Management Group, "OMG unified modeling language (OMG UML), version 2.5.1," OMG Document formal/17-12-05, 2017. [Online]. Available: https://www.omg.org/spec/UML/2.5.1/

[16] A. Cockburn, *Writing Effective Use Cases*. Boston, MA, USA: Addison-Wesley, 2000.

[17] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA, USA: Addison-Wesley, 2002.

[18] D. L. Parnas, "On the criteria to be used in decomposing systems into modules," *Communications of the ACM*, vol. 15, no. 12, pp. 1053–1058, 1972. [Online]. Available: https://doi.org/10.1145/361598.361623

[19] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975. [Online]. Available: https://doi.org/10.1109/PROC.1975.9939

[20] E. F. Codd, "A relational model of data for large shared data banks," *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, 1970. [Online]. Available: https://doi.org/10.1145/362384.362685

[21] S. Brown, "The C4 model for visualising software architecture," c4model.com, accessed: Oct. 23, 2026. [Online]. Available: https://c4model.com/

[22] Meta Open Source, "React documentation," accessed: Oct. 23, 2026. [Online]. Available: https://react.dev/

[23] OpenJS Foundation, "Node.js documentation," accessed: Oct. 23, 2026. [Online]. Available: https://nodejs.org/docs/latest/api/

[24] ——, "Express – fast, unopinionated, minimalist web framework for Node.js," accessed: Oct. 23, 2026. [Online]. Available: https://expressjs.com/

[25] Microsoft, "TypeScript documentation," accessed: Oct. 23, 2026. [Online]. Available: https://www.typescriptlang.org/docs/

[26] Prisma Data, Inc., "Prisma ORM documentation," accessed: Oct. 23, 2026. [Online]. Available: https://www.prisma.io/docs

[27] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, University of California, Irvine, Irvine, CA, USA, 2000. [Online]. Available: https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

[28] The PostgreSQL Global Development Group, “PostgreSQL documentation,” accessed: Oct. 23, 2026. [Online]. Available: https://www.postgresql.org/docs/

[29] R. T. Fielding, M. Nottingham, and J. Reschke, “HTTP semantics,” RFC 9110, Internet Engineering Task Force, 2022. [Online]. Available: https://doi.org/10.17487/RFC9110

[30] T. Bray, “The JavaScript object notation (JSON) data interchange format,” RFC 8259, Internet Engineering Task Force, 2017. [Online]. Available: https://doi.org/10.17487/RFC8259

[31] P. P.-S. Chen, “The entity-relationship model – toward a unified view of data,” *ACM Transactions on Database Systems*, vol. 1, no. 1, pp. 9–36, 1976. [Online]. Available: https://doi.org/10.1145/320434.320440

[32] M. B. Jones, J. Bradley, and N. Sakimura, “JSON web token (JWT),” RFC 7519, Internet Engineering Task Force, 2015. [Online]. Available: https://doi.org/10.17487/RFC7519

[33] Joint Task Force, “Security and privacy controls for information systems and organizations,” National Institute of Standards and Technology, Gaithersburg, MD, USA, NIST Special Publication 800-53 Rev. 5, 2020. [Online]. Available: https://doi.org/10.6028/NIST.SP.800-53r5

[34] Nodemailer, “Nodemailer documentation,” accessed: Oct. 23, 2026. [Online]. Available: https://nodemailer.com/

[35] shadcn, “shadcn/ui documentation,” accessed: Oct. 23, 2026. [Online]. Available: https://ui.shadcn.com/docs

[36] Tailwind Labs, “Tailwind CSS documentation,” accessed: Oct. 23, 2026. [Online]. Available: https://tailwindcss.com/docs

[37] pnpm, “pnpm – fast, disk space efficient package manager,” accessed: Oct. 23, 2026. [Online]. Available: https://pnpm.io/

[38] R. Potvin and J. Levenberg, “Why Google stores billions of lines of code in a single repository,” *Communications of the ACM*, vol. 59, no. 7, pp. 78–87, 2016. [Online]. Available: https://doi.org/10.1145/2854146

[39] tsoa community, “tsoa documentation,” accessed: Oct. 23, 2026. [Online]. Available: https://tsoa-community.github.io/docs/

[40] OpenAPI Initiative, “OpenAPI specification,” accessed: Oct. 23, 2026. [Online]. Available: https://spec.openapis.org/oas/latest.html

[41] Vite, “Vite documentation,” accessed: Oct. 23, 2026. [Online]. Available: https://vite.dev/

[42] Remix Software, "React router documentation," accessed: Oct. 23, 2026. [Online]. Available: https://reactrouter.com/

[43] G. J. Myers, C. Sandler, and T. Badgett, *The Art of Software Testing*, 3rd ed. Hoboken, NJ, USA: John Wiley & Sons, 2011.

[44] Vitest, "Vitest – next generation testing framework," accessed: Oct. 23, 2026. [Online]. Available: https://vitest.dev/

[45] ladjs, "Supertest: Super-agent driven library for testing HTTP servers," GitHub repository, accessed: Oct. 23, 2026. [Online]. Available: https://github.com/ladjs/supertest

[46] O. C. Z. Gotel and A. C. W. Finkelstein, "An analysis of the requirements traceability problem," in *Proceedings of the First IEEE International Conference on Requirements Engineering (ICRE '94)*, Colorado Springs, CO, USA, 1994, pp. 94–101. [Online]. Available: https://doi.org/10.1109/ICRE.1994.292398

Appendix A

Supplementary Use Case Specifications

A.1 Guest: Login

Table A.1: Use Case Specification: Login.

Field	Specification
Use Case ID	UC-AUTH-02
Use Case Name	Login
Primary Actor	Guest
Description	Allows a guest to authenticate using registered credentials and become an authenticated user.
Goal	Authenticate a registered user and grant access to protected system functions.
Preconditions	• The user has a registered account. • The user is not currently authenticated.
Trigger	The user clicks "Log In" button from the website's header

Table A.1 (Continued)

Field	Specification
Main Success Scenario	1. Guest selects "Login". 2. System displays the login form. 3. Guest enters email and password. 4. Guest submits the login form. 5. System validates the submitted credentials. 6. System authenticates the user. 7. System establishes an authenticated session. 8. System redirects the user to the authenticated homepage. 9. System displays a successful login message.
Alternative Flows	**A1. Invalid credentials** 1. The system reports the error "Invalid email or password". **A2. Invalid email address** 1. The system reports the error "Invalid email". **A3. Missing required input fields** 1. The system reports the error "Please fill out required fields". **A4. Account is suspended** 1. System denies authentication. 2. System informs the user that the account has been suspended.
Postconditions	• The user is successfully authenticated. • The user gains access to functions available to authenticated users.

A.2 Authenticated User: Update Personal Information

Table A.2: Use Case Specification: Update Personal Information.

Field	**Specification**
Use Case ID	UC-PROFILE-01
Use Case Name	Update Personal Information
Primary Actor	Authenticated User
Goal	Maintain accurate personal information within the platform.
Description	Allows an authenticated user to update personal profile information.
Preconditions	• User is authenticated.
Trigger	The user selects the Profile page.
Main Success Scenario	1. User opens the profile page. 2. System displays current profile information. 3. User edits profile information. 4. User submits the changes. 5. System validates the submitted information. 6. System updates the profile. 7. System confirms the successful update.
Alternative Flows	**A1. Invalid input** 1. System highlights invalid fields. 2. User corrects the information.
Postconditions	• Updated profile information is stored.

A.3 Back Office Worker: Approve Job Posting

Table A.3: Use Case Specification: Approve Job Posting.

Field	Specification
Use Case ID	UC-ADMIN-01
Use Case Name	Approve Job Posting
Primary Actor	Back Office Worker
Goal	Review and approve job postings before they become publicly available.
Description	Allows a Back Office Worker to review submitted job postings and determine whether they comply with platform policies.
Preconditions	• Back Office Worker is authenticated. • A job posting is awaiting approval.
Trigger	Administrator opens the pending job posting list.
Main Success Scenario	1. Back Office Worker reviews the job posting. 2. Back Office Worker verifies the submitted information. 3. Back Office Worker approves the posting. 4. System updates the posting status. 5. System makes the job posting publicly available.
Alternative Flows	**A1. Job posting rejected** 1. Administrator rejects the posting. 2. System records the decision. 3. System notifies the Employer.
Postconditions	• The job posting is approved or rejected.

Appendix B

Supplementary Entity Relationship Groups

The entity relationship groups that support the remaining modules are detailed here for reference.

B.1 Auth and Account

Figure B.1 details the entities involved in authentication and account recovery.

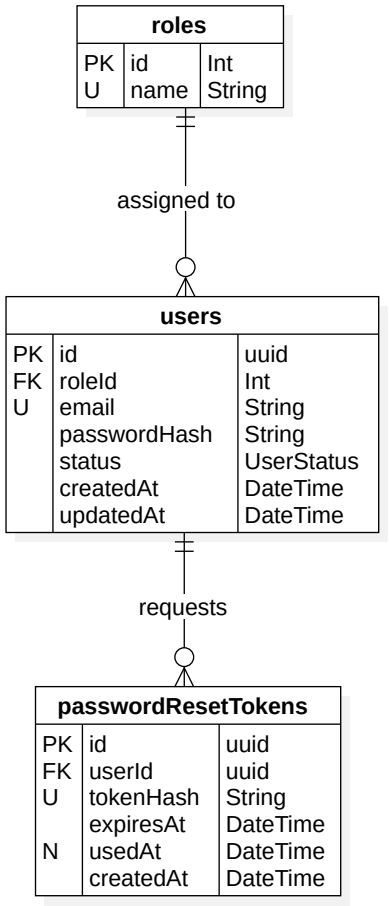


Figure B.1: Auth and Account Entity Relationship Diagram.

Each role may be assigned to any number of users, while every user is assigned exactly one role. A user may request multiple password reset tokens over time, but each token belongs to exactly one user.

B.2 Profile and CV Management

Figure B.2 details the entities involved in managing a user's profile, job preferences, and resumes.

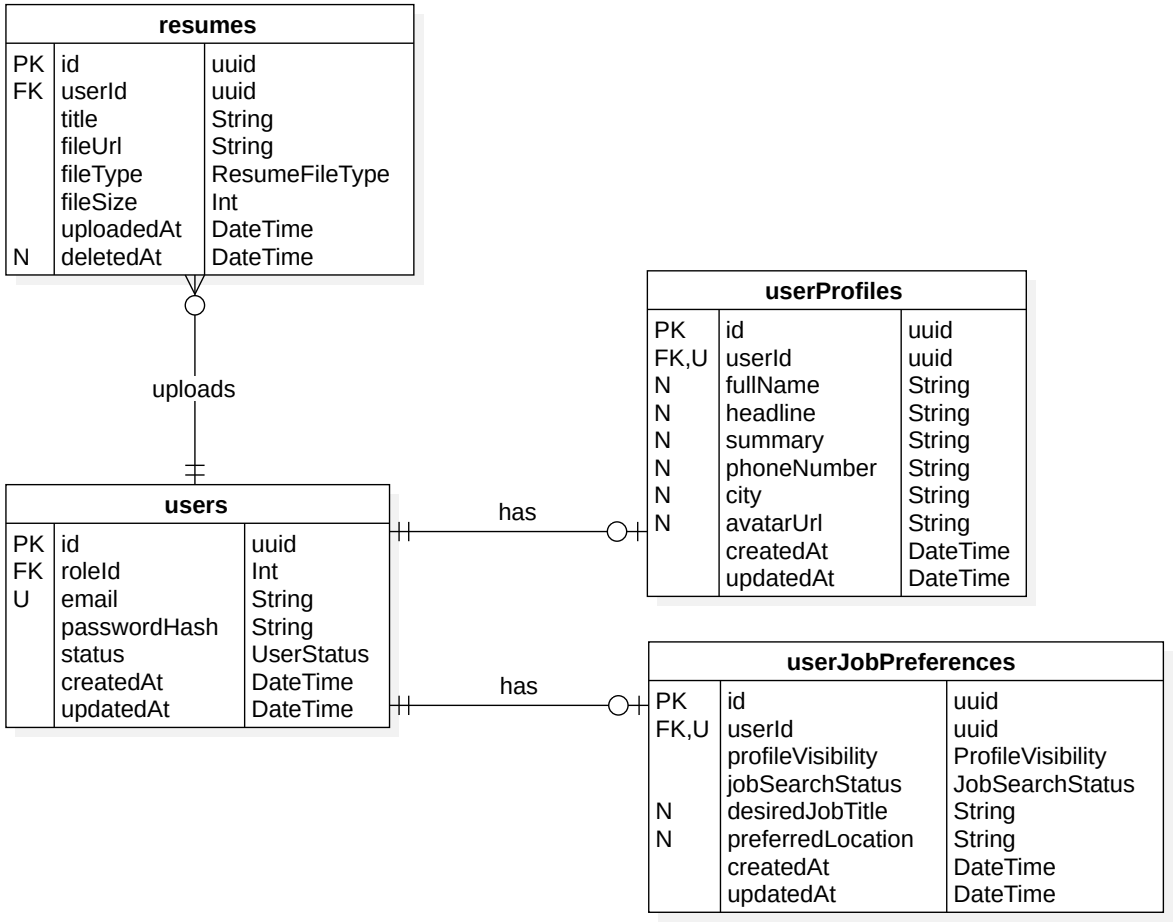


Figure B.2: Profile and CV Management Entity Relationship Diagram.

Each user has at most one profile and at most one job preference record, both enforced through a unique foreign key, forming one-to-one relationships. A user may upload multiple resumes, but each resume belongs to exactly one user.

B.3 Administration and Audit

Figure B.3 details the entities involved in role assignment and system activity logging.

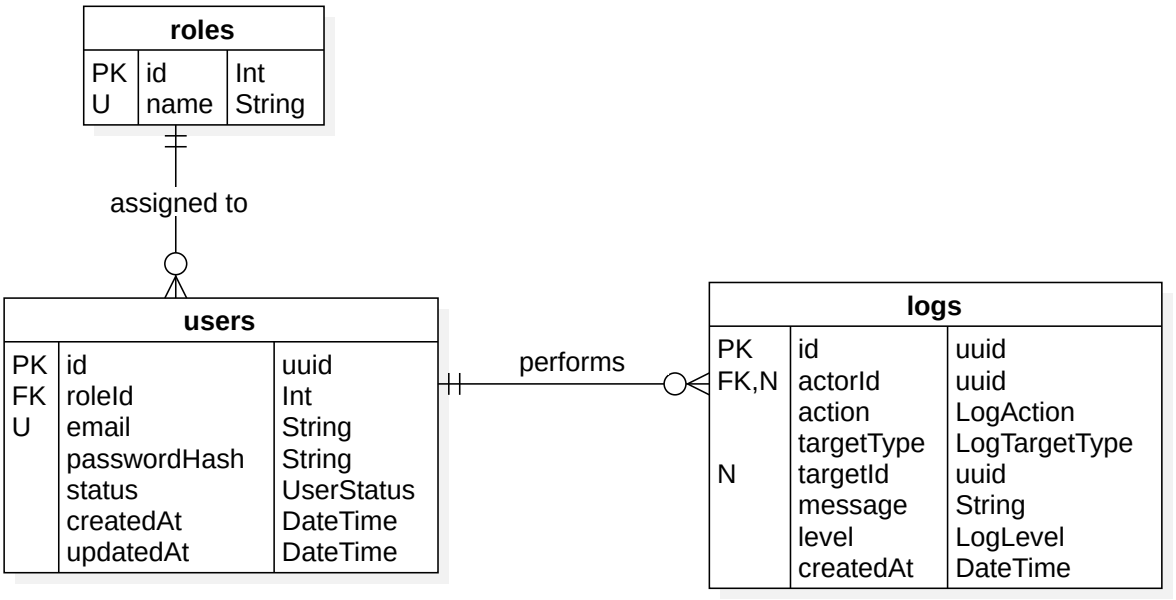


Figure B.3: Administration and Audit Entity Relationship Diagram.

As with the Auth and Account group, each role may be assigned to any number of users, while every user is assigned exactly one role. A user may perform any number of logged actions, and each log entry optionally references the user who performed it, since some actions may be attributed to the system itself rather than a specific user.

Appendix C

Supplementary Module Designs

This chapter collects the class-level design of the modules that are not featured in the main body. Each module follows the same controller-service pattern described in Chapter 3; as with the featured modules, each Controller delegates directly to its Service, which owns all business logic and Prisma access.

C.1 Authentication Module

The Authentication Module implements user registration, login, logout, and password recovery. It resides in `api-external`, since these operations are invoked directly by external clients, and is the module other modules depend on for issuing and validating access tokens.

Figure C.1 presents the class diagram of the Authentication Module.

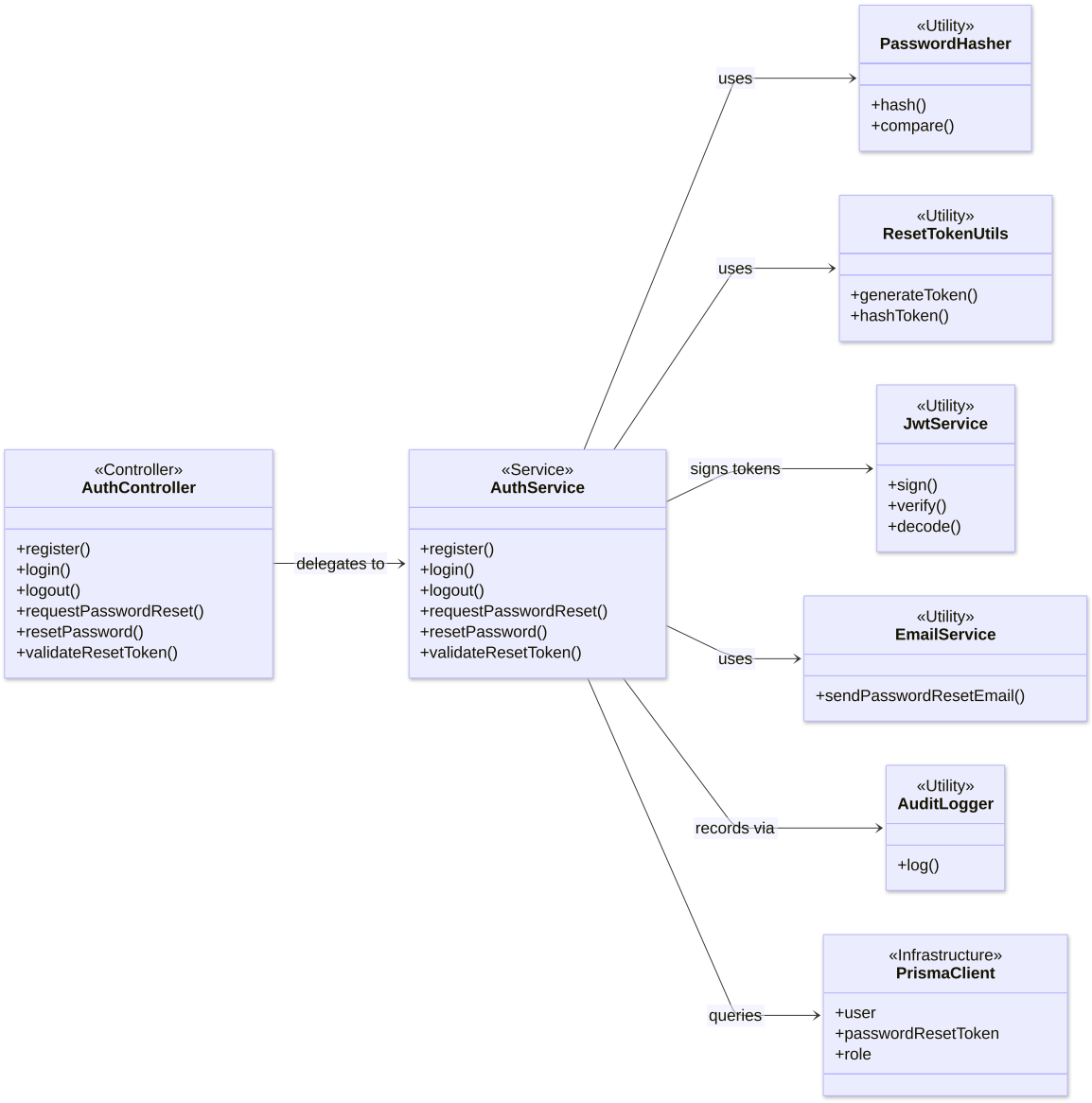


Figure C.1: Authentication Module Class Diagram.

AuthController exposes the authentication endpoints and delegates each directly to a matching method on AuthService. AuthService depends on the shared utility components: PasswordHasher, ResetTokenUtils, EmailService, and Prisma Client. Each AuthService method implements one authentication use case. register validates that the email is not already in use, assigns the default `JOB_SEEKER` role, and stores the hashed password. login verifies the submitted password against the stored hash, rejects banned accounts, and issues a signed JWT containing the user's identifier and role. logout returns a simple confirmation message, since JWTs are stateless and token discarding is handled by the frontend. requestPasswordReset issues a hashed, time-limited reset token and emails the corresponding raw token to the user; the same response is returned whether or not the account exists, so the email address is not revealed. resetPassword validates the submitted token against the stored hash and expiration, then updates the password and marks the token as used within a single transaction. validateResetToken lets the frontend check whether a reset token is still valid without consuming it.

C.2 Profile Management Module

The Profile Management Module lets authenticated users view and update their personal profile, job search preferences, avatar, and account password. It resides in `api-external` and depends on the JWT middleware to identify the requesting user on every operation.

Figure C.2 presents the class diagram of the Profile Management Module.

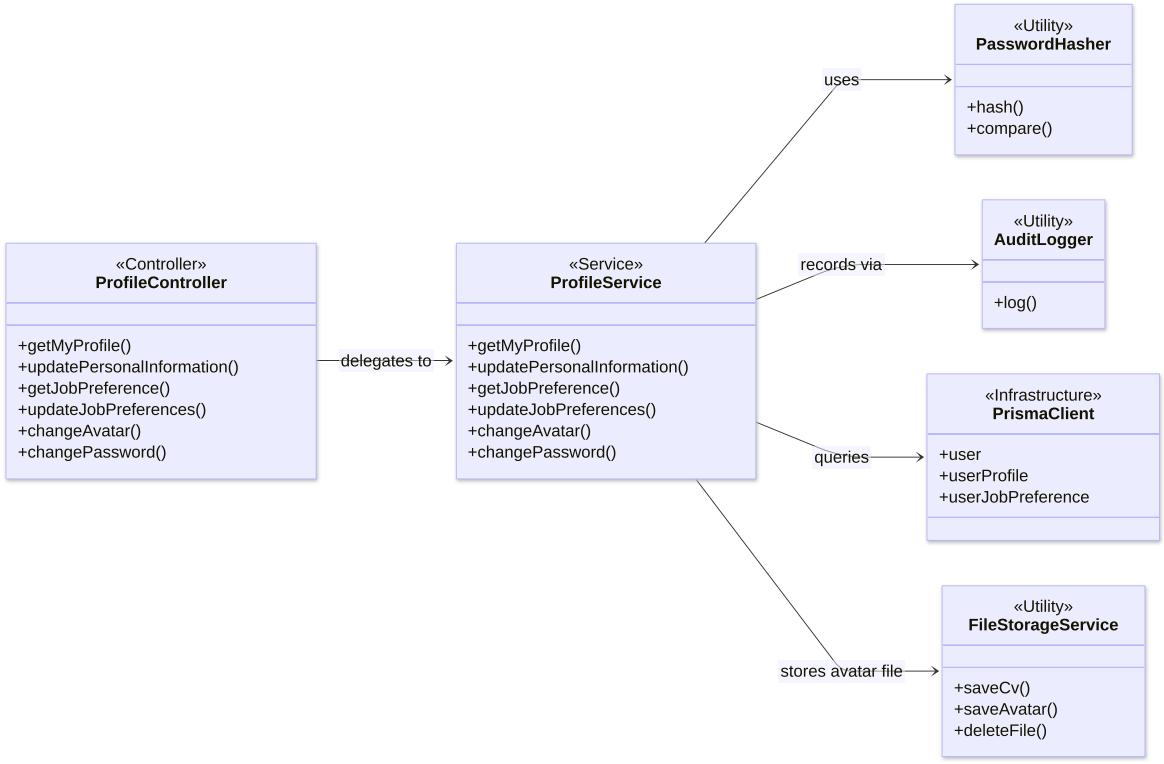


Figure C.2: Profile Management Module Class Diagram.

ProfileController delegates each of its endpoints directly to a method on ProfileService. ProfileService depends on PasswordHasher, FileStorageService, and Prisma Client. getMyProfile and getJobPreference return the authenticated user's records, selecting only fields safe for display. updatePersonalInformation and updateJobPreference use an upsert operation, creating the record on first use and updating it thereafter; updateJobPreference additionally distinguishes an omitted field (left unchanged) from a field explicitly set to `null` (cleared). changeAvatar validates the uploaded file's type and size before storing it. changePassword verifies the submitted current password, rejects the request if the new password is identical to the old one, and otherwise hashes and stores the new password.

C.3 Resume Management Module

The Resume Management Module lets job seekers list, upload, and delete their resumes, which are later selected when submitting job applications. It resides in `api-external` and depends on the JWT middleware.

Figure C.3 presents the class diagram of the Resume Management Module.

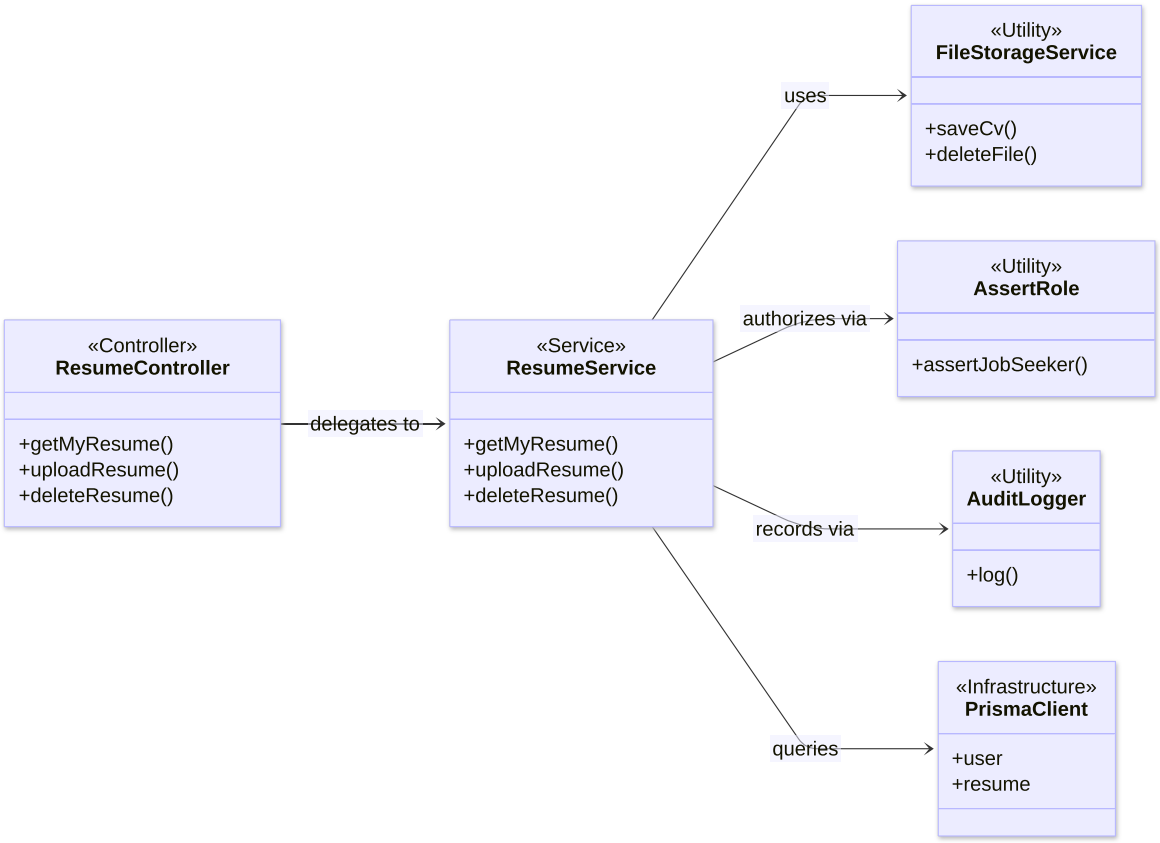


Figure C.3: Resume Management Module Class Diagram.

ResumeController delegates its endpoints to ResumeService, which depends on FileStorageService and Prisma Client. getMyResumes returns the authenticated user's non-deleted resumes, ordered from newest to oldest. uploadResume validates that a title and file are provided, rejects files larger than 5 MB or of an unsupported type (only PDF, DOC, and DOCX are accepted), stores the file through FileStorageService, then creates the resume record. deleteResume verifies that the resume exists, is not already deleted, and belongs to the requesting user, then checks whether it is referenced by any application. If it is, the resume is soft-deleted so existing applications retain a valid reference; otherwise it is hard-deleted from both the database and file storage.

C.4 Job Discovery Module

The Job Discovery Module lets guests and authenticated users search active job postings and view their details. Unlike the previous modules, its endpoints are not protected by the JWT middleware, since job browsing is a public capability. It also includes the public-facing company profile and active job listing endpoints operated by `CompanyController` and `CompanyService`.

Figure C.4 presents the Public class diagram, which covers both the job discovery and company profile viewing capabilities.

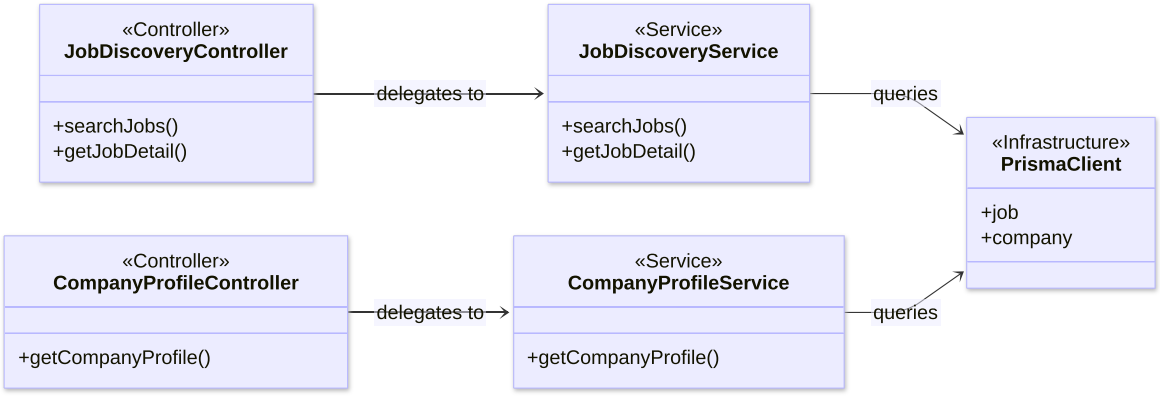


Figure C.4: Public Class Diagram.

JobDiscoveryService depends on the shared Prisma Client, which queries the Job and Company entities. searchJobs restricts results to `ACTIVE` jobs and accepts an optional keyword and location, either of which may be provided independently or together. An empty string is treated as not provided, each value is capped at 255 characters, and the keyword matches against the job title or company name while the location matches against the job location or the company's city and district, case-insensitively. Neither is required: if no keyword is given, all active jobs are returned, and a query that matches nothing returns an empty list rather than an error. getJobDetail returns the full detail of a single active job by identifier, with a 404 error if the job does not exist or is not currently active.

C.5 Back Office Module

The Back Office module is hosted under `api-internal`. Figures C.5, C.6, and C.7 present its class diagrams.

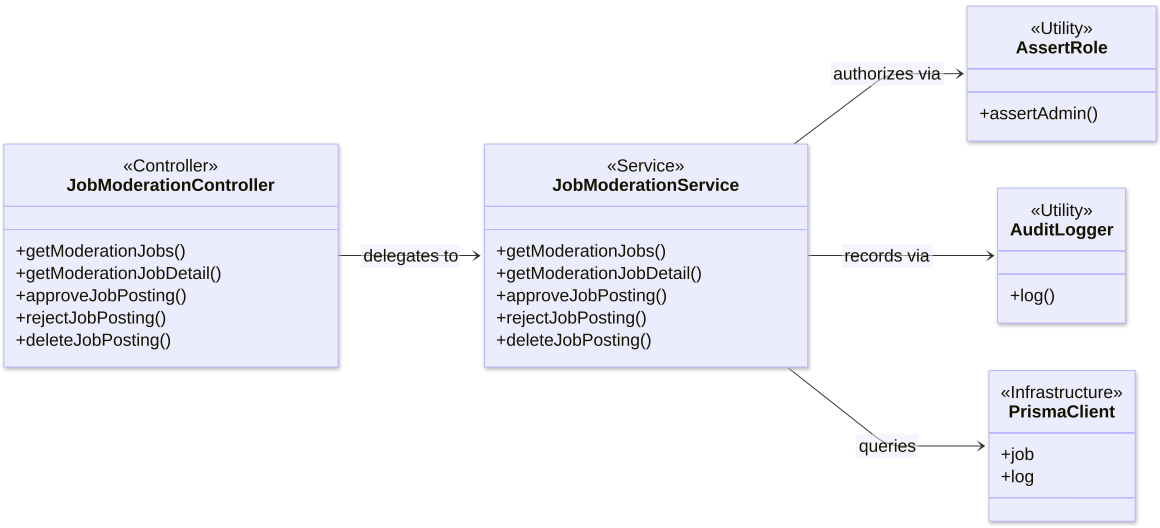


Figure C.5: Job Moderation Module Class Diagram.

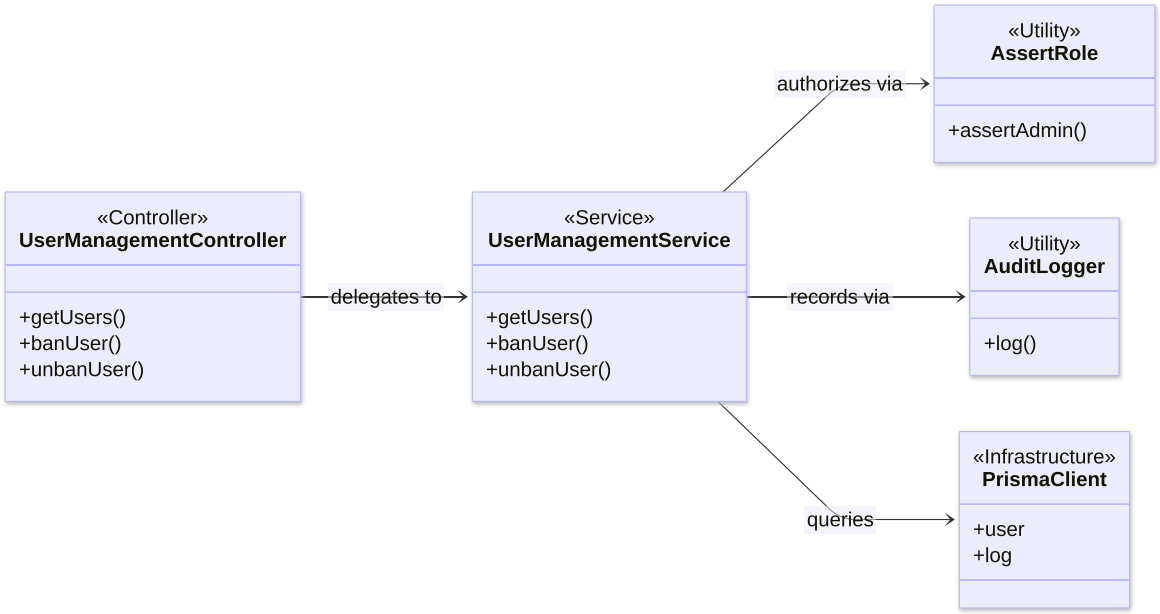


Figure C.6: User Management Module Class Diagram.

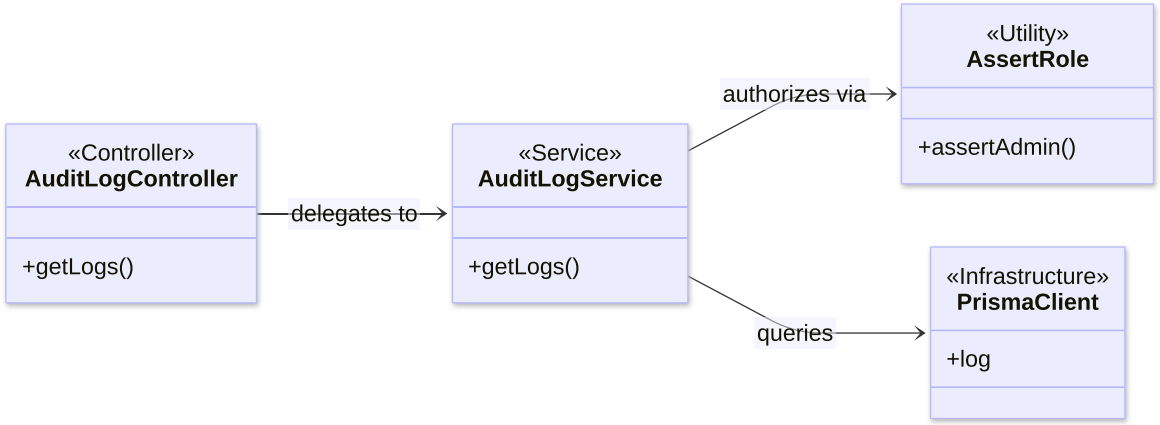


Figure C.7: Audit Log Module Class Diagram.

`JobModerationController` delegates to `JobModerationService`: `getModerationJobs` lists postings with an optional status filter, `approveJobPosting` moves a posting to `ACTIVE`, and `rejectJobPosting` moves it to `REJECTED` with a required reason capped at 255 characters; `deleteJobPosting` removes any posting that is not already `DELETED`. `UserManagementController` delegates to `UserManagementService`: `getUsers` searches accounts by a case-insensitive, partial email match, and an administrator can ban or unban an account, a status the JWT middleware re-checks on every subsequent protected request. `AuditLogController` delegates to `AuditLogService`, which returns the 50 most recent log entries, newest first, with no filtering. All three services depend on the shared Prisma Client and record their mutations through `AuditLogger`.

C.5.1 Back Office Worker UI

On the presentation side, the back office is realized as a role-guarded route group under `/admin` that the Application Shell exposes only to users with the `ADMIN` role.

`AdminLayout` provides a persistent sidebar that navigates between three page areas mirroring the modules above: Job Moderation, a list of postings filterable by status and defaulting to `PENDING_APPROVAL`, with a detail page carrying the approve, reject, and delete actions; Users, a searchable account list with a detail page offering ban and unban actions; and Audit Logs, a read-only listing of administrative actions.

Appendix D

Sequence Diagrams

This chapter collects the sequence diagrams of the platform's use cases as reference material. They illustrate the interaction between the actor, the frontend, and the backend service for each flow, and supplement the design discussion in Chapter 3 and the implementation discussion in Chapter 4.

D.1 Job Application

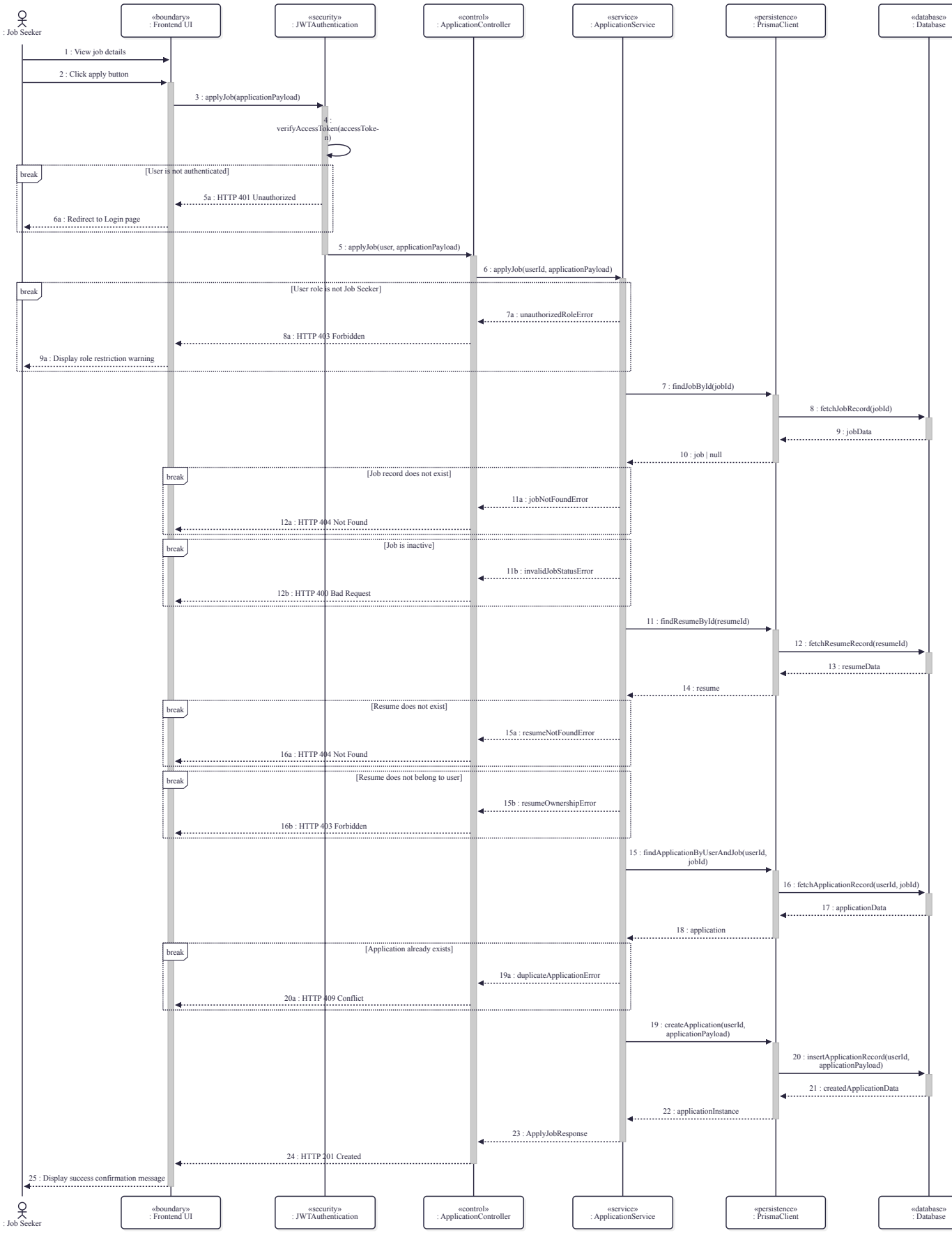


Figure D.1: Sequence Diagram: Apply for Jobs.

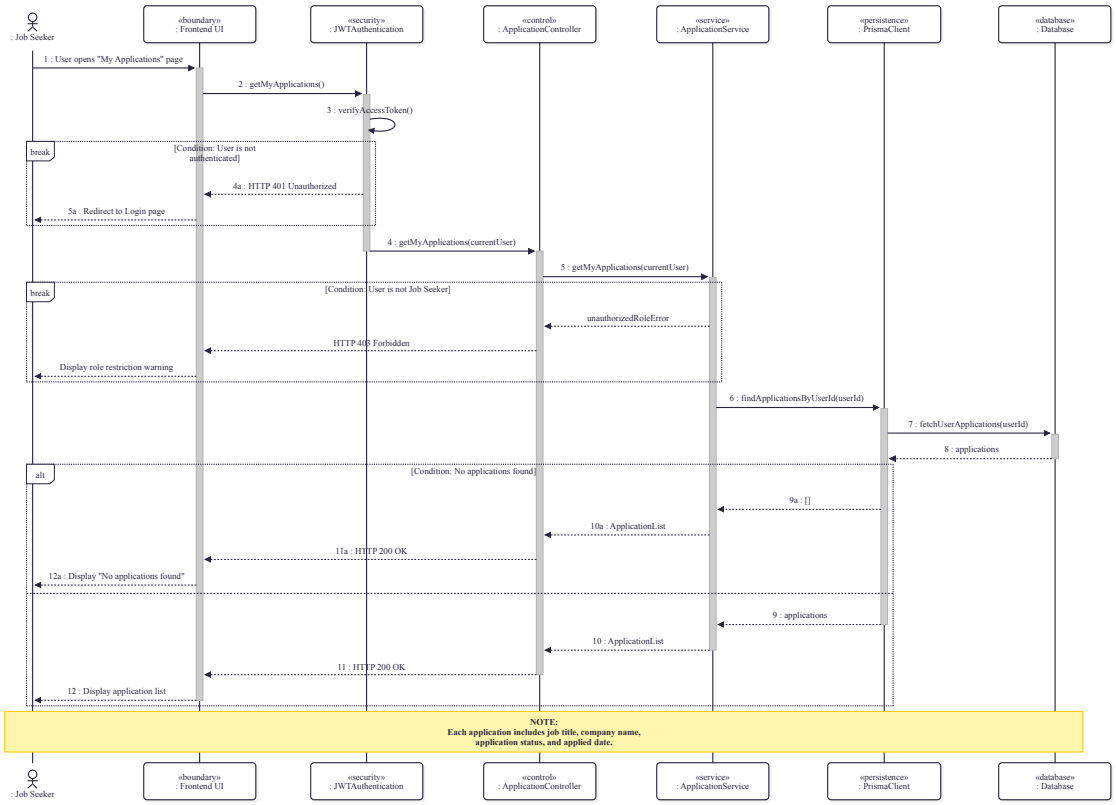


Figure D.2: Sequence Diagram: Track Applications.

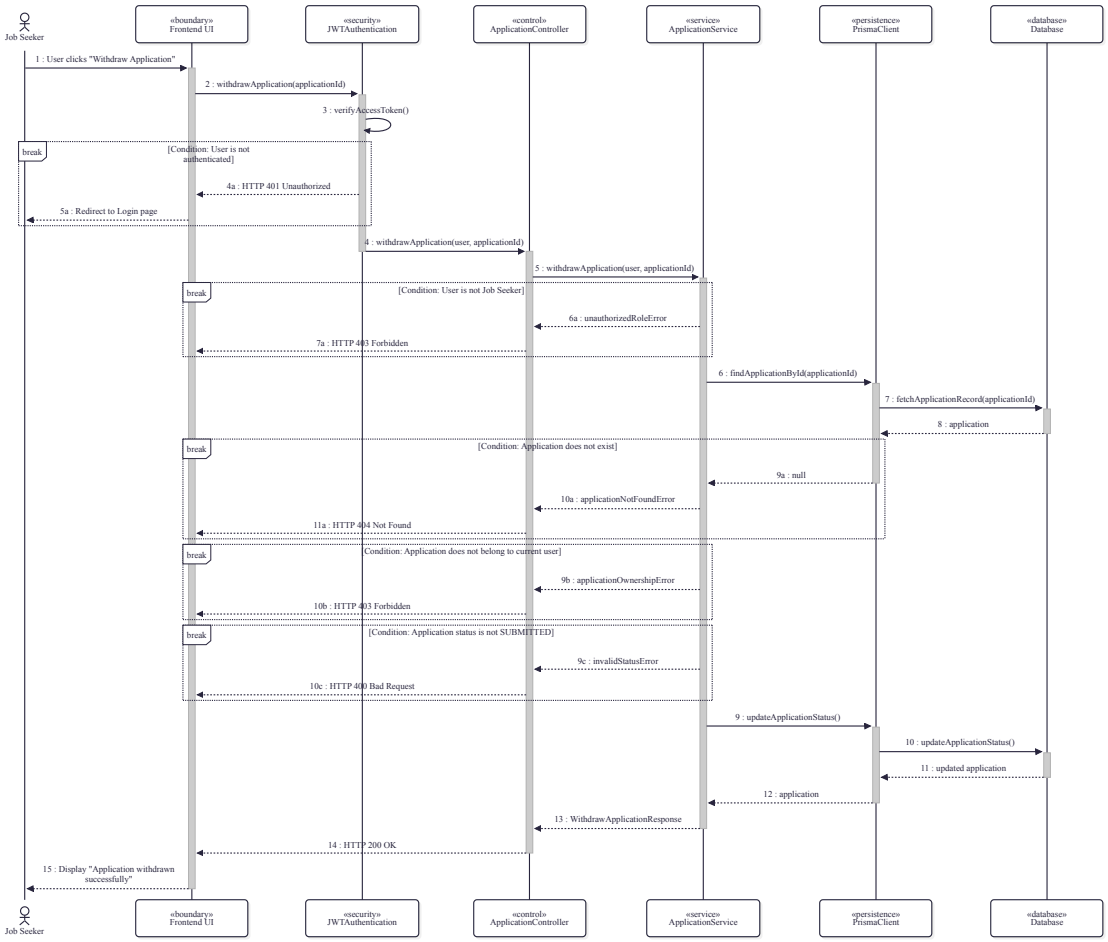


Figure D.3: Sequence Diagram: Cancel Applications.

D.2 Employer Job Management

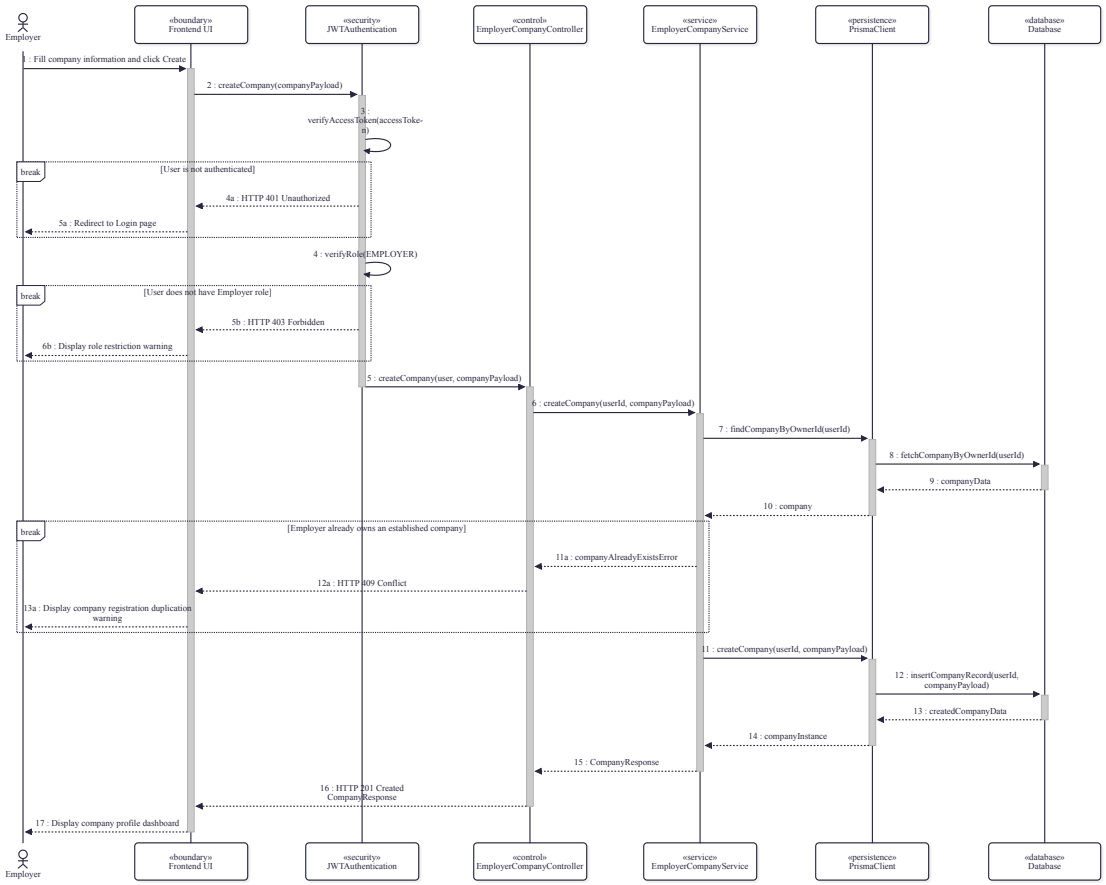


Figure D.4: Sequence Diagram: Create Company Profile.

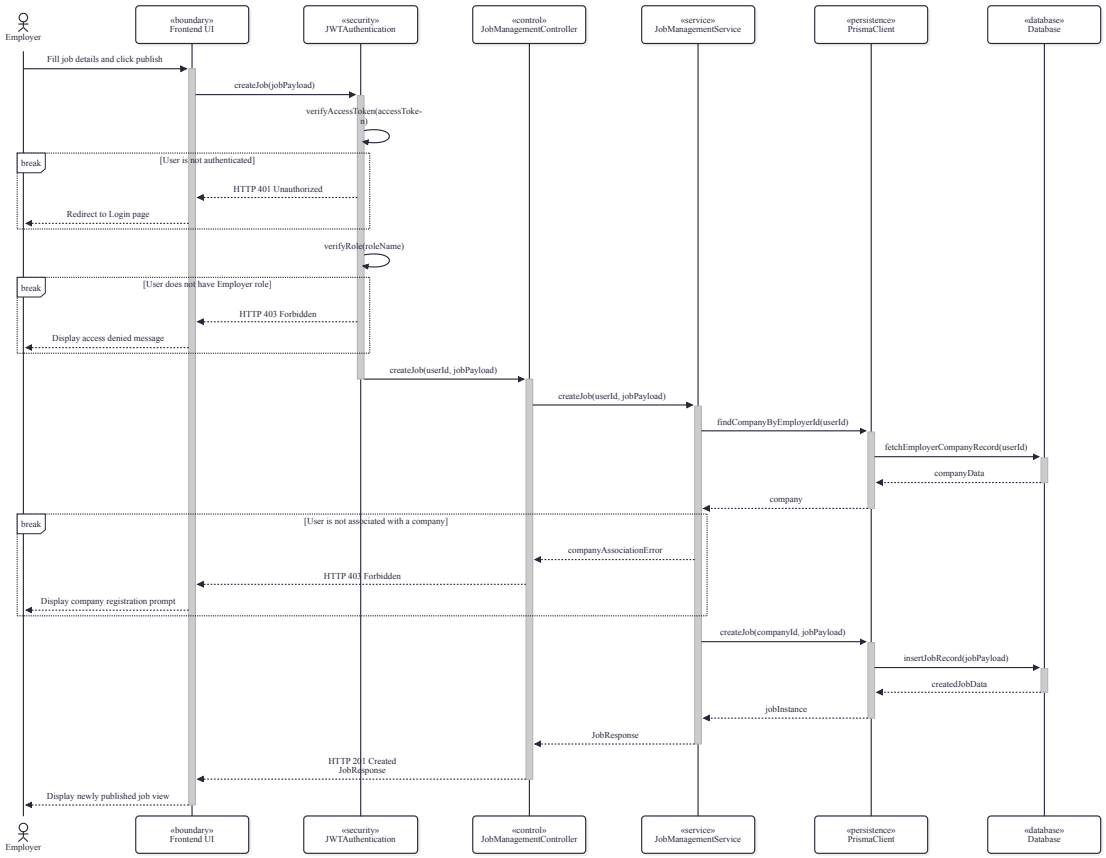


Figure D.5: Sequence Diagram: Create Job Posting.

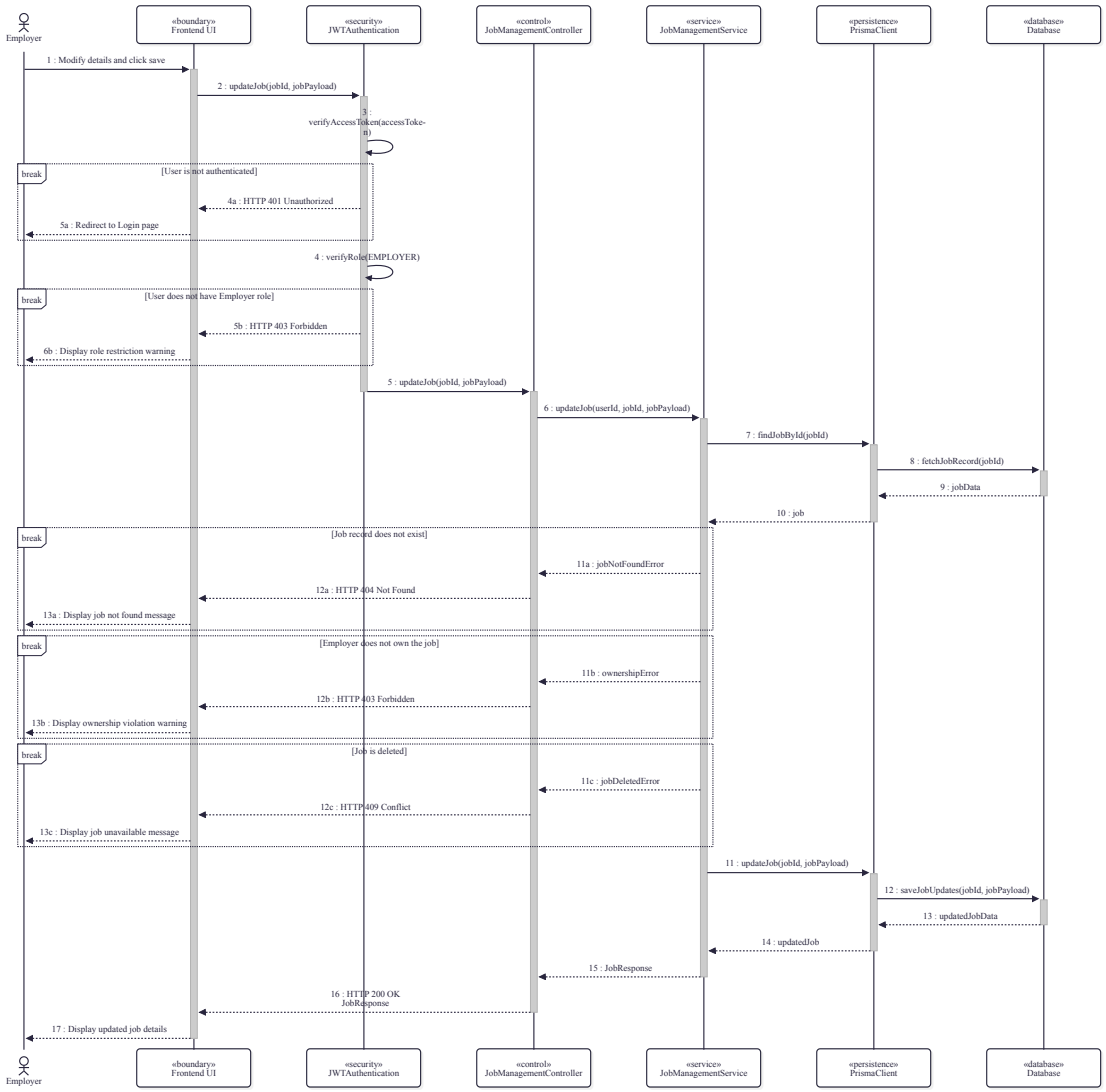


Figure D.6: Sequence Diagram: Edit Job Posting.

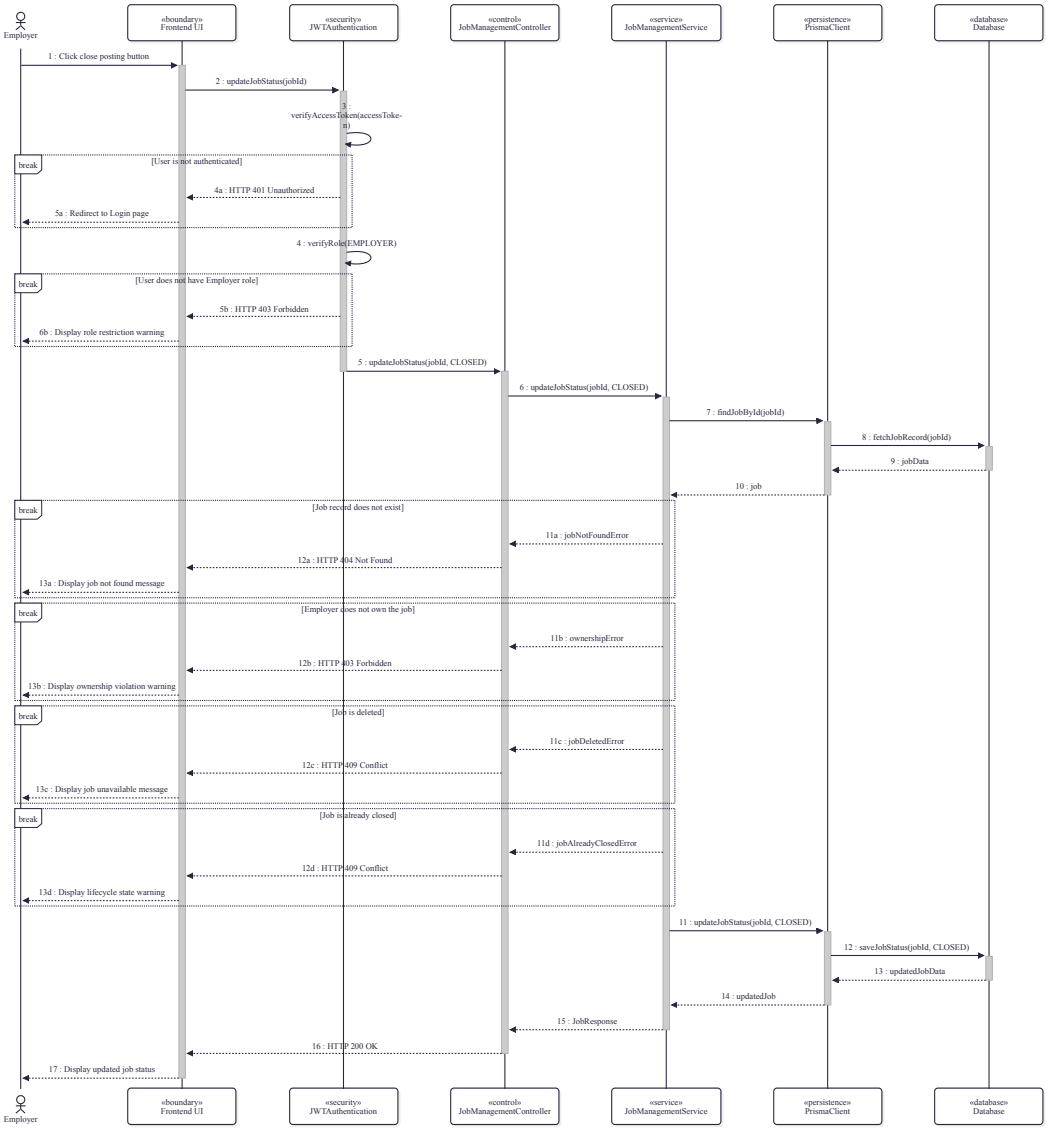


Figure D.7: Sequence Diagram: Close Job Posting.

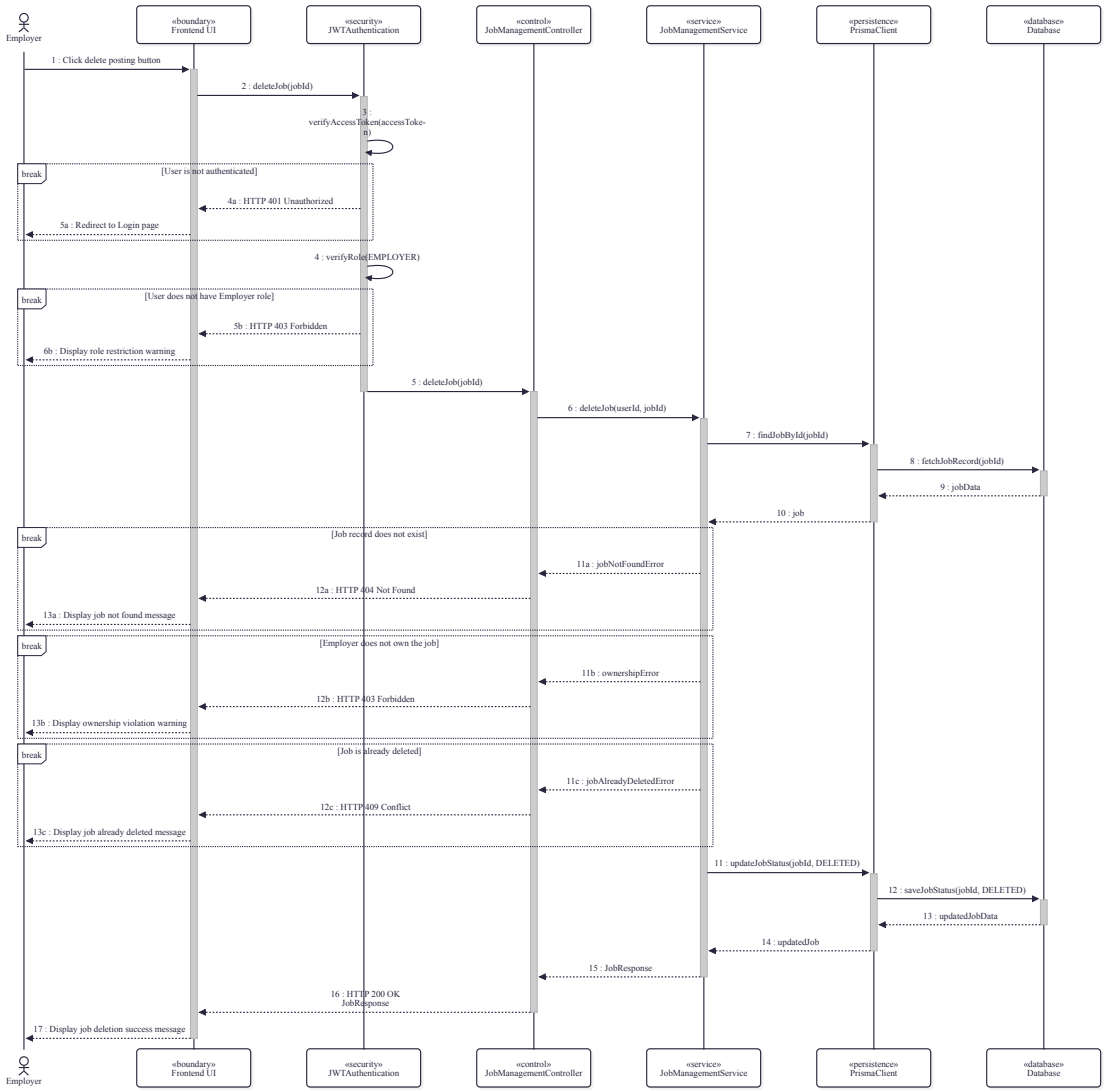


Figure D.8: Sequence Diagram: Delete Job Posting.

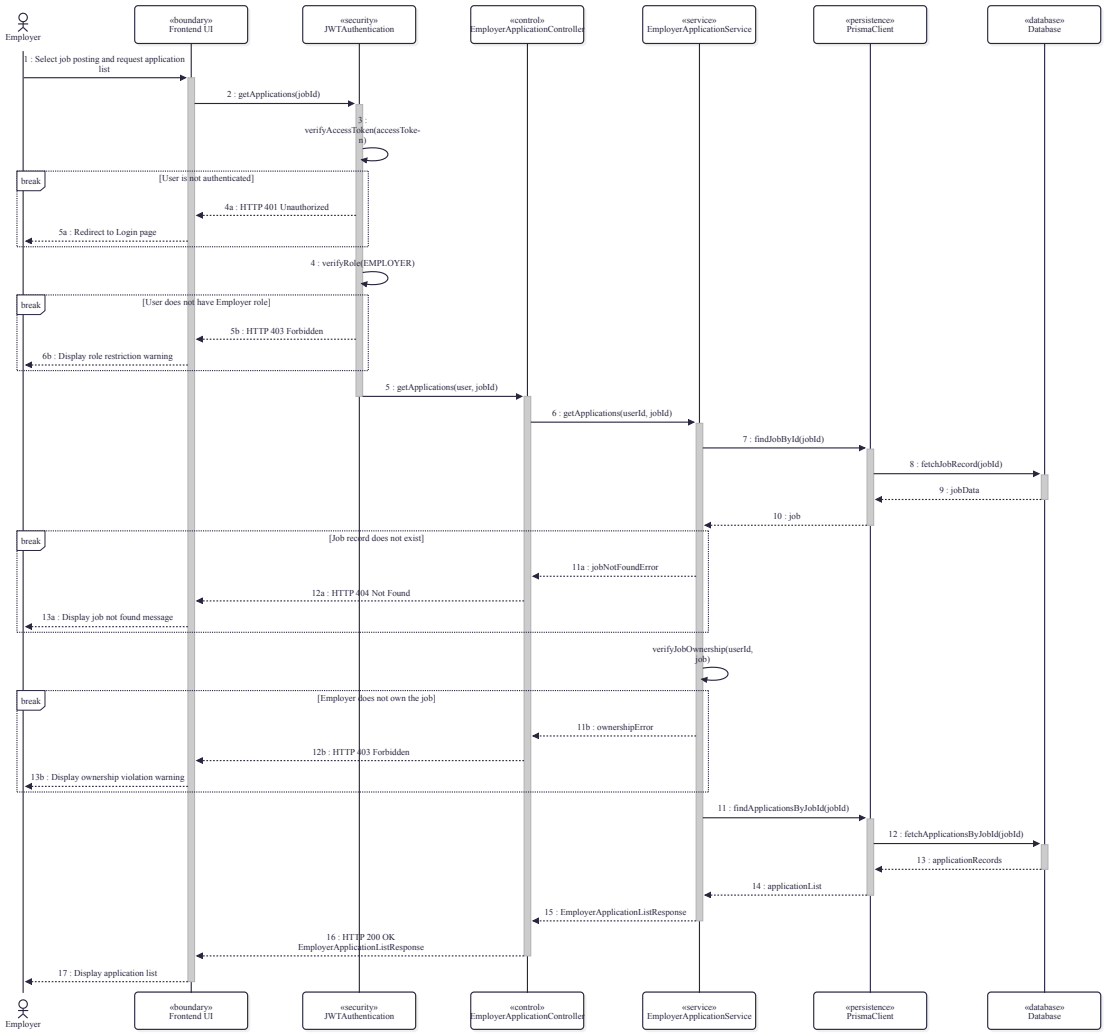


Figure D.9: Sequence Diagram: Review Application.

D.3 Authentication and Profile

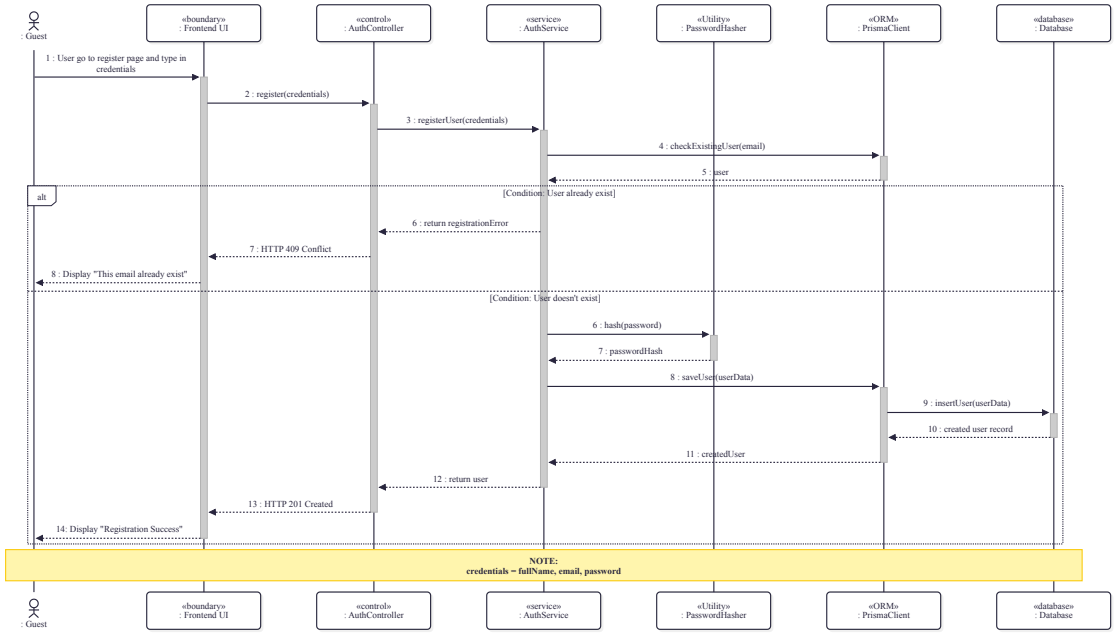


Figure D.10: Sequence Diagram: Register.

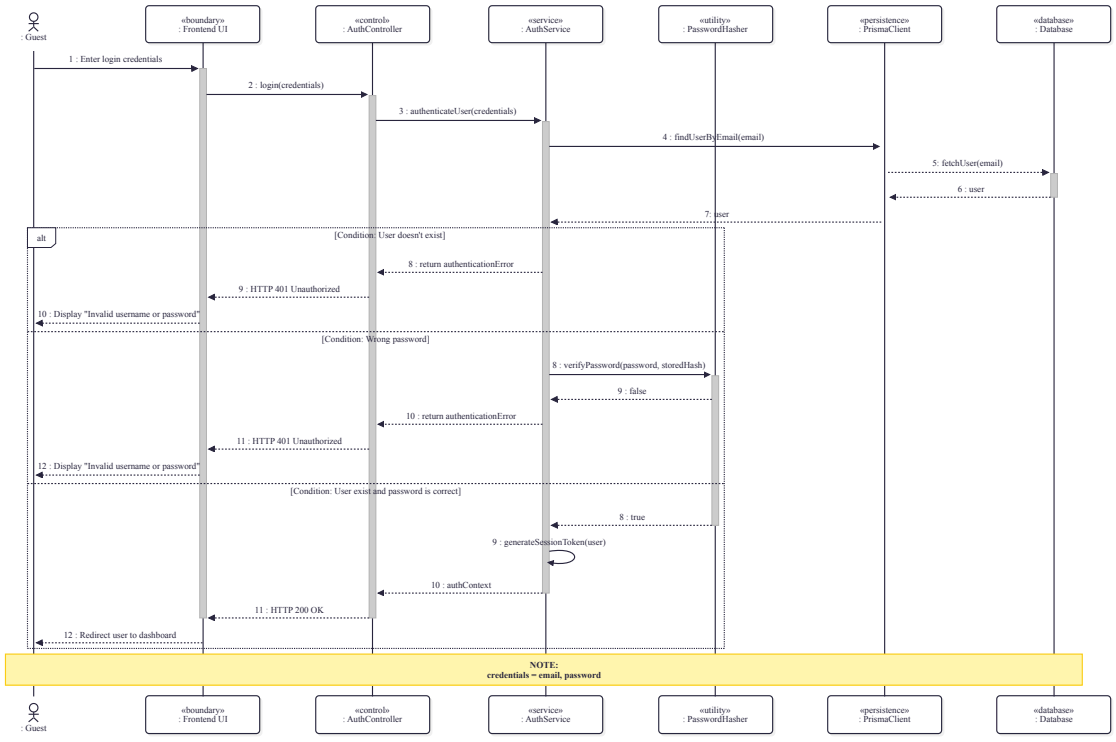


Figure D.11: Sequence Diagram: Login.

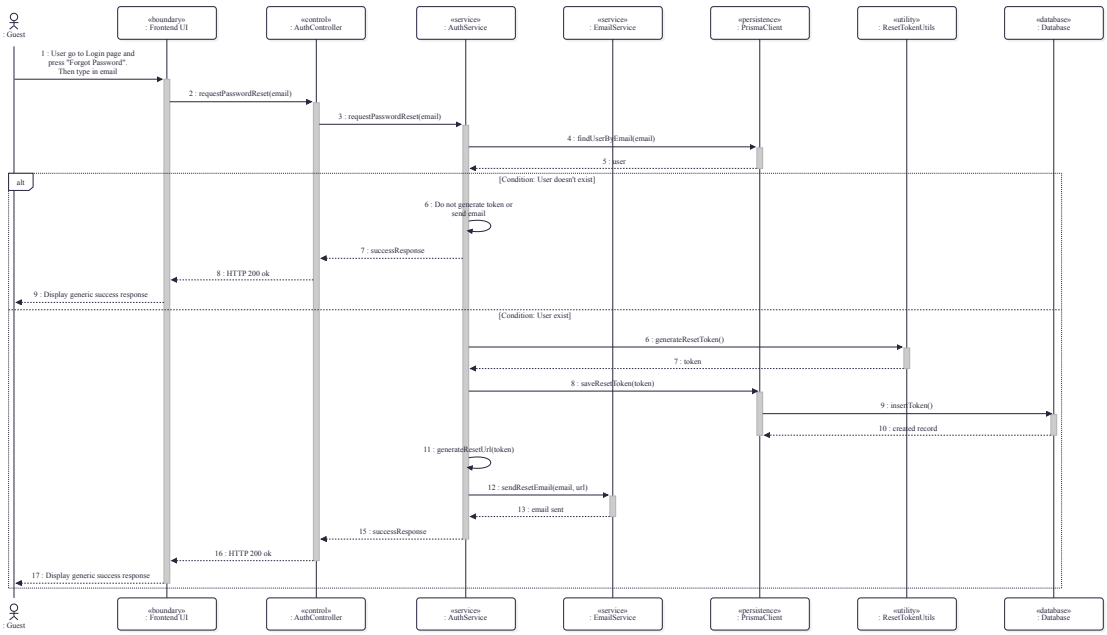


Figure D.12: Sequence Diagram: Forgot Password.

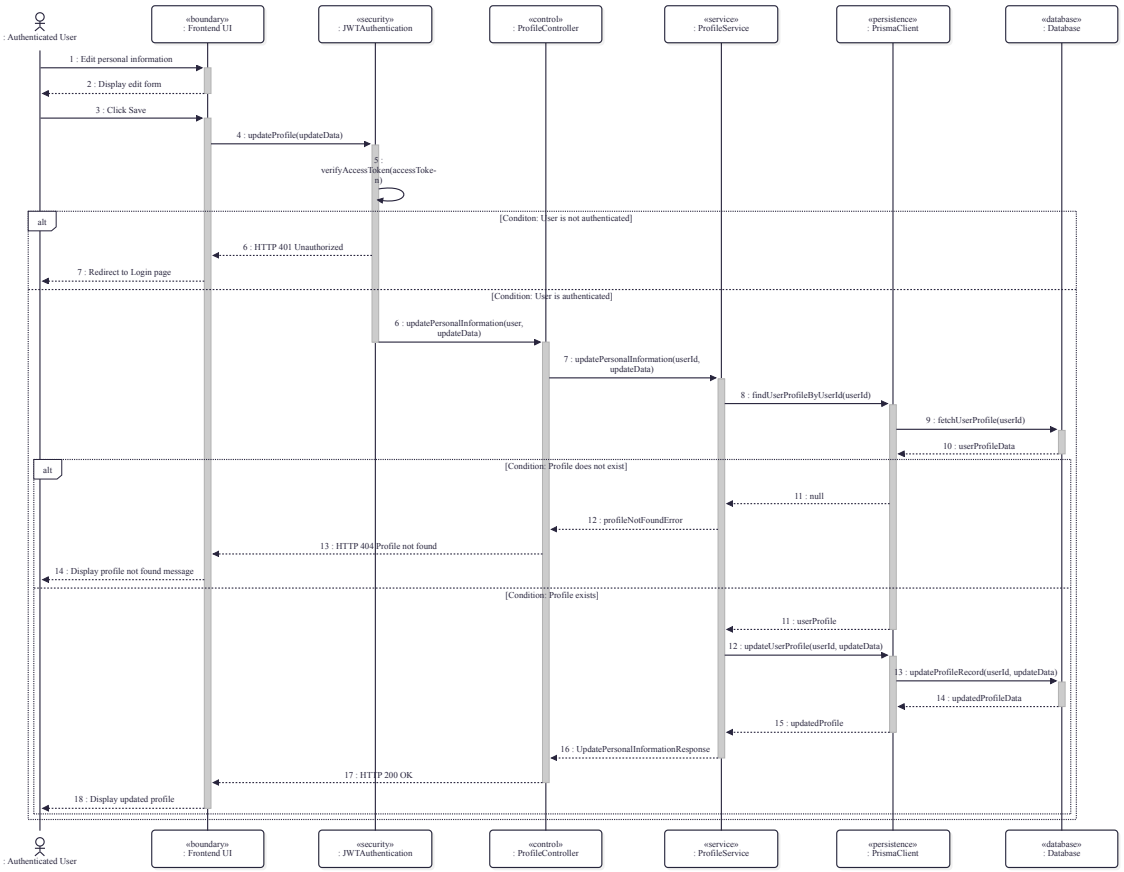


Figure D.13: Sequence Diagram: Update Personal Information.

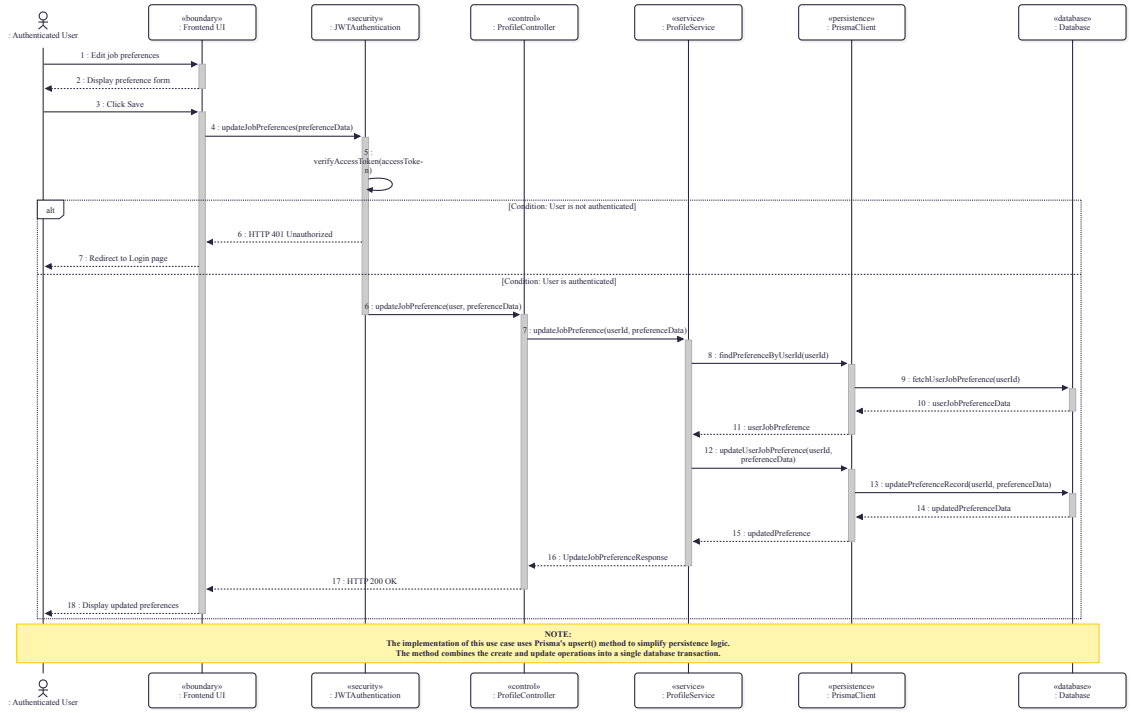


Figure D.14: Sequence Diagram: Update Job Preferences.

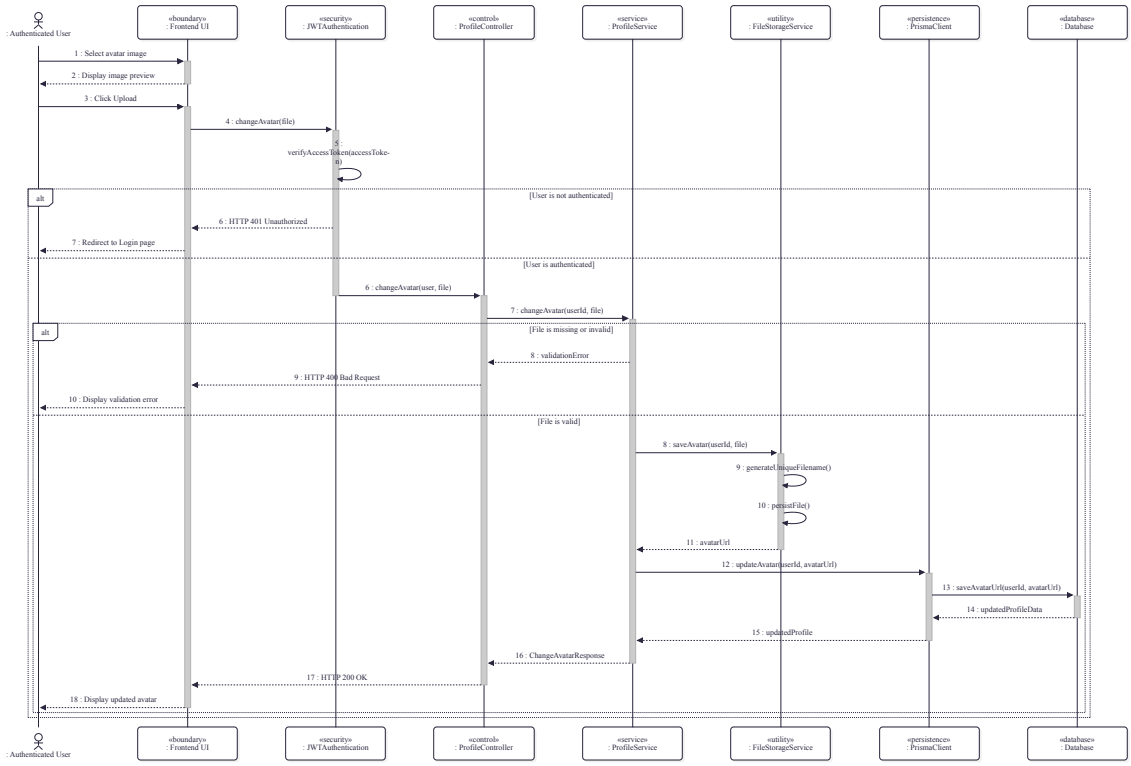


Figure D.15: Sequence Diagram: Change Avatar.

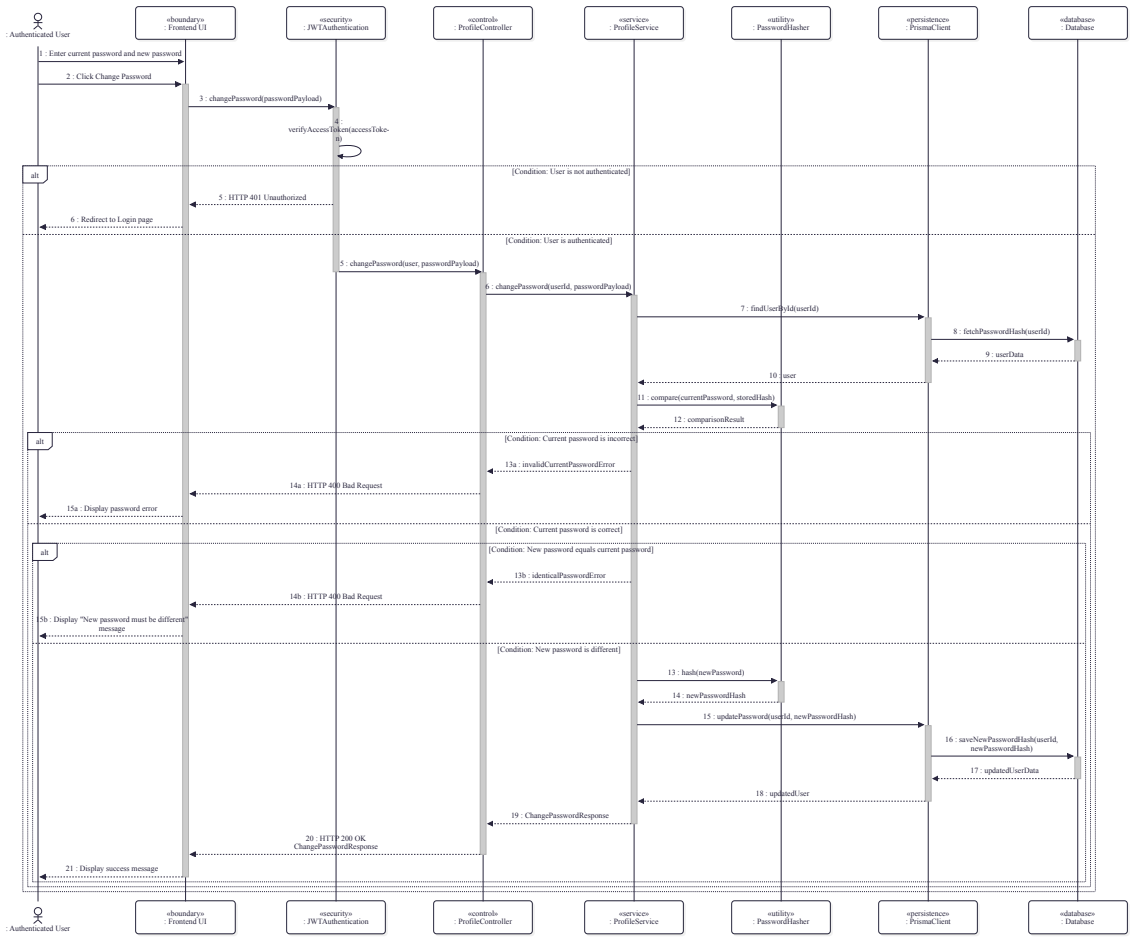


Figure D.16: Sequence Diagram: Change Password.

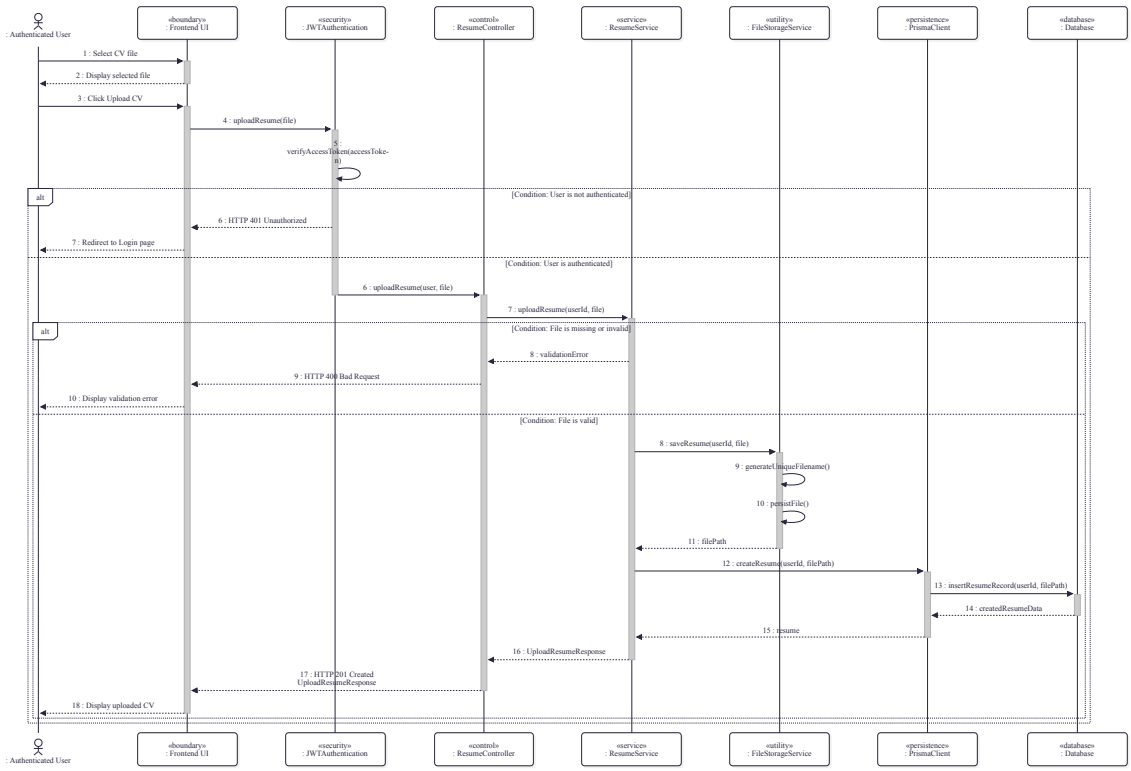


Figure D.17: Sequence Diagram: Upload CV.

D.4 Public

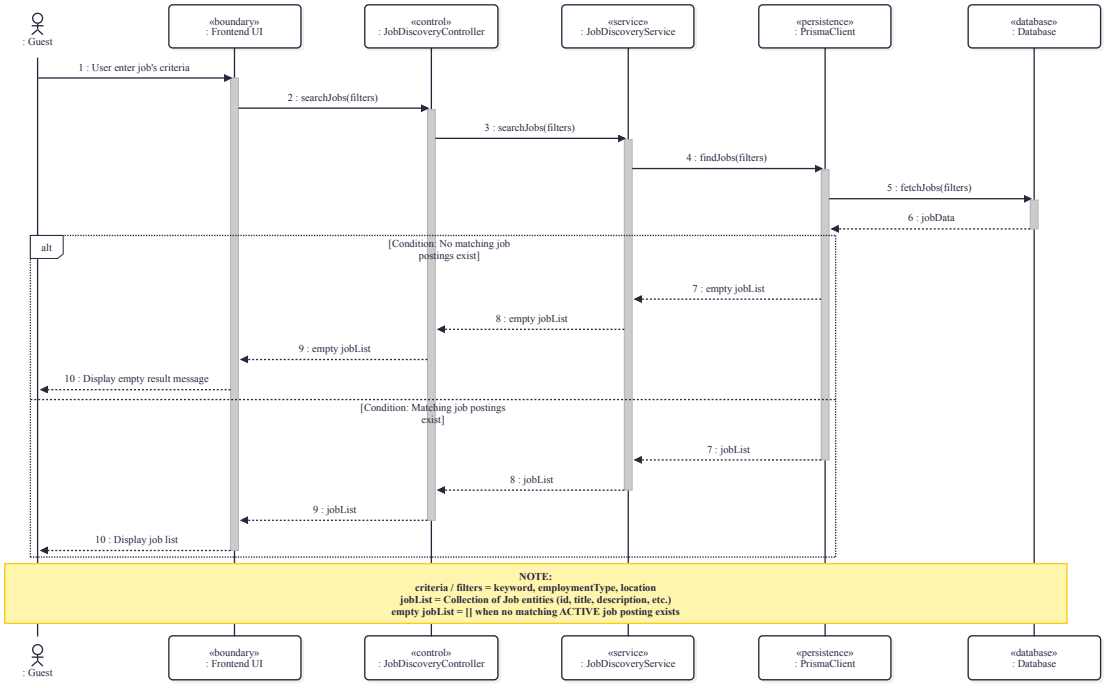


Figure D.18: Sequence Diagram: Search Jobs.

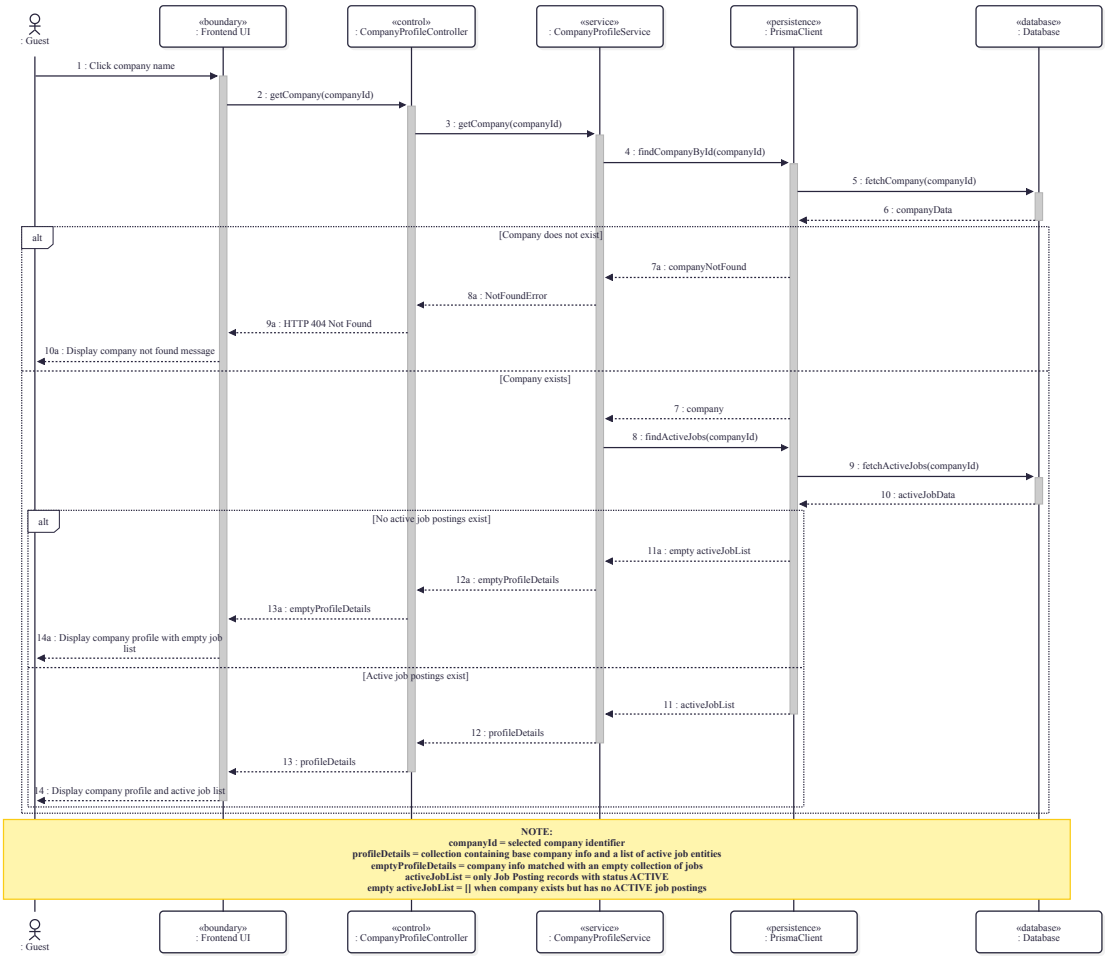


Figure D.19: Sequence Diagram: View Company Profile.

D.5 Back Office

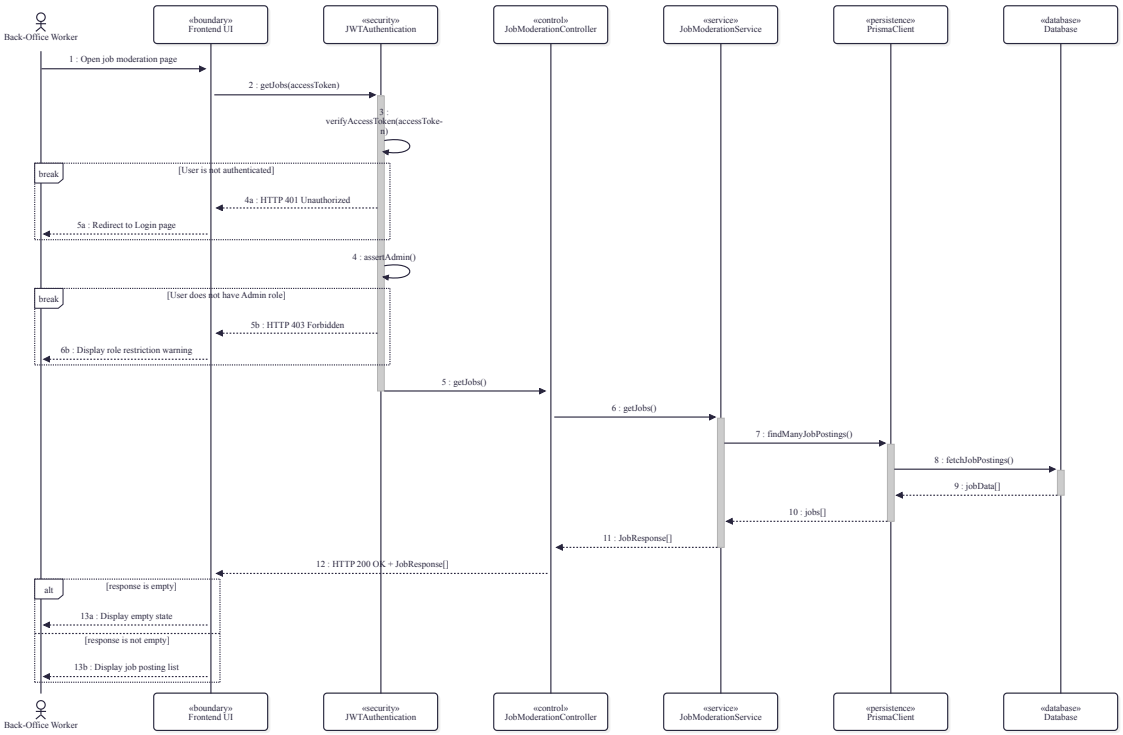


Figure D.20: Sequence Diagram: Get Job Postings List.

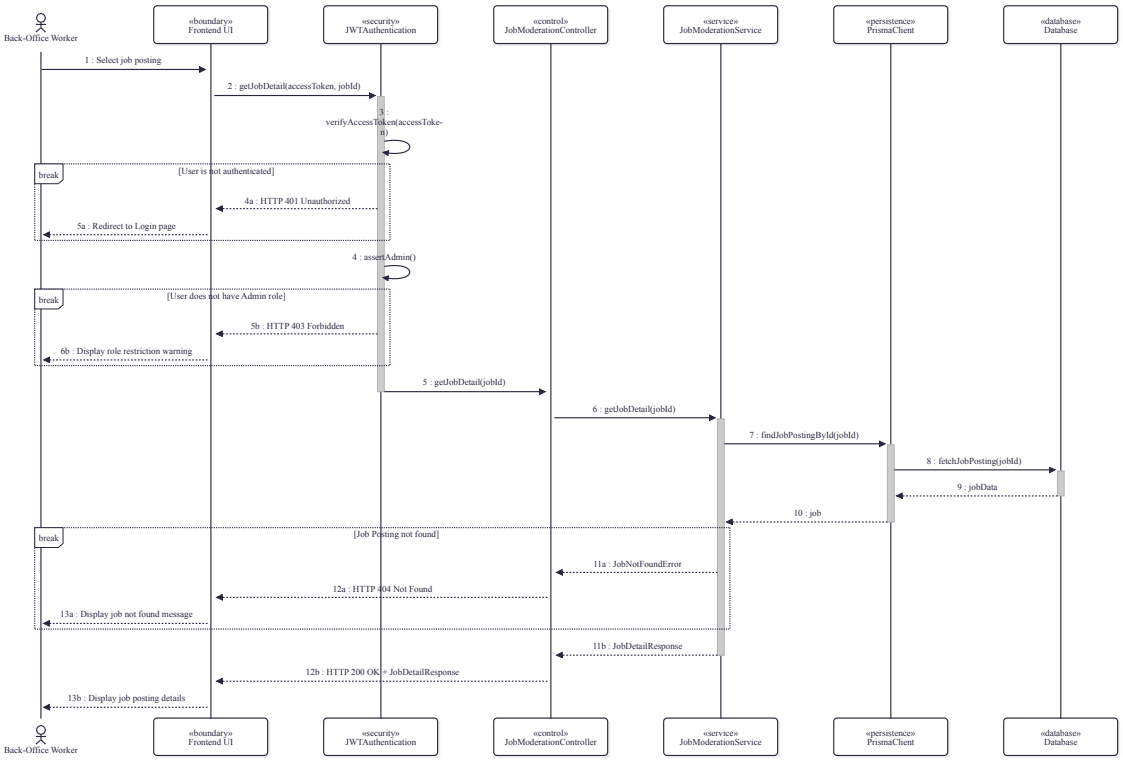


Figure D.21: Sequence Diagram: Get Job Posting Detail.



Figure D.22: Sequence Diagram: Approve Job.

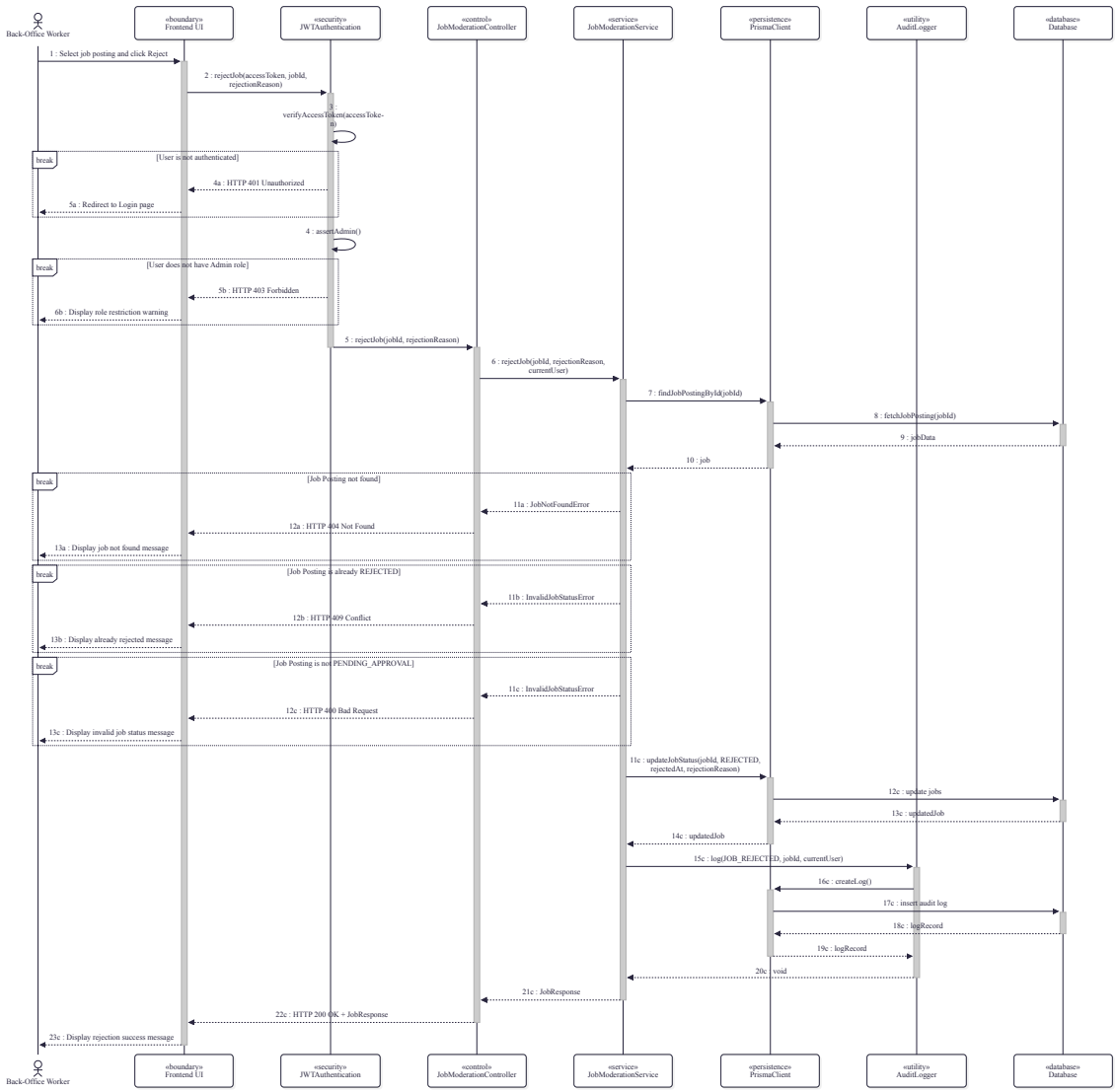


Figure D.23: Sequence Diagram: Reject Job.

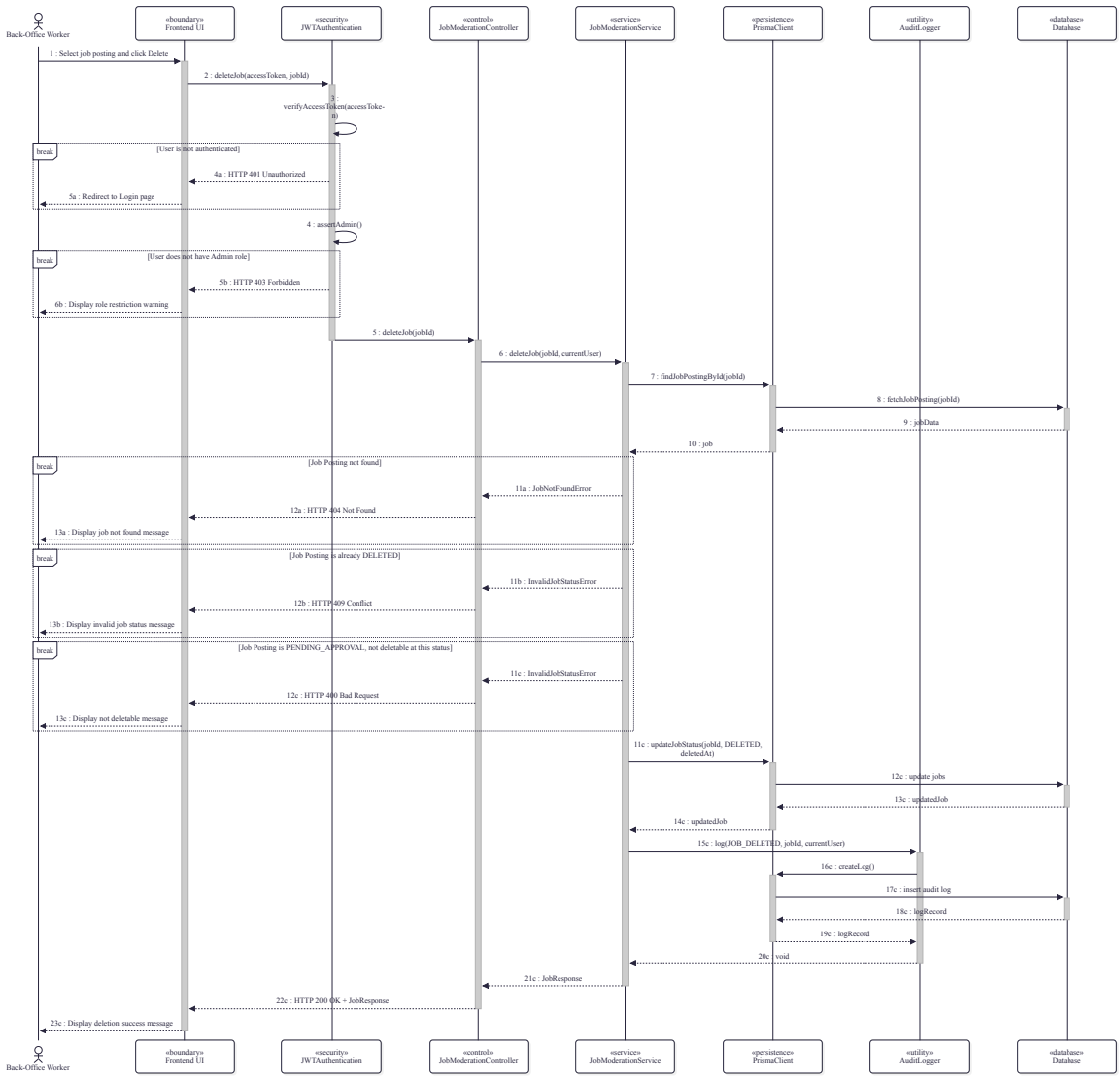


Figure D.24: Sequence Diagram: Delete Job.

Appendix E

Domain Enumerations

This chapter lists the domain enumerations referenced by name in the entity-relationship diagrams and the module designs. The complete value set of each enumeration is provided here for reference.

E.1 JobStatus

Referenced in Chapter 3 as the type of the job posting's `status` attribute (Figure 3.6).

Value	Description
PENDING_APPROVAL	A newly created or edited posting awaiting moderation approval.
ACTIVE	Approved by an administrator and publicly visible.
REJECTED	Rejected by an administrator during moderation.
CLOSED	Closed by the employer before the deadline.
EXPIRED	Past the application deadline; no longer accepting applications.
DELETED	Soft-deleted by the employer; the record is retained for history.

E.2 ApplicationStatus

Referenced in Chapter 3 as the type of the application's `status` attribute (Figure 3.7).

Value	Description
SUBMITTED	Initial state when a job seeker applies.
UNDER_REVIEW	The employer has acknowledged the application and is reviewing it.
ACCEPTED	The employer has accepted the application.
REJECTED	The employer has rejected the application.
WITHDRAWN	The job seeker has withdrawn the application.